

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN



SINH VIÊN THỰC HIỆN

ĐẶNG THÀNH TÀI
MSSV: 223209
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ THIẾT BỊ IOT THEO DÕI SỨC KHỎE BỆNH NHÂN NGƯỜI CAO
TUỔI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KỸ THUẬT PHẦN MỀM
Mã ngành: 7480103

Cần Thơ, tháng 5 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

HỌ TÊN TÁC GIẢ

ĐẶNG THÀNH TÀI
MSSV: 223209
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ THIẾT BỊ IOT THEO DÕI SỨC KHỎE BỆNH NHÂN NGƯỜI CAO
TUỔI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KỸ THUẬT PHẦN MỀM
Mã ngành: 7480103

GIẢNG VIÊN HƯỚNG DẪN
TS. TRẦN VĂN THIỆN

TÓM TẮT

Đề tài tập trung nghiên cứu, thiết kế và hiện thực một hệ thống IoT theo dõi sức khỏe người cao tuổi theo mô hình ba lớp gồm thiết bị ESP32, backend FastAPI và ứng dụng Flutter. Ở lớp thiết bị, hệ thống hỗ trợ thu thập các chỉ số sức khỏe cơ bản như nhịp tim, SpO₂, nhiệt độ, nhịp thở và dữ liệu ECG; đồng thời có khả năng gửi dữ liệu định kỳ hoặc theo yêu cầu về máy chủ trung tâm.

Ở lớp xử lý trung tâm, backend cung cấp REST API cho ứng dụng và thiết bị, triển khai cơ chế xác thực tách biệt giữa người dùng và thiết bị, quản lý phiên bằng JWT và refresh token, lưu trữ dữ liệu trong MongoDB, áp dụng Redis cho rate limiting và hỗ trợ sinh cảnh báo theo ngưỡng cấu hình. Ở lớp người dùng, ứng dụng Flutter thực hiện đăng ký, đăng nhập, đồng bộ thiết bị, xem dữ liệu latest/history/alerts/ECG và gửi yêu cầu ECG on-demand thông qua backend.

Kết quả thực hiện cho thấy hệ thống đã đạt mức nguyên mẫu tích hợp tương đối hoàn chỉnh, có thể triển khai bằng Docker Compose và vận hành được chuỗi xử lý từ thiết bị đến backend và ứng dụng. Báo cáo được hoàn thiện theo hướng học thuật hơn thông qua việc chuẩn hóa văn phong, bổ sung phần tóm tắt, ma trận kiểm thử và ma trận đánh giá mức độ hoàn thành mục tiêu. Bên cạnh các kết quả đã đạt được, đề tài vẫn còn những giới hạn như command queue hiện sử dụng Mongo polling, hạ tầng observability chưa ở mức production đầy đủ và cần mở rộng thêm kiểm thử end-to-end ở các giai đoạn phát triển tiếp theo.

Từ khóa: *IoT; theo dõi sức khỏe từ xa; ESP32; FastAPI; Flutter; MongoDB; ECG on-demand.*

ABSTRACT

This thesis presents the design and implementation of an IoT-based remote health monitoring system for elderly patients. The proposed architecture consists of three main layers: an ESP32-based device layer, a FastAPI backend, and a Flutter application for end users. The device layer is responsible for collecting core health indicators such as heart rate, SpO₂, body temperature, respiratory rate, and ECG-related data, then sending them to the server through a structured communication flow.

At the backend layer, the system provides REST APIs for both user applications and IoT devices, separates user authentication from device authentication, manages sessions through JWT access tokens and refresh tokens, stores operational data in MongoDB, and uses Redis for rate limiting. The Flutter application supports user registration, login, session recovery, device synchronization, visualization of latest and historical readings, alert monitoring, and ECG on-demand requests.

The implementation results indicate that the system has reached the level of a well-integrated prototype that can be deployed with Docker Compose and can coordinate the end-to-end flow from device ingestion to backend processing and user interaction. The revised report also standardizes the testing appendix and the objective-completion matrix to improve academic presentation quality. Current limitations mainly include the MongoDB-based polling command queue, the absence of a full production-grade observability stack, and the need for broader end-to-end integration testing in future work.

Keywords: *IoT; remote health monitoring; ESP32; FastAPI; Flutter; MongoDB; ECG on-demand.*

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ đã tận tình giảng dạy, truyền đạt cho em những kiến thức quý báu trong suốt quá trình học tập. Những nền tảng chuyên môn mà em tiếp nhận được từ quý Thầy Cô là cơ sở quan trọng để em thực hiện và hoàn thiện đề tài này.

Em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn, TS. Trần Văn Thiện, người đã tận tình định hướng, góp ý, hỗ trợ chuyên môn và đồng hành cùng em trong suốt quá trình thực hiện đồ án tốt nghiệp. Sự nghiêm túc trong học thuật và những nhận xét sát thực tế của thầy đã giúp em hoàn thiện báo cáo theo hướng rõ ràng, chặt chẽ và bám sát sản phẩm.

Do thời gian thực hiện, phạm vi đề tài và kinh nghiệm thực tế của bản thân còn hạn chế, báo cáo chắc chắn vẫn còn những thiếu sót nhất định. Em rất mong tiếp tục nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô để có thể bổ sung kiến thức, hoàn thiện kỹ năng và phát triển đề tài tốt hơn trong thời gian tới.

Cuối lời, em xin kính chúc quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ thật nhiều sức khỏe, hạnh phúc và thành công trong công tác giảng dạy và nghiên cứu.

Em xin chân thành cảm ơn!

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

ĐẶNG THÀNH TÀI

LỜI CAM ĐOAN

Em xin cam đoan báo cáo này là kết quả nghiên cứu, phân tích, thiết kế, triển khai và tổng hợp do chính em thực hiện dưới sự hướng dẫn của giảng viên phụ trách. Các nội dung tham khảo từ tài liệu, chuẩn kỹ thuật, công trình nghiên cứu, tài liệu hướng dẫn và mã nguồn liên quan đều đã được trích dẫn trong phần tài liệu tham khảo. Em chịu trách nhiệm về tính trung thực của nội dung báo cáo.

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

ĐẶNG THÀNH TÀI

[illegible]

GIẢNG VIÊN HƯỚNG DẪN

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

GIẢNG VIÊN PHẢN BIỆN

MỤC LỤC

CHƯƠNG 1 MỞ ĐẦU.....	1
1.1 Lý do chọn đề tài.....	1
1.2 Mục tiêu nghiên cứu.....	2
1.2.1 Mục tiêu tổng quát.....	2
1.2.2 Mục tiêu cụ thể.....	2
1.2.3 Kết quả mong đợi.....	3
1.3 Đối tượng nghiên cứu.....	4
1.4 Phạm vi nghiên cứu.....	5
1.4.1 Phạm vi chức năng.....	6
1.4.2 Phạm vi công nghệ.....	7
1.4.3 Phạm vi người dùng.....	8
1.4.4 Phạm vi dữ liệu.....	8
1.4.5 Những nội dung ngoài phạm vi đề tài.....	9
1.5 Phương pháp nghiên cứu.....	9
1.5.1 Phương pháp nghiên cứu lý thuyết.....	9
1.5.2 Phương pháp phân tích yêu cầu.....	10
1.5.3 Phương pháp thiết kế hệ thống.....	10
1.5.4 Phương pháp triển khai thực nghiệm.....	10
1.5.5 Phương pháp kiểm thử và đánh giá.....	11
1.6 Ý nghĩa khoa học và thực tiễn của đề tài.....	11
1.6.1 Ý nghĩa khoa học.....	11
1.6.2 Ý nghĩa thực tiễn.....	12
1.6.3 Ý nghĩa đối với học tập và phát triển kỹ năng.....	12
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	13
2.1 Tổng quan về hệ thống theo dõi sức khỏe từ xa.....	13
2.2 Internet of Things trong theo dõi sức khỏe.....	14
2.3 Các chỉ số sức khỏe được sử dụng trong hệ thống.....	14
2.3.1 Nhịp tim.....	15
2.3.2 Nồng độ oxy trong máu.....	15
2.3.3 Nhiệt độ cơ thể.....	15
2.3.4 Nhịp thở.....	15
2.3.5 Điện tim.....	15
2.3.6 Vai trò của các chỉ số trong hệ thống.....	15

2.4 Kiến trúc client-server và REST API	16
2.4.1 Khái niệm client-server.....	16
2.4.2 REST API.....	16
2.5 Xác thực và quản lý phiên bằng JWT, refresh token	17
2.5.1 JWT trong hệ thống	17
2.5.2 Refresh token.....	18
2.5.3 Quản lý session ở phía Flutter.....	18
2.5.4 Ý nghĩa của mô hình xác thực hiện tại	19
2.6 Xác thực thiết bị bằng Device Token	19
2.7 Phân quyền RBAC trong hệ thống.....	20
2.8 FastAPI và lý do lựa chọn.....	20
2.8.1 Tổng quan về FastAPI	20
2.8.2 Vai trò của service layer.....	21
2.9 MongoDB và lý do lựa chọn.....	21
2.9.1 Tổng quan về MongoDB.....	21
2.9.2 Tổ chức dữ liệu trong MongoDB	21
2.9.3 Index và TTL.....	21
2.10 Redis và vai trò trong hệ thống	21
2.11 Docker và Docker Compose	21
2.11.1 Docker	21
2.11.2 Docker Compose	22
2.11.3 Vai trò trong đề tài.....	22
2.11.4 Cloudflare Tunnel và Nginx.....	22
2.12 Flutter và lý do lựa chọn	22
2.12.1 Tổng quan về Flutter.....	22
2.12.2 Vai trò của Flutter trong hệ thống	22
2.12.3 Kiến trúc ứng dụng Flutter, quản lý trạng thái.....	22
2.13 Cơ chế command queue cho ECG on-demand	22
2.14 Các nguyên tắc bảo mật cơ bản được áp dụng.....	23
2.15 Logging, health check và giám sát hệ thống.....	23
2.16 Nghiên cứu liên quan.....	23
2.16.1 Nhóm hệ thống chi phí thấp dựa trên ESP32	23
2.16.2 Nhóm hệ thống RPM tích hợp cloud/edge và AI.....	23
2.16.3 Nhóm hệ thống tập trung vào trực quan hóa và trải nghiệm giám sát	24
2.17 Bảng so sánh đề tài với các hệ thống liên quan	24

2.18 Khoảng trống nghiên cứu và định hướng đóng góp của đề tài	25
CHƯƠNG 3 PHÂN TÍCH YÊU CẦU, THIẾT KẾ VÀ KIẾN TRÚC HỆ THỐNG	27
3.1 Mục tiêu và định hướng của chương.....	27
3.2 Bối cảnh bài toán và yêu cầu hệ thống.....	27
3.2.1 Yêu cầu chức năng của hệ thống.....	28
3.2.2 Yêu cầu phi chức năng của hệ thống.....	29
3.2.3 Tác nhân tham gia trong hệ thống	30
3.3 Kiến trúc tổng thể và nguyên tắc phân lớp	32
3.3.1 Kiến trúc giao tiếp giữa các thành phần	33
3.3.2 Activity tổng quát của hệ thống	34
3.3.3 Sequence tổng quát giữa ứng dụng, backend và thiết bị	35
3.4 Thiết kế tổng thể phần cứng thiết bị Eldercare	36
3.4.1 Kiến trúc phần cứng hai bo	37
3.4.2 Lựa chọn mô-đun và lý do kỹ thuật.....	39
3.4.3 Sơ đồ kết nối nguyên lý bo ESP32-C3	40
3.4.4 Sơ đồ kết nối nguyên lý bo ESP32-S3.....	42
3.4.5 Nguồn cấp, chống nhiễu và nguyên tắc lắp ráp	44
3.4.6 Thiết kế PCB, bố trí linh kiện và minh chứng phần cứng	45
3.5 Thiết kế tổng thể backend và lớp dữ liệu.....	47
3.5.1 Nhóm API và cơ chế xác thực.....	47
3.5.2 Pipeline xử lý readings, cảnh báo và ECG on-demand	49
3.5.3 Thiết kế dữ liệu MongoDB và Redis.....	49
3.5.4 Quan sát vận hành và độ sẵn sàng của backend.....	51
3.6 Thiết kế tổng thể ứng dụng Flutter	52
3.6.1 Kiến trúc provider, service và storage	52
3.6.2 Quản lý phiên, device scope và trạng thái lỗi	53
3.6.3 Tổ chức giao diện và luồng sử dụng chính	54
3.7 Thiết kế tích hợp phần cứng – phần mềm.....	60
3.7.1 Chuỗi vận hành từ đo lường đến hiển thị.....	60
3.7.2 Đồng bộ trạng thái, lỗi và cơ chế phục hồi	61
3.7.3 Phần cứng và phần mềm trong thiết kế.....	61
3.7.4 Phân vai tài nguyên.....	62
3.7.5 Dữ liệu và giao tiếp.....	62
3.7.6 Khả năng mở rộng tương lai	63

3.7.7	bối cảnh sử dụng cho người cao tuổi.....	63
3.8	Đánh giá phương án thiết kế	64
3.9	Các kịch bản vận hành điển hình và đối chiếu với kiến trúc hệ thống.....	65
3.9.1	Kịch bản vận hành bình thường của hệ thống.....	65
3.9.2	Kịch bản gián đoạn BLE giữa bo C3 và bo S3	66
3.9.3	Kịch bản mất Wi-Fi hoặc backend chưa sẵn sàng.....	66
3.9.4	Kịch bản phiên ứng dụng hết hạn và cơ chế phục hồi session.....	67
3.9.5	Kịch bản readings trùng lặp và bảo đảm tính đúng đắn dữ liệu.....	67
3.9.6	Kịch bản thay thế mô-đun, bảo trì và mở rộng thiết bị	68
3.10	Phân tích bổ sung về vận hành, tính khả dụng và khả năng kế thừa.....	69
3.10.1	Khả năng vận hành và chạy thử theo từng lớp.....	69
3.10.2	Tính khả dụng và mức độ thân thiện với người dùng	69
3.10.3	Bảo trì, thay thế mô-đun và giá trị kế thừa	70
3.10.4	Mối liên hệ giữa phần cứng thật và contract phần mềm.....	70
3.10.5	Định hướng mở rộng thành hệ thống nhiều thiết bị hoặc nhiều người theo dõi	71
CHƯƠNG 4 TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG		74
4.1	Môi trường triển khai hệ thống	74
4.1.1	Môi trường chạy backend	74
4.1.2	Thành phần hạ tầng đi kèm	74
4.1.3	Các lệnh triển khai cơ bản.....	75
4.2	Cấu hình và khởi tạo backend.....	75
4.2.1	Biến môi trường cần thiết.....	75
4.2.2	Trình tự khởi động hệ thống.....	75
4.2.3	Kiểm tra sau triển khai.....	76
4.3	Triển khai và kiểm tra lớp dữ liệu MongoDB – Redis.....	76
4.3.1	MongoDB trong môi trường triển khai.....	76
4.3.2	Redis trong môi trường triển khai	76
4.3.3	Kiểm tra kết nối backend với MongoDB và Redis	77
4.4	Triển khai và kiểm thử API backend.....	77
4.4.1	Nhóm API chính được triển khai	77
4.4.2	Kiểm thử health, live, ready và metrics	77
4.4.3	Kiểm thử nhóm API ESP	77
4.5	Triển khai và kiểm thử lớp thiết bị ESP32	77
4.5.1	Kiến trúc triển khai thiết bị thực tế.....	78

4.5.2 Chức năng của bo gateway ESP32-S3.....	78
4.5.3 Vai trò của bo đo sinh hiệu ESP32-C3	78
4.5.4 Kiểm thử thu thập và tổng hợp dữ liệu trên thiết bị	78
4.5.5 Kiểm thử gửi readings lên backend	79
4.5.6 Kiểm thử đồng bộ thời gian và trạng thái mạng.....	79
4.5.7 Giới hạn hiện tại của lớp thiết bị	79
4.6 Kiểm thử và đánh giá ứng dụng Flutter	79
4.7 Kiểm thử thực nghiệm và tiêu chí đánh giá định lượng	80
4.7.1 Nguyên tắc xây dựng bộ kiểm thử thực nghiệm	80
4.7.2 Bộ chỉ số và ngưỡng nghiệm thu ở mức nguyên mẫu	80
4.8 Kết quả tổng hợp kiểm thử thực nghiệm trên nguyên mẫu	81
4.8.1 Kết quả theo nhóm chức năng.....	81
4.8.2 Kết quả đối chiếu mục tiêu đề tài	82
4.8.3 Nhận xét về bộ số liệu thực nghiệm	83
4.9 Đánh giá mức độ hoàn thiện của hệ thống.....	83
4.10 Hướng cải tiến sau triển khai	84
4.11 Kết luận chương	84
CHƯƠNG 5 KẾT LUẬN VÀ KIẾN NGHỊ.....	85
5.1 Kết quả đạt được của đề tài.....	85
5.1.1 Xây dựng được hệ thống theo dõi sức khỏe từ xa hoàn chỉnh ở mức nguyên mẫu tích hợp	85
5.1.2 Xây dựng được backend FastAPI có chức năng tương đối đầy đủ.....	85
5.1.3 Thiết kế được cơ sở dữ liệu MongoDB phù hợp với bài toán	86
5.1.4 Xây dựng được ứng dụng Flutter đóng vai trò client chính cho hệ thống.....	86
5.1.5 Hoàn thiện được lớp thiết bị ở mức phần cứng – firmware thực nghiệm ..	87
5.2 Đánh giá mức độ hoàn thành mục tiêu đề tài.....	87
5.2.1 Mức độ hoàn thành về mặt kiến trúc	87
5.2.2 Mức độ hoàn thành về mặt chức năng.....	87
5.2.3 Mức độ hoàn thành về mặt triển khai và kiểm thử.....	88
5.3 Những điểm mạnh nổi bật của hệ thống.....	88
5.3.1 Kiến trúc rõ ràng, bám sát bài toán thực tế	88
5.3.2 Có lớp bảo mật và kiểm soát truy cập tương đối tốt	88
5.3.3 Có command queue ECG on-demand mang tính đặc trưng	88
5.3.4 Có nền tảng vận hành khá tốt ở mức nguyên mẫu	88
5.3.5 Lớp thiết bị đã có giá trị thực nghiệm rõ rệt	89

5.4 Hạn chế hiện tại của hệ thống	89
5.4.1 Command queue vẫn dùng Mongo polling.....	89
5.4.2 Quan hệ thiết bị – backend – app đã mạnh, nhưng pipeline ECG waveform thật vẫn còn dư địa hoàn thiện thêm ở mức end-to-end	89
5.4.3 Hạ tầng observability chưa phải mức production đầy đủ	89
5.4.4 Integration test end-to-end còn có thể mở rộng thêm.....	89
5.5 Bài học kinh nghiệm rút ra từ đề tài	90
5.5.1 Tính thống nhất giữa báo cáo và mã nguồn là yếu tố quyết định	90
5.5.2 Hệ thống nhiều lớp đòi hỏi tư duy tích hợp	90
5.5.3 Quan sát và vận hành quan trọng không kém chức năng nghiệp vụ	90
5.6 Kiến nghị và hướng phát triển.....	90
5.6.1 Chuẩn hóa sâu hơn luồng ECG waveform thật theo hướng end-to-end ...	90
5.6.2 Nâng cấp command queue khi quy mô tăng	90
5.6.3 Mở rộng kiểm thử end-to-end	91
5.6.4 Hoàn thiện observability và dashboard vận hành.....	91
5.6.5 Mở rộng tính năng quản trị và bảo mật	91
5.7 Kết luận chương	91
TÀI LIỆU THAM KHẢO	92
PHỤ LỤC A. TỔNG HỢP CÔNG NGHỆ SỬ DỤNG.....	94
PHỤ LỤC B. TỔNG HỢP NHÓM API CHÍNH CỦA HỆ THỐNG.....	95
PHỤ LỤC C. TỔNG HỢP COLLECTION DỮ LIỆU CHÍNH.....	96
PHỤ LỤC D. MA TRẬN CA KIỂM THỬ CHỨC NĂNG TRONG TÂM	97
PHỤ LỤC E. MA TRẬN ĐÁNH GIÁ MỨC ĐỘ HOÀN THÀNH MỤC TIÊU	99
PHỤ LỤC F. CHUẨN HÓA QUY TRÌNH LẮP RÁP VÀ TRIỂN KHAI.....	100

DANH SÁCH BẢNG

<i>Bảng 2.1. So sánh một số hệ thống theo dõi sức khỏe từ xa tiêu biểu với đề tài.....</i>	<i>24</i>
<i>Bảng 3.1. Ma trận yêu cầu chức năng theo thành phần.</i>	<i>28</i>
<i>Bảng 3.2. Yêu cầu phi chức năng của hệ thống.....</i>	<i>29</i>
<i>Bảng 3.3. Tác nhân chính và phạm vi tương tác.</i>	<i>30</i>
<i>Bảng 3.4. Ma trận giao tiếp và xác thực giữa các thành phần.</i>	<i>33</i>
<i>Bảng 3.5. Các khối phần cứng chính và vai trò trong hệ thống.....</i>	<i>38</i>
<i>Bảng 3.6. Lý do lựa chọn các mô-đun phần cứng chính.</i>	<i>40</i>
<i>Bảng 3.7. Bảng chân kết nối chính của bo ESP32-C3.....</i>	<i>42</i>
<i>Bảng 3.8. Bảng chân kết nối chính của bo ESP32-S3.</i>	<i>44</i>
<i>Bảng 3.9. Nhóm API và lớp xác thực tương ứng.</i>	<i>48</i>
<i>Bảng 3.10. Các collection dữ liệu chính của hệ thống.</i>	<i>51</i>
<i>Bảng 3.11. Vai trò của các provider chính trong ứng dụng Flutter.....</i>	<i>53</i>
<i>Bảng 3.12. Đối chiếu đóng góp của phần cứng và phần mềm trong kiến trúc Eldercare.....</i>	<i>64</i>
<i>Bảng 3.13. Đối chiếu các kịch bản vận hành với lớp chịu trách nhiệm chính.....</i>	<i>68</i>
<i>Bảng 3.14. Tổng hợp các điểm giúp hệ thống có giá trị kế thừa sau đồ án.....</i>	<i>73</i>
<i>Bảng 4.1. Bộ chỉ số đánh giá thực nghiệm và ngưỡng chấp nhận ở mức nguyên mẫu.</i>	<i>80</i>
<i>Bảng 4.2. Tổng hợp kết quả kiểm thử chức năng theo nhóm trên nguyên mẫu.</i>	<i>81</i>
<i>Bảng 4.3. Tổng hợp mức độ hoàn thành mục tiêu đề tài từ Phụ lục E.</i>	<i>82</i>
<i>Bảng D.1. Bộ ca kiểm thử chức năng trọng tâm của hệ thống.....</i>	<i>97</i>
<i>Bảng E.1. Ma trận đối chiếu mục tiêu và mức độ hoàn thành</i>	<i>99</i>
<i>Bảng F1. Checklist rút gọn để tái lập nguyên mẫu Eldercare.</i>	<i>100</i>
<i>Bảng F2. Các điểm cần chuẩn hóa để bảo đảm dữ liệu nhất quán giữa các tầng.</i>	<i>101</i>

DANH SÁCH HÌNH

Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống Eldercare.	31
Hình 3.2. Kiến trúc tổng thể phần cứng – phần mềm của hệ thống Eldercare.....	32
Hình 3.3. Kiến trúc phân lớp của hệ thống Eldercare.	33
Hình 3.4. Activity tổng quát của chuỗi vận hành hệ thống.	35
Hình 3.5. Sequence tổng quát giữa người dùng, ứng dụng, backend và thiết bị.	36
Hình 3.6. Kiến trúc phần cứng hai bo của thiết bị Eldercare.	38
Hình 3.7. Các mô-đun phần cứng chính sử dụng trong nguyên mẫu Eldercare.....	39
Hình 3.8. Sơ đồ kết nối nguyên lý bo ESP32-C3.	41
Hình 3.9. Sơ đồ kết nối nguyên lý bo ESP32-S3.....	43
Hình 3.10. Thiết kế PCB bo ESP32-S3 kích thước 70 mm × 42,5 mm.....	45
Hình 3.12. Thiết kế PCB bo ESP32-C3 và cụm đầu nối cảm biến/gateway.....	46
Hình 3.13. Hình ảnh phần cứng thực tế của cụm thiết bị sau khi lắp ráp.	46
Hình 3.14. Deployment diagram tổng quan Docker/Production	47
Hình 3.16. Phân nhóm API và cơ chế xác thực	48
Hình 3.17. Các nhóm dữ liệu logic của hệ thống.....	50
Hình 3.18. Mô hình tổng quát của database wearable và các nhóm collection chính.	51
Hình 3.19. Sơ đồ dependency graph khi khởi tạo ứng dụng bằng MultiProvider....	53
Hình 3.20. Luồng bootstrap, đăng nhập và refresh token của ứng dụng.	54
Hình 3.21. Cụm màn hình đầu vào của ứng dụng: đăng nhập, lỗi xác thực và đăng ký tài khoản.....	55
Hình 3.22. Cụm màn hình quản lý thiết bị, liên kết thiết bị và phân quyền người xem.	56
Hình 3.23. Cụm màn hình theo dõi thiết bị, dashboard sức khỏe và lịch sử theo ngày.	58
Hình 3.24. Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng.	59
Hình 3.25. Chuỗi vận hành từ đo lường phần cứng đến hiển thị phần mềm.....	61
Hình 3.26. Cấu trúc lắp ráp thực tế của bo gateway ESP32-S3.	72
Hình 3.27. Toàn cảnh nguyên mẫu phần cứng khi kết nối bo đo và bo gateway.....	73
Hình 3.28. Minh họa bổ sung về nguyên mẫu bo gateway đang hiển thị readings thực tế.	101

DANH MỤC TỪ VIẾT TẮT

Bảng dưới đây tổng hợp các từ viết tắt được sử dụng trong báo cáo.

Từ viết tắt	Tiếng Anh	Diễn giải
API	Application Programming Interface	Giao diện lập trình ứng dụng
ECG	Electrocardiogram	Điện tim
ESP32	Espressif 32-bit MCU	Vi điều khiển ESP32 dùng cho thiết bị IoT
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Phiên bản bảo mật của HTTP
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi dạng văn bản
JWT	JSON Web Token	Mã thông báo xác thực dạng JSON
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Phong cách kiến trúc cho web service/API
SpO ₂	Peripheral Capillary Oxygen Saturation	Độ bão hòa oxy ngoại vi trong máu
TTL	Time To Live	Thời gian tồn tại của dữ liệu/bản ghi
UI	User Interface	Giao diện người dùng

CHƯƠNG 1

MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh của Internet of Things, điện toán đám mây, thiết bị nhúng và ứng dụng di động, các hệ thống theo dõi sức khỏe từ xa đang trở thành một hướng nghiên cứu và ứng dụng rất quan trọng. Nhu cầu này đặc biệt rõ ở nhóm người cao tuổi, người có bệnh nền, bệnh nhân cần theo dõi định kỳ hoặc người cần được giám sát liên tục các chỉ số sinh tồn mà không nhất thiết phải ở trong bệnh viện. Thay vì chỉ đo thủ công tại một thời điểm, hệ thống theo dõi sức khỏe từ xa cho phép thu thập dữ liệu liên tục, truyền dữ liệu về máy chủ, xử lý và hiển thị kết quả trên ứng dụng di động để người dùng hoặc người chăm sóc có thể theo dõi nhanh chóng, thuận tiện. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024)

Thực tế cho thấy những chỉ số sức khỏe cơ bản như nhịp tim, nồng độ oxy trong máu, nhiệt độ cơ thể, nhịp thở và điện tim có giá trị lớn trong việc đánh giá tình trạng sức khỏe ban đầu. Nếu các dữ liệu này được thu thập liên tục và hiển thị kịp thời, người sử dụng có thể phát hiện sớm những bất thường để có hướng xử lý phù hợp. Do đó, việc xây dựng một hệ thống có thể thu thập, lưu trữ, hiển thị và cảnh báo dữ liệu sức khỏe là một bài toán có ý nghĩa thực tiễn cao.

Từ góc độ kỹ thuật, đề tài này hấp dẫn ở chỗ nó là một bài toán tích hợp nhiều thành phần khác nhau thành một hệ thống hoàn chỉnh. Theo mã nguồn hiện tại trong Git, hệ thống không chỉ gồm một ứng dụng di động hay một API server riêng lẻ, mà là một kiến trúc tương đối đầy đủ gồm ba lớp chính. Lớp thứ nhất là thiết bị ESP32 đóng vai trò thiết bị đầu cuối thu thập dữ liệu. Lớp thứ hai là backend xây dựng bằng FastAPI đóng vai trò trung tâm xử lý, xác thực, lưu trữ dữ liệu vào MongoDB, quản lý command ECG, phân quyền người dùng, theo dõi tình trạng hệ thống và giới hạn truy cập bằng Redis-backed rate limit. Lớp thứ ba là ứng dụng Flutter dùng để đăng nhập, đồng bộ phiên, lấy danh sách thiết bị liên kết, hiển thị dữ liệu sức khỏe và gửi yêu cầu ECG on-demand.

Một lý do quan trọng khác để chọn đề tài là tính thời sự và khả năng mở rộng thực tế. Hệ thống hiện tại trong Git đã vượt ra ngoài mức của một mô hình demo đơn giản. Backend không chỉ nhận readings rồi trả dữ liệu, mà còn có mô hình xác thực tách biệt giữa người dùng và thiết bị: app/admin sử dụng Authorization: Bearer <JWT>, còn thiết bị ESP dùng X-Device-Token; hệ thống có refresh_token được rotate qua endpoint /api/v1/auth/refresh, lưu dưới dạng hash trong auth_sessions, có phân quyền theo admin / caregiver / patient, có collection device_commands để hỗ trợ ECG on-demand, có các endpoint health check /live, /ready, /health, và có cả metrics để giám sát vận hành. Điều đó cho thấy hệ thống không chỉ phục vụ mục tiêu học tập mà còn mang tính ứng dụng cao, là một nền tảng tốt để phát triển thành sản phẩm hoàn thiện hơn trong tương lai.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng một hệ thống theo dõi sức khỏe từ xa hoạt động theo mô hình kết nối giữa thiết bị phần cứng ESP32, máy chủ xử lý trung tâm sử dụng FastAPI và ứng dụng di động phát triển bằng Flutter. Hệ thống được định hướng nhằm tạo ra một nền tảng thống nhất cho phép thu thập dữ liệu sức khỏe từ thiết bị đo, truyền dữ liệu lên máy chủ, lưu trữ có tổ chức trong cơ sở dữ liệu, hỗ trợ truy vấn và hiển thị thông tin trên ứng dụng một cách trực quan, chính xác và thuận tiện cho người sử dụng.

Bên cạnh chức năng thu thập và hiển thị dữ liệu, đề tài còn hướng đến việc bảo đảm sự phối hợp đồng bộ giữa các thành phần trong toàn bộ hệ thống, từ lớp thiết bị nhúng, lớp dịch vụ xử lý dữ liệu đến lớp giao diện người dùng. Hệ thống cần có cơ chế xác thực và quản lý truy cập phù hợp để bảo đảm tính an toàn, hạn chế truy cập trái phép và hỗ trợ quản lý người dùng theo vai trò hoặc phạm vi sử dụng. Đồng thời, hệ thống phải đáp ứng khả năng cập nhật dữ liệu gần thời gian thực ở mức cơ bản, giúp người dùng có thể theo dõi các chỉ số sức khỏe và trạng thái thiết bị trong thời gian ngắn sau khi dữ liệu được ghi nhận.

Ngoài ra, mục tiêu của đề tài không chỉ dừng lại ở việc xây dựng một mô hình thử nghiệm có khả năng vận hành, mà còn tạo tiền đề cho việc mở rộng trong tương lai. Cụ thể, kiến trúc hệ thống cần được thiết kế theo hướng linh hoạt, dễ nâng cấp và dễ tích hợp thêm các chức năng mới như cảnh báo bất thường, theo dõi lịch sử sức khỏe, quản lý nhiều thiết bị, hỗ trợ nhiều đối tượng người dùng hoặc kết nối với các dịch vụ xử lý dữ liệu nâng cao. Qua đó, đề tài góp phần xây dựng một giải pháp công nghệ có tính ứng dụng thực tiễn trong lĩnh vực chăm sóc và giám sát sức khỏe từ xa.

1.2.2 Mục tiêu cụ thể

Trước hết, hệ thống cần xây dựng được khối backend đóng vai trò trung tâm trong việc tiếp nhận, xử lý và cung cấp dữ liệu. Backend phải có khả năng nhận dữ liệu đo từ thiết bị ESP32, thực hiện kiểm tra và chuẩn hóa dữ liệu đầu vào trước khi lưu trữ vào cơ sở dữ liệu MongoDB. Đồng thời, backend phải cung cấp các API phục vụ ứng dụng di động trong việc truy xuất dữ liệu sức khỏe mới nhất, dữ liệu lịch sử, dữ liệu ECG và các thông tin tổng hợp về tình trạng sức khỏe người dùng. Ngoài ra, backend cũng phải hỗ trợ các chức năng quản lý người dùng, quản lý thiết bị, lưu trữ và quản lý phiên xác thực, tạo và xử lý cảnh báo, đồng thời bảo đảm khả năng vận hành trong môi trường triển khai bằng Docker Compose.

Tiếp theo, hệ thống cần xây dựng được cơ chế xác thực, phân quyền và quản lý phiên phù hợp với đặc thù nhiều tác nhân tham gia. Cụ thể, ứng dụng Flutter và tài khoản quản trị sử dụng cơ chế xác thực dựa trên JWT Bearer khi truy cập API, trong khi thiết bị ESP32 sử dụng mã định danh truy cập riêng thông qua X-Device-Token. Bên cạnh đó, hệ thống phải hỗ trợ cơ chế làm mới

phiên đăng nhập bằng `refresh_token`, trong đó token được xoay vòng, băm và lưu trữ trong tập dữ liệu phiên xác thực để tăng mức độ an toàn. Hệ thống đồng thời phải bảo đảm khả năng phân quyền theo vai trò như quản trị viên, người chăm sóc và người được theo dõi, từ đó kiểm soát phạm vi truy cập và thao tác phù hợp với từng nhóm người dùng.

Một mục tiêu quan trọng khác là xây dựng ứng dụng di động Flutter phục vụ người dùng cuối. Ứng dụng phải hỗ trợ đầy đủ các chức năng cơ bản gồm đăng ký tài khoản, đăng nhập, khôi phục phiên làm việc, truy xuất thông tin người dùng hiện tại, đồng bộ danh sách thiết bị đã liên kết, hiển thị dữ liệu sức khỏe mới nhất, tra cứu dữ liệu lịch sử và gửi yêu cầu đo ECG. Về mặt kiến trúc, ứng dụng cần được tổ chức theo hướng tách biệt các tầng chức năng như tầng giao tiếp API, tầng lưu trữ cục bộ, tầng mô hình dữ liệu nghiệp vụ và tầng quản lý trạng thái giao diện. Cách tổ chức này nhằm bảo đảm tính rõ ràng trong thiết kế, thuận lợi cho bảo trì, kiểm thử và mở rộng chức năng về sau.

Đề tài cũng đặt ra mục tiêu xây dựng và kiểm chứng luồng đo ECG theo yêu cầu như một chức năng đặc trưng của hệ thống. Khác với mô hình chỉ tiếp nhận dữ liệu đo định kỳ từ thiết bị, hệ thống phải hỗ trợ cơ chế tương tác hai chiều giữa ứng dụng và thiết bị. Theo đó, ứng dụng di động có thể gửi yêu cầu đo ECG đến backend; backend ghi nhận yêu cầu và tạo lệnh tương ứng trong hàng đợi xử lý của thiết bị; thiết bị ESP32 định kỳ kiểm tra lệnh, thực hiện phép đo và gửi tín hiệu phản hồi kèm dữ liệu kết quả về hệ thống. Cơ chế này cho phép hệ thống không chỉ giám sát thụ động mà còn hỗ trợ điều khiển tác vụ đo từ xa, qua đó nâng cao tính thực tiễn và giá trị ứng dụng của giải pháp.

Cuối cùng, hệ thống cần được xây dựng theo hướng có thể triển khai và vận hành thực tế chứ không chỉ dừng lại ở mức nguyên mẫu chức năng. Cụ thể, toàn bộ các thành phần chính của hệ thống phải được tổ chức để có thể chạy trong môi trường Docker Compose, bao gồm backend, cơ sở dữ liệu MongoDB, Redis và các dịch vụ hỗ trợ liên quan. Hệ thống cũng cần có các cơ chế phục vụ giám sát và bảo trì như kiểm tra tình trạng hoạt động, kiểm tra khả năng sẵn sàng, thu thập thông tin vận hành cơ bản, cũng như hỗ trợ sao lưu và khôi phục dữ liệu. Mục tiêu này nhằm bảo đảm hệ thống có tính ổn định, dễ triển khai, dễ kiểm tra và tạo nền tảng cho việc mở rộng trong các giai đoạn phát triển tiếp theo.

1.2.3 Kết quả mong đợi

Kết quả mong đợi của đề tài là xây dựng được một nguyên mẫu hệ thống theo dõi sức khỏe từ xa có khả năng hoạt động thực tế ở mức cơ bản, bảo đảm sự phối hợp giữa thiết bị phần cứng, máy chủ xử lý trung tâm và ứng dụng di động. Nguyên mẫu này cần thể hiện được đầy đủ các luồng chức năng chính đã đặt ra trong phạm vi nghiên cứu và triển khai của đề tài.

Cụ thể, ở lớp thiết bị, hệ thống được kỳ vọng cho phép thiết bị ESP32 thực hiện việc thu thập dữ liệu sức khỏe, truyền dữ liệu về backend theo đúng định

dạng quy định và tiếp nhận các lệnh điều khiển từ máy chủ khi có yêu cầu. Điều này thể hiện rằng thiết bị không chỉ đóng vai trò là nguồn phát sinh dữ liệu mà còn tham gia vào cơ chế tương tác hai chiều với hệ thống.

Ở lớp xử lý trung tâm, backend xây dựng bằng FastAPI được kỳ vọng có khả năng tiếp nhận dữ liệu gửi lên từ thiết bị, xử lý dữ liệu đầu vào, lưu trữ dữ liệu trong cơ sở dữ liệu và cung cấp các giao diện lập trình ứng dụng phục vụ cho việc truy vấn thông tin. Các API này cần đáp ứng được nhu cầu lấy dữ liệu sức khỏe mới nhất, dữ liệu lịch sử, dữ liệu ECG và các thông tin liên quan đến người dùng hoặc thiết bị, từ đó bảo đảm vai trò trung tâm điều phối của backend trong toàn bộ hệ thống.

Ở lớp ứng dụng, ứng dụng di động Flutter được kỳ vọng hỗ trợ người dùng trong việc đăng nhập, duy trì và khôi phục phiên làm việc, đồng thời hiển thị thông tin sức khỏe dưới các góc nhìn phù hợp với mô hình dữ liệu của hệ thống. Cụ thể, ứng dụng cần có khả năng trình bày dữ liệu theo người dùng hoặc theo thiết bị, giúp người sử dụng dễ dàng theo dõi trạng thái sức khỏe, tra cứu lịch sử và tương tác với các chức năng giám sát từ xa.

Bên cạnh các chức năng theo dõi cơ bản, một kết quả quan trọng được mong đợi là hệ thống phải hỗ trợ được cơ chế ECG on-demand. Nghĩa là từ ứng dụng di động, người dùng có thể gửi yêu cầu đo ECG; backend tiếp nhận và phát sinh lệnh điều khiển tương ứng; thiết bị nhận lệnh, thực hiện tác vụ đo và phản hồi kết quả về hệ thống. Việc hiện thực thành công luồng chức năng này cho thấy hệ thống có khả năng hỗ trợ tương tác hai chiều giữa người dùng và thiết bị, vượt ra ngoài mô hình giám sát thụ động thông thường.

Ngoài ra, nguyên mẫu hệ thống cũng được kỳ vọng tích hợp các thành phần kỹ thuật cần thiết để phục vụ quá trình triển khai thử nghiệm. Các thành phần này bao gồm cơ chế bảo mật cơ bản trong xác thực và quản lý truy cập, cơ chế giới hạn tần suất yêu cầu nhằm giảm nguy cơ lạm dụng tài nguyên hệ thống, cũng như các chức năng giám sát vận hành ở mức cơ bản để hỗ trợ kiểm tra trạng thái và đánh giá khả năng hoạt động của toàn bộ hệ thống trong môi trường triển khai thực tế.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là một hệ thống theo dõi sức khỏe từ xa được xây dựng theo mô hình client-server-IoT, trong đó các thành phần phần cứng, phần mềm và hạ tầng truyền thông phối hợp với nhau để thực hiện chức năng thu thập, truyền nhận, xử lý, lưu trữ và hiển thị dữ liệu sức khỏe. Trong phạm vi nghiên cứu của đề tài, các đối tượng chính được xem xét bao gồm thiết bị nhúng, hệ thống backend, ứng dụng di động và cơ chế giao tiếp giữa các thành phần trong toàn bộ hệ thống.

Trước hết, một đối tượng nghiên cứu quan trọng là thiết bị ESP32 đóng vai trò là nút IoT đầu cuối. Thiết bị này có nhiệm vụ thu thập dữ liệu sức khỏe từ cảm biến

hoặc từ mô-đun đo liên kết, đóng gói dữ liệu theo định dạng quy định và gửi dữ liệu về backend thông qua kết nối mạng. Bên cạnh chức năng gửi dữ liệu, thiết bị còn là đối tượng nghiên cứu trong cơ chế tiếp nhận lệnh điều khiển từ máy chủ, đặc biệt trong các tình huống yêu cầu đo dữ liệu theo tác vụ, chẳng hạn như ECG on-demand. Vì vậy, thiết bị không chỉ được xem xét ở góc độ phát sinh dữ liệu mà còn ở khả năng tương tác hai chiều với hệ thống.

Đối tượng nghiên cứu tiếp theo là backend của hệ thống, được xây dựng theo kiến trúc dịch vụ sử dụng FastAPI. Backend giữ vai trò trung tâm trong việc tiếp nhận dữ liệu từ thiết bị, xử lý và chuẩn hóa dữ liệu đầu vào, tổ chức lưu trữ trong cơ sở dữ liệu, đồng thời cung cấp các API phục vụ cho ứng dụng di động và các tác nhân quản trị. Ngoài chức năng xử lý dữ liệu, backend còn là đối tượng nghiên cứu ở các khía cạnh xác thực, phân quyền, quản lý phiên đăng nhập, quản lý thiết bị, xử lý cảnh báo và điều phối các luồng nghiệp vụ giữa ứng dụng và thiết bị IoT.

Một đối tượng nghiên cứu khác là ứng dụng di động Flutter, đóng vai trò là giao diện tương tác chính của người dùng cuối với hệ thống. Ứng dụng được nghiên cứu ở phương diện tổ chức chức năng và hiển thị dữ liệu, bao gồm các khả năng như đăng ký, đăng nhập, duy trì phiên làm việc, đồng bộ thiết bị liên kết, hiển thị dữ liệu sức khỏe mới nhất, tra cứu lịch sử đo và gửi yêu cầu đo ECG. Đồng thời, ứng dụng cũng là đối tượng được xem xét về mặt kiến trúc phần mềm, cụ thể là cách tổ chức các lớp giao tiếp API, lưu trữ cục bộ, mô hình nghiệp vụ và quản lý trạng thái giao diện.

Ngoài các thành phần cốt lõi nêu trên, đề tài còn nghiên cứu cơ chế giao tiếp và tích hợp giữa các thành phần trong hệ thống. Nội dung này bao gồm giao tiếp giữa ESP32 và backend thông qua API hoặc token thiết bị, giao tiếp giữa ứng dụng Flutter và backend thông qua cơ chế JWT Bearer, cũng như các luồng truyền dữ liệu và phản hồi lệnh trong quá trình vận hành. Việc nghiên cứu cơ chế tích hợp này nhằm bảo đảm rằng hệ thống hoạt động đồng bộ, dữ liệu được trao đổi đúng ngữ nghĩa và các thành phần có thể phối hợp ổn định trong môi trường thực tế.

Bên cạnh đó, đề tài cũng xem xét môi trường triển khai và vận hành hệ thống như một phần của đối tượng nghiên cứu. Cụ thể, việc tổ chức các dịch vụ backend, cơ sở dữ liệu, bộ nhớ đệm và các thành phần hỗ trợ bằng Docker Compose là một nội dung cần được khảo sát nhằm đánh giá khả năng triển khai, giám sát, kiểm tra trạng thái hoạt động và mở rộng hệ thống trong tương lai.

1.4 Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài được giới hạn nhằm phù hợp với thời gian thực hiện, nguồn lực triển khai và mục tiêu của đề án, đồng thời vẫn bảo đảm xây dựng được một nguyên mẫu hệ thống có tính hoàn chỉnh về mặt chức năng và có khả năng vận hành ở mức cơ bản. Trong phạm vi đó, đề tài tập trung vào việc phân tích, thiết kế và xây dựng một hệ thống theo dõi sức khỏe từ xa theo mô hình client-server-IoT, bao gồm thiết bị ESP32, backend FastAPI, ứng dụng di động Flutter và môi trường triển khai hỗ trợ vận hành hệ thống.

Về mặt chức năng, đề tài tập trung nghiên cứu và hiện thực các chức năng cốt lõi của hệ thống, bao gồm thu thập dữ liệu sức khỏe từ thiết bị, truyền dữ liệu về máy chủ, lưu trữ dữ liệu trong cơ sở dữ liệu, cung cấp API cho ứng dụng di động truy vấn dữ liệu mới nhất và dữ liệu lịch sử, đồng thời hỗ trợ hiển thị dữ liệu sức khỏe trên giao diện người dùng. Bên cạnh đó, đề tài cũng giới hạn trong việc xây dựng một số chức năng quản lý cơ bản như quản lý người dùng, quản lý thiết bị liên kết, quản lý phiên đăng nhập, tiếp nhận và xử lý yêu cầu đo ECG theo yêu cầu.

Về mặt kỹ thuật, phạm vi nghiên cứu tập trung vào việc xây dựng cơ chế giao tiếp giữa các thành phần chính của hệ thống. Cụ thể, đề tài xem xét luồng trao đổi dữ liệu giữa thiết bị ESP32 và backend, luồng truy cập dịch vụ giữa ứng dụng Flutter và backend, cũng như cơ chế xác thực và phân quyền tương ứng cho từng loại tác nhân tham gia. Ngoài ra, đề tài chỉ hướng đến mức cập nhật dữ liệu gần thời gian thực ở mức cơ bản, phục vụ mục tiêu theo dõi và kiểm tra trạng thái hệ thống, chưa đi sâu vào các cơ chế truyền thông thời gian thực có yêu cầu cao về độ trễ hoặc khả năng chịu tải lớn.

Về mặt triển khai, đề tài giới hạn trong môi trường thử nghiệm có tổ chức bằng Docker Compose, bao gồm các thành phần cần thiết như backend, cơ sở dữ liệu MongoDB, Redis và các dịch vụ hỗ trợ liên quan. Việc triển khai trong phạm vi này nhằm phục vụ mục đích kiểm thử, đánh giá khả năng vận hành và kiểm chứng tính khả thi của giải pháp, chứ chưa đặt mục tiêu triển khai ở quy mô sản xuất hoặc thương mại hóa thực tế.

Bên cạnh những nội dung nằm trong phạm vi nghiên cứu, đề tài không đi sâu vào thiết kế vi mạch cảm biến ở mức chuyên sâu hoặc kiểm định y khoa theo chuẩn thiết bị y tế. Tuy nhiên, báo cáo vẫn mô tả đầy đủ lớp phần cứng của nguyên mẫu ở các khía cạnh cần thiết cho đồ án, gồm sơ đồ kết nối, bảng chân, bố trí PCB, nguyên lý hoạt động, lắp ráp và kiểm thử thực nghiệm. Các nội dung ngoài phạm vi vẫn bao gồm phân tích y khoa chuyên sâu, tối ưu hiệu năng quy mô lớn, yêu cầu pháp lý – tiêu chuẩn y tế mức cao và tích hợp với các hệ thống y tế bên ngoài.

1.4.1 Phạm vi chức năng

Trong phạm vi của đề tài, hệ thống được giới hạn triển khai ở các chức năng cốt lõi, đủ để hình thành một nguyên mẫu theo dõi sức khỏe từ xa có khả năng vận hành tương đối hoàn chỉnh. Các chức năng này được xác định dựa trên kiến trúc và mã nguồn hiện tại của hệ thống, bao gồm cả phía backend và ứng dụng di động.

Trước hết, hệ thống hỗ trợ các chức năng liên quan đến tài khoản và phiên người dùng, bao gồm đăng ký tài khoản, đăng nhập, khôi phục phiên làm việc và quản lý trạng thái xác thực trong quá trình sử dụng. Đây là nhóm chức năng nền tảng, bảo đảm người dùng có thể truy cập hệ thống một cách hợp lệ và duy trì phiên làm việc khi tương tác với ứng dụng.

Bên cạnh đó, hệ thống hỗ trợ quản lý thiết bị liên kết với người dùng. Chức năng này cho phép người dùng đồng bộ danh sách thiết bị đã được liên kết, lựa chọn thiết bị đang theo dõi và làm cơ sở để truy vấn dữ liệu sức khỏe theo từng thiết bị cụ thể. Đây là một điểm quan trọng vì hệ thống hiện tại được tổ chức theo hướng device-centric, trong đó phần lớn dữ liệu và quyền truy cập đều gắn với thiết bị.

Ở lớp tiếp nhận dữ liệu, hệ thống hỗ trợ nhận dữ liệu sức khỏe từ thiết bị ESP32, sau đó thực hiện chuẩn hóa và lưu trữ dữ liệu vào MongoDB. Các dữ liệu được xử lý trong phạm vi chức năng của đề tài bao gồm các chỉ số sức khỏe cơ bản và dữ liệu ECG. Việc lưu trữ này là cơ sở cho các chức năng theo dõi thời gian thực, xem lại lịch sử và tạo cảnh báo tự động.

Sau khi dữ liệu được lưu trữ, hệ thống cho phép truy vấn dữ liệu sức khỏe mới nhất và dữ liệu lịch sử, phục vụ nhu cầu theo dõi trạng thái hiện tại cũng như xem lại diễn biến chỉ số theo thời gian. Ngoài ra, hệ thống còn hỗ trợ truy vấn dữ liệu ECG và bản tóm tắt sức khỏe, giúp người dùng có thể tiếp cận thông tin sức khỏe ở cả mức chi tiết và mức tổng hợp.

Một chức năng đặc trưng khác trong phạm vi của đề tài là tạo và xử lý yêu cầu ECG on-demand. Thay vì chỉ tiếp nhận dữ liệu đo định kỳ, hệ thống còn cho phép ứng dụng gửi yêu cầu đo ECG, backend tạo command tương ứng và thiết bị thực hiện tác vụ khi nhận được lệnh. Cơ chế này làm cho hệ thống có khả năng tương tác hai chiều giữa ứng dụng và thiết bị.

Bên cạnh đó, hệ thống còn hỗ trợ sinh cảnh báo tự động theo ngưỡng khi dữ liệu sức khỏe vượt quá các giá trị được cấu hình. Chức năng này giúp tăng giá trị sử dụng thực tế của hệ thống, vì dữ liệu không chỉ được lưu trữ và hiển thị mà còn được dùng để phát hiện các trạng thái bất thường ở mức cơ bản.

Cuối cùng, ở góc độ vận hành, hệ thống cung cấp các chức năng hỗ trợ kiểm tra trạng thái hoạt động như health check và cơ chế rate limit cơ bản để bảo vệ API. Đây là các chức năng không trực tiếp phục vụ nghiệp vụ người dùng, nhưng có ý nghĩa quan trọng trong việc bảo đảm hệ thống có thể vận hành ổn định trong môi trường thử nghiệm.

1.4.2 Phạm vi công nghệ

Phạm vi công nghệ của đề tài được giới hạn theo đúng các công nghệ đang sử dụng trong hệ thống hiện thực, nhằm bảo đảm báo cáo bám sát kiến trúc triển khai thực tế và phản ánh đúng giải pháp đã xây dựng. Việc giới hạn này giúp tránh mở rộng sang các công nghệ không xuất hiện trong mã nguồn hoặc không tham gia trực tiếp vào vận hành hệ thống.

Cụ thể, ở lớp thiết bị đầu cuối, đề tài sử dụng ESP32 làm nền tảng phần cứng cho khối thu thập và truyền dữ liệu sức khỏe. Ở lớp xử lý trung tâm, FastAPI được dùng để xây dựng backend, tổ chức các dịch vụ API và xử lý các

ng nghiệp vụ chính của hệ thống. Về lưu trữ dữ liệu, hệ thống sử dụng MongoDB làm cơ sở dữ liệu chính để quản lý thông tin người dùng, thiết bị và dữ liệu sức khỏe. Bên cạnh đó, Redis được sử dụng cho các chức năng hỗ trợ hạ tầng, đặc biệt là cơ chế rate limiting nhằm kiểm soát tần suất truy cập API.

Ở lớp ứng dụng người dùng, đề tài sử dụng Flutter để phát triển ứng dụng di động, phục vụ các chức năng tương tác với người dùng và kết nối với backend. Về môi trường triển khai, Docker Compose được dùng để tổ chức và vận hành các thành phần như backend, MongoDB và Redis trong cùng một môi trường chạy.

1.4.3 Phạm vi người dùng

Phạm vi người dùng của đề tài được giới hạn ở các tác nhân đang tham gia trực tiếp vào quá trình vận hành hệ thống trong phiên bản hiện tại, nhằm bảo đảm việc phân tích, thiết kế và triển khai bám sát mô hình sử dụng thực tế.

Trong phạm vi này, hệ thống gồm ba nhóm tác nhân chính. Thứ nhất là quản trị viên hệ thống, có vai trò quản lý tài khoản, thiết bị, cấu hình và hỗ trợ duy trì hoạt động chung của hệ thống. Thứ hai là người dùng ứng dụng di động, bao gồm người được theo dõi hoặc người chăm sóc, với các chức năng như đăng nhập, đồng bộ thiết bị, xem dữ liệu sức khỏe, tra cứu lịch sử và gửi một số yêu cầu nghiệp vụ. Thứ ba là thiết bị ESP32, được xem như một tác nhân kỹ thuật, tham gia vào hệ thống thông qua việc gửi dữ liệu sức khỏe, nhận lệnh điều khiển và phản hồi kết quả xử lý.

Ngoài các nhóm trên, đề tài chưa mở rộng đến các đối tượng như bác sĩ, cơ sở y tế, đơn vị giám sát tổ chức hoặc các hệ thống bên thứ ba. Việc xác định phạm vi người dùng như vậy giúp báo cáo bám sát hiện trạng triển khai, đồng thời làm rõ cơ sở xây dựng các mô hình chức năng và cơ chế phân quyền của hệ thống.

1.4.4 Phạm vi dữ liệu

Phạm vi dữ liệu của đề tài được giới hạn ở các loại dữ liệu sức khỏe cơ bản mà hệ thống hiện tại có khả năng thu thập, truyền nhận, lưu trữ, truy vấn và hiển thị. Cách xác định này giúp đề tài bám sát chức năng thực tế của hệ thống và phù hợp với mục tiêu xây dựng nguyên mẫu theo dõi sức khỏe từ xa.

Cụ thể, hệ thống tập trung xử lý các chỉ số chính gồm nhịp tim, SpO2, nhiệt độ, nhịp thở và dữ liệu ECG. Các dữ liệu này được thiết bị thu thập, gửi về backend và phục vụ cho việc theo dõi, tra cứu lịch sử, hiển thị trên ứng dụng di động và cảnh báo theo ngưỡng ở mức cơ bản.

Tuy nhiên, đề tài không đi sâu vào phân tích y khoa chuyên sâu và không nhằm thay thế các thiết bị chẩn đoán chuyên dụng. Hệ thống cũng chưa mở rộng

sang các hướng xử lý nâng cao như dự đoán bệnh lý, phân loại tình trạng sức khỏe bằng AI hay hỗ trợ chẩn đoán lâm sàng.

1.4.5 Những nội dung ngoài phạm vi đề tài

Để bảo đảm phù hợp với thời gian, nguồn lực và mục tiêu của đề án, đề tài không mở rộng sang một số nội dung ngoài phạm vi hiện thực chính của hệ thống. Việc giới hạn này giúp tập trung vào các chức năng cốt lõi của nguyên mẫu và bảo đảm tính khả thi trong quá trình triển khai.

Cụ thể, đề tài không đi sâu vào thiết kế phần cứng cảm biến y tế ở mức vi mạch chuyên dụng và không bao gồm kiểm định hay chứng nhận độ chính xác y khoa của thiết bị. Phạm vi phần cứng của đề án dừng ở mức thiết kế nguyên mẫu có thể chế tạo, lắp ráp và chạy thử, đủ để chứng minh khả năng tích hợp giữa cảm biến, hai bo ESP32, backend và ứng dụng Flutter. Bên cạnh đó, đề tài cũng không hướng đến phân tích dữ liệu ECG ở mức lâm sàng hoặc hỗ trợ chẩn đoán bệnh.

Về hạ tầng, đề tài chưa mở rộng đến hệ thống thông báo đẩy ở quy mô production, triển khai cloud hoàn chỉnh với autoscaling, message broker chuyên dụng hay dashboard giám sát production đầy đủ. Các cơ chế hiện tại chủ yếu phục vụ vận hành thử nghiệm và kiểm tra nguyên mẫu.

1.5 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu để bảo đảm vừa có cơ sở lý thuyết, vừa có triển khai thực nghiệm.

1.5.1 Phương pháp nghiên cứu lý thuyết

Đề tài tiến hành nghiên cứu các cơ sở lý thuyết liên quan trực tiếp đến bài toán xây dựng hệ thống theo dõi sức khỏe từ xa, làm nền tảng cho quá trình phân tích, thiết kế và triển khai hệ thống.

Nội dung nghiên cứu tập trung vào mô hình theo dõi sức khỏe từ xa, nguyên lý IoT và thiết bị nhúng trong y tế, kiến trúc client-server, mô hình REST API, cùng với các cơ chế xác thực và quản lý phiên như JWT và refresh token. Bên cạnh đó, đề tài cũng khảo sát các công nghệ hiện thực đang sử dụng như FastAPI, Flutter, MongoDB, Redis và Docker Compose để làm rõ vai trò và phạm vi ứng dụng của từng thành phần trong hệ thống.

Nhìn chung, phương pháp nghiên cứu lý thuyết giúp đề tài xây dựng cơ sở khoa học cho việc lựa chọn mô hình, công nghệ và cách tổ chức hệ thống phù hợp với mục tiêu của đề án.

1.5.2 Phương pháp phân tích yêu cầu

Từ bài toán xây dựng hệ thống theo dõi sức khỏe từ xa, đề tài tiến hành phân tích yêu cầu theo hai nhóm chính là yêu cầu chức năng và yêu cầu phi chức năng. Quá trình này nhằm xác định rõ các chức năng hệ thống cần cung cấp, các tác nhân tham gia và các điều kiện kỹ thuật cần đáp ứng trong quá trình triển khai.

Trước hết, đề tài xác định các tác nhân chính của hệ thống gồm người dùng ứng dụng di động, người chăm sóc, quản trị viên và thiết bị ESP32. Trên cơ sở đó, các yêu cầu chức năng được xác định như đăng nhập, quản lý phiên, theo dõi dữ liệu sức khỏe, quản lý thiết bị, gửi yêu cầu đo ECG, tiếp nhận dữ liệu từ thiết bị và thực hiện các chức năng quản trị.

Bên cạnh đó, đề tài cũng phân tích các yêu cầu phi chức năng như xác thực, phân quyền, quản lý phiên, tính nhất quán dữ liệu, bảo mật, giới hạn truy cập và khả năng triển khai bằng Docker Compose. Sau khi phân tích, các yêu cầu này được đối chiếu với các thành phần hiện thực trong mã nguồn backend và ứng dụng Flutter nhằm bảo đảm sự thống nhất giữa phần phân tích và phần triển khai.

1.5.3 Phương pháp thiết kế hệ thống

Sau khi xác định và phân tích yêu cầu, đề tài tiến hành thiết kế hệ thống theo kiến trúc nhiều lớp nhằm bảo đảm tính phân tách chức năng, tính rõ ràng trong tổ chức thành phần và thuận lợi cho việc mở rộng. Hệ thống gồm bốn lớp chính: lớp thiết bị IoT, lớp backend API, lớp cơ sở dữ liệu và lớp ứng dụng Flutter.

Trong đó, thiết bị IoT đảm nhiệm việc thu thập và gửi dữ liệu sức khỏe, backend thực hiện xử lý nghiệp vụ và cung cấp API, cơ sở dữ liệu lưu trữ thông tin hệ thống, còn ứng dụng Flutter hỗ trợ người dùng theo dõi dữ liệu và thực hiện các thao tác cần thiết. Bên cạnh cấu trúc nhiều lớp, đề tài cũng thiết kế các luồng dữ liệu và luồng xác thực bám sát cách tổ chức thực tế của hệ thống.

Cụ thể, ứng dụng sử dụng JWT Bearer kết hợp refresh token để xác thực và duy trì phiên làm việc, trong khi thiết bị sử dụng X-Device-Token khi giao tiếp với backend. Về lưu trữ, dữ liệu sức khỏe được tổ chức trong `health_readings`, lệnh điều khiển ECG trong `device_commands`, và thông tin phiên trong `auth_sessions`. Cách thiết kế này giúp hệ thống vừa phản ánh đúng hiện trạng triển khai, vừa tạo cơ sở rõ ràng cho các bước hiện thực và kiểm thử tiếp theo.

1.5.4 Phương pháp triển khai thực nghiệm

Phương pháp triển khai thực nghiệm của đề tài được thực hiện trực tiếp trên các thành phần hệ thống đã được xây dựng thực tế, nhằm bảo đảm nội dung

báo cáo phản ánh đúng hiện trạng triển khai và cho phép đối chiếu giữa phân tích, thiết kế và kết quả hiện thực.

Cụ thể, quá trình triển khai thực nghiệm được thực hiện trên cả ba lớp của hệ thống. Ở lớp thiết bị, đề tài tiến hành lắp ráp hai bo ESP32, đầu nối cảm biến, kiểm tra nguồn, kiểm tra giao tiếp I²C/1-Wire/SPI/BLE và chạy thử từng mô-đun trước khi tích hợp toàn hệ thống. Ở lớp xử lý trung tâm, backend FastAPI được vận hành cùng MongoDB, Redis và Docker Compose để kiểm chứng khả năng tiếp nhận dữ liệu, xử lý nghiệp vụ, lưu trữ và quản lý phiên. Ở lớp ứng dụng, Flutter được dùng để kiểm tra luồng đăng nhập, đồng bộ thiết bị, xem dữ liệu sức khỏe, lịch sử và gửi yêu cầu ECG.

Bên cạnh đó, đề tài tiến hành kiểm thử theo chuỗi tích hợp: từ việc cảm biến đo được tín hiệu, bo S3 tổng hợp và hiển thị tại chỗ, đến việc backend nhận readings và ứng dụng Flutter truy xuất lại kết quả. Thông qua phương pháp này, báo cáo có cơ sở đánh giá không chỉ phần mềm mà cả tính khả thi của nguyên mẫu phần cứng trong điều kiện vận hành thực tế.

1.5.5 Phương pháp kiểm thử và đánh giá

Sau khi hoàn thành triển khai, hệ thống được kiểm thử theo các luồng chức năng chính nhằm xác định mức độ hoạt động đúng đắn và sự phù hợp với các yêu cầu đã đặt ra. Nội dung kiểm thử tập trung vào các chức năng như đăng nhập và quản lý phiên, tải danh sách thiết bị, xem dữ liệu sức khỏe, xử lý các trạng thái biên và gửi yêu cầu đo ECG.

Bên cạnh đó, hệ thống còn được kiểm tra ở góc độ vận hành thông qua các nội dung như readiness, health check và log hệ thống, nhằm đánh giá khả năng hoạt động ổn định trong môi trường triển khai thử nghiệm. Trên cơ sở kết quả kiểm thử, hệ thống được đánh giá theo các tiêu chí chính gồm mức độ đáp ứng yêu cầu chức năng, tính đúng đắn của luồng dữ liệu, khả năng vận hành ổn định và tiềm năng mở rộng.

1.6 Ý nghĩa khoa học và thực tiễn của đề tài

1.6.1 Ý nghĩa khoa học

Về mặt khoa học, đề tài có ý nghĩa trong việc nghiên cứu và minh họa mô hình hệ thống theo dõi sức khỏe từ xa theo hướng tích hợp nhiều lớp công nghệ, bao gồm thiết bị IoT, backend, cơ sở dữ liệu và ứng dụng di động. Cách tiếp cận này giúp làm rõ mối quan hệ giữa các thành phần trong một hệ thống chăm sóc sức khỏe hiện đại.

Bên cạnh đó, đề tài còn có giá trị ở khía cạnh phân tích và hiện thực một số vấn đề kỹ thuật như xác thực nhiều tác nhân, quản lý phiên bằng access token và refresh token, tổ chức dữ liệu sức khỏe trên cơ sở dữ liệu tài liệu, cơ chế polling queue cho yêu cầu ECG, rate limiting và health check. Những nội dung

này góp phần minh họa cách xây dựng một hệ thống IoT có khả năng vận hành ổn định và phù hợp với yêu cầu thực tế.

Nhìn chung, ý nghĩa khoa học của đề tài nằm ở việc cung cấp một mô hình tham khảo tương đối hoàn chỉnh cho bài toán theo dõi sức khỏe từ xa trên nền tảng IoT, làm cơ sở cho các nghiên cứu và phát triển tiếp theo trong cùng lĩnh vực.

1.6.2 Ý nghĩa thực tiễn

Về mặt thực tiễn, đề tài có ý nghĩa trong việc xây dựng một nguyên mẫu hệ thống theo dõi sức khỏe từ xa có khả năng đáp ứng các nhu cầu giám sát sức khỏe cơ bản. Hệ thống cho phép thu thập dữ liệu từ thiết bị, lưu trữ tập trung, truy vấn theo người dùng hoặc thiết bị và hiển thị thông tin trên ứng dụng di động, đồng thời hỗ trợ yêu cầu đo ECG và cảnh báo theo ngưỡng cấu hình.

Đề tài đặc biệt phù hợp với các nhu cầu như theo dõi sức khỏe tại nhà, hỗ trợ người cao tuổi, theo dõi sau điều trị và chăm sóc từ xa trong gia đình. Bên cạnh giá trị ở mức nguyên mẫu, hệ thống còn tạo ra một nền tảng ban đầu để tiếp tục mở rộng thêm các chức năng như giao diện quản trị, thông báo đẩy, hạ tầng xử lý chuyên dụng và công cụ giám sát production.

Ý nghĩa thực tiễn của đề tài nằm ở khả năng xây dựng một hệ thống có thể vận hành, đáp ứng nhu cầu giám sát sức khỏe cơ bản và có tiềm năng phát triển thành giải pháp ứng dụng thực tế trong tương lai.

1.6.3 Ý nghĩa đối với học tập và phát triển kỹ năng

Đối với người thực hiện, đề tài có ý nghĩa trong việc rèn luyện năng lực chuyên môn theo hướng tổng hợp, thông qua toàn bộ quá trình phân tích yêu cầu, thiết kế, triển khai, kiểm thử và đánh giá hệ thống. Qua đó, người thực hiện không chỉ tiếp cận một kỹ thuật riêng lẻ mà còn có cái nhìn toàn diện về quy trình xây dựng một hệ thống phần mềm hoàn chỉnh.

Bên cạnh đó, đề tài giúp phát triển các kỹ năng quan trọng như phân tích yêu cầu, thiết kế kiến trúc backend, xây dựng cơ sở dữ liệu, lập trình API, phát triển ứng dụng Flutter, quản lý xác thực và phiên làm việc, triển khai bằng Docker Compose, kiểm thử và đánh giá hệ thống. Đây đều là những kỹ năng có giá trị thực tiễn cao trong học tập và công việc.

Nhìn chung, đề tài không chỉ mang lại kết quả ở mức sản phẩm, mà còn góp phần hình thành tư duy kỹ thuật toàn diện, giúp người thực hiện tích lũy kiến thức, kinh nghiệm và kỹ năng phục vụ cho học tập, nghiên cứu và định hướng nghề nghiệp sau này.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan về hệ thống theo dõi sức khỏe từ xa

Theo dõi sức khỏe từ xa là mô hình thu thập, truyền, lưu trữ và hiển thị các chỉ số sức khỏe của người dùng thông qua các thiết bị điện tử có kết nối mạng. Trong mô hình này, dữ liệu không còn bị giới hạn ở việc đo thủ công tại từng thời điểm, mà có thể được ghi nhận liên tục hoặc theo chu kỳ, sau đó được gửi về máy chủ để xử lý và cung cấp lại cho ứng dụng giám sát. Đây là một hướng phát triển quan trọng trong bối cảnh nhu cầu chăm sóc sức khỏe cá nhân, chăm sóc người cao tuổi và giám sát bệnh nhân tại nhà ngày càng tăng. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024)

Về bản chất, hệ thống theo dõi sức khỏe từ xa là một hệ thống tích hợp đa thành phần. Nó thường gồm thiết bị cảm biến hoặc thiết bị nhúng để thu nhận dữ liệu, một máy chủ trung tâm để tiếp nhận và xử lý dữ liệu, và một ứng dụng phía người dùng để hiển thị và theo dõi. Trong đề tài này, kiến trúc đó được hiện thực rõ ràng trong Git: thiết bị ESP32 gửi dữ liệu sức khỏe và nhận command ECG; backend FastAPI tiếp nhận dữ liệu, lưu MongoDB, kiểm tra ngưỡng cảnh báo và cung cấp API; còn app Flutter lấy dữ liệu latest, history, summary và ECG để hiển thị cho người dùng.

Theo source backend hiện tại, hệ thống server có các mục tiêu rất cụ thể: nhận dữ liệu sức khỏe từ thiết bị đeo như nhịp tim, SpO2, nhiệt độ, nhịp thở và ECG; chuẩn hóa và lưu trữ dữ liệu vào MongoDB; phân tích ngưỡng để tạo cảnh báo tự động; đồng thời cung cấp API cho app/web truy vấn dữ liệu realtime và dữ liệu lịch sử. Ngoài luồng từ thiết bị lên server, hệ thống còn có luồng điều khiển ngược từ app đến thiết bị thông qua command ECG on-demand, cho thấy đây không phải hệ thống một chiều đơn thuần mà là hệ thống tương tác hai chiều giữa phần mềm và phần cứng.

So với các mô hình IoT cơ bản chỉ dừng ở việc gửi số liệu cảm biến lên cloud, hệ thống của đề tài có mức hoàn thiện cao hơn ở vài điểm. Thứ nhất, backend đã có tách biệt rõ giữa app/admin và thiết bị, mỗi bên dùng một hình thức xác thực riêng. Thứ hai, hệ thống có quản lý phiên người dùng bằng JWT và refresh token, thay vì chỉ cấp một access token đơn giản. Thứ ba, hệ thống có command queue cho ECG và background recovery loop cho command bị timeout. Thứ tư, hệ thống có các endpoint vận hành như /live, /ready, /health, /metrics, đồng thời có Redis-backed rate limit để kiểm soát truy cập. Những đặc điểm đó cho thấy đề tài không chỉ có ý nghĩa mô phỏng mà còn có giá trị như một nguyên mẫu hệ thống thực tế.

Từ góc độ học thuật, hệ thống theo dõi sức khỏe từ xa là một bài toán phù hợp để nghiên cứu nhiều chủ đề cùng lúc, bao gồm: IoT, giao tiếp REST, xác thực nhiều tác nhân, lưu trữ dữ liệu thời gian thực, quản lý phiên người dùng, xử lý dữ liệu sức khỏe và thiết kế ứng dụng di động. Đây cũng là lý do chương cơ sở lý thuyết của báo cáo cần không chỉ trình bày kiến thức nền chung mà còn phải gắn với cách hệ thống đang được hiện thực trong Git.

2.2 Internet of Things trong theo dõi sức khỏe

Internet of Things là mô hình kết nối các đối tượng vật lý với Internet để chúng có thể thu thập, truyền nhận và trao đổi dữ liệu. Trong lĩnh vực chăm sóc sức khỏe, IoT giúp tạo ra các thiết bị đeo hoặc thiết bị giám sát có khả năng ghi nhận liên tục các chỉ số sinh học của cơ thể và truyền dữ liệu về hệ thống xử lý trung tâm. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024; Espressif Systems, 2026)

Việc áp dụng IoT vào theo dõi sức khỏe mang lại nhiều lợi ích. Trước hết, nó cho phép đo lường liên tục hoặc định kỳ mà không phụ thuộc hoàn toàn vào thao tác thủ công của con người. Thứ hai, dữ liệu có thể được lưu trữ tập trung để phục vụ truy vấn, so sánh theo thời gian và phát hiện bất thường. Thứ ba, người thân hoặc người chăm sóc có thể theo dõi tình trạng của bệnh nhân từ xa thông qua ứng dụng.

Về mặt nguyên lý, mô hình IoT trong hệ thống gồm các bước chính:

- Thiết bị đo hoặc mô phỏng dữ liệu sức khỏe.
- Thiết bị đóng gói dữ liệu theo JSON schema chuẩn của backend.
- Thiết bị gửi dữ liệu lên server qua HTTP/HTTPS.
- Backend lưu dữ liệu, phân tích ngưỡng, tạo cảnh báo hoặc phục vụ truy vấn.
- Ứng dụng lấy dữ liệu qua API để hiển thị.
- Khi có yêu cầu đặc biệt như ECG on-demand, app gửi yêu cầu đến server, server tạo command, thiết bị poll và phản hồi kết quả.

Cách tổ chức này phù hợp với hệ thống IoT quy mô nhỏ đến vừa vì đơn giản trong triển khai, ít phụ thuộc vào broker phức tạp và dễ kiểm thử tích hợp.

2.3 Các chỉ số sức khỏe được sử dụng trong hệ thống

Trong hệ thống theo dõi sức khỏe từ xa, các chỉ số sức khỏe đóng vai trò trung tâm vì phản ánh trực tiếp tình trạng cơ bản của người dùng. Theo phạm vi hiện tại, hệ thống xử lý các dữ liệu chính gồm nhịp tim, SpO2, nhiệt độ và ECG. Các dữ liệu này được thiết bị gửi về backend, lưu trữ trong hệ thống và hiển thị trên ứng dụng di động theo thời gian thực, lịch sử hoặc theo các tác vụ chuyên biệt như đo ECG.

Trong đó, nhịp tim, SpO2, nhiệt độ và nhịp thở là các chỉ số sức khỏe cơ bản, hỗ trợ theo dõi trạng thái sinh lý và phát hiện bất thường ở mức nền tảng. Bên cạnh đó, ECG là dữ liệu đặc trưng hơn, phản ánh hoạt động điện của tim theo thời gian và được hệ thống hỗ trợ thu thập, lưu trữ, truy vấn và hiển thị, đặc biệt trong cơ chế đo theo yêu cầu.

Về mặt xử lý, backend thực hiện tiếp nhận, lưu trữ và hỗ trợ cảnh báo theo ngưỡng đối với các chỉ số sức khỏe, trong khi ứng dụng di động đảm nhiệm việc hiển thị dữ liệu thời gian thực, dữ liệu lịch sử và kết quả ECG. Việc lựa chọn các chỉ số này giúp hệ thống phản ánh tình trạng sức khỏe ở mức tương đối toàn diện trong

phạm vi đề tài, đồng thời tạo cơ sở cho việc mở rộng các chức năng phân tích sâu hơn trong tương lai.

2.3.1 Nhịp tim

Nhịp tim là số lần tim đập trong một phút. Đây là một trong những chỉ số sinh tồn cơ bản, thường được dùng để đánh giá tình trạng hoạt động của hệ tim mạch và phản ứng cơ thể đối với vận động, căng thẳng hoặc bệnh lý.

2.3.2 Nồng độ oxy trong máu

SpO₂ là chỉ số phản ánh mức độ bão hòa oxy trong máu ngoại vi. Đây là thông số đặc biệt quan trọng đối với người có bệnh lý hô hấp hoặc tuần hoàn.

2.3.3 Nhiệt độ cơ thể

Nhiệt độ là một chỉ số cơ bản giúp phát hiện sốt, rối loạn thân nhiệt hoặc những biến đổi trạng thái cơ thể.

2.3.4 Nhịp thở

Nhịp thở phản ánh số lần hít thở trong một phút. Đây là chỉ số quan trọng trong đánh giá chức năng hô hấp.

2.3.5 Điện tim

ECG là tín hiệu điện sinh học phản ánh hoạt động điện của tim. So với các chỉ số: app gửi yêu cầu, backend tạo command, thiết bị thực hiện và gửi lại dữ liệu. Vì vậy, ECG là phần dữ liệu nổi bật nhất của đề tài về mặt kiến trúc và luồng xử lý.

2.3.6 Vai trò của các chỉ số trong hệ thống

Trong hệ thống theo dõi sức khỏe từ xa, các chỉ số sức khỏe không chỉ được thu thập và lưu trữ mà còn tham gia trực tiếp vào nhiều chức năng nghiệp vụ quan trọng. Theo thiết kế hiện tại, dữ liệu sức khỏe là đầu vào cho các chức năng hiển thị dữ liệu mới nhất, truy vấn lịch sử, tổng hợp thống kê, sinh cảnh báo theo ngưỡng và hỗ trợ các tác vụ như đo ECG theo yêu cầu.

Trước hết, các chỉ số sức khỏe được sử dụng để hiển thị trạng thái hiện tại của người dùng thông qua dữ liệu mới nhất từ thiết bị. Bên cạnh đó, hệ thống còn hỗ trợ truy vấn dữ liệu lịch sử theo thời gian, giúp người dùng theo dõi sự thay đổi của các chỉ số trong từng giai đoạn khác nhau. Ngoài ra, dữ liệu sức khỏe còn được dùng để tạo summary sức khỏe, bao gồm các giá trị trung bình, nhỏ nhất và lớn nhất của một số chỉ số quan trọng.

Một vai trò quan trọng khác là hỗ trợ cảnh báo theo ngưỡng cấu hình. Sau khi dữ liệu được tiếp nhận và lưu trữ, hệ thống sẽ kiểm tra các ngưỡng cảnh báo

liên quan đến SpO2, nhiệt độ, nhịp tim, nhịp thở và chất lượng ECG để phát hiện bất thường ở mức cơ bản. Đối với ECG, dữ liệu này vừa được lưu trữ và hiển thị như một bản ghi sức khỏe đặc biệt, vừa tham gia vào luồng xử lý đo theo yêu cầu giữa ứng dụng, backend và thiết bị.

2.4 Kiến trúc client-server và REST API

2.4.1 Khái niệm client-server

Kiến trúc client-server là mô hình trong đó client gửi yêu cầu hoặc dữ liệu đến server, còn server đảm nhiệm việc tiếp nhận, xác thực, xử lý nghiệp vụ, truy xuất dữ liệu và trả về kết quả. Mô hình này giúp tách biệt rõ giữa nơi phát sinh yêu cầu và nơi xử lý trung tâm, từ đó tăng tính nhất quán dữ liệu, thuận lợi cho bảo trì, mở rộng và kiểm soát bảo mật. (Fielding, 2000)

Trong các hệ thống web, mobile và IoT, kiến trúc client-server đặc biệt phù hợp vì cho phép nhiều loại tác nhân cùng giao tiếp với một trung tâm xử lý thống nhất. Thay vì mỗi thành phần tự xử lý độc lập, toàn bộ yêu cầu được đưa về server để kiểm tra, chuẩn hóa, lưu trữ và cung cấp lại thông qua các giao diện chung.

Trong phạm vi đề tài, mô hình này được thể hiện rõ qua việc ESP32, ứng dụng Flutter/mobile và công cụ admin đóng vai trò client, trong khi backend FastAPI là server trung tâm. Thiết bị ESP32 gửi dữ liệu sức khỏe và nhận lệnh điều khiển như đo ECG; ứng dụng di động gửi các yêu cầu như đăng nhập, truy vấn thiết bị, dữ liệu mới nhất, lịch sử, cảnh báo và ECG; công cụ admin phục vụ quản trị người dùng, thiết bị và vận hành hệ thống.

Ở phía server, FastAPI tiếp nhận yêu cầu từ các client, thực hiện xác thực theo từng loại tác nhân, xử lý nghiệp vụ, truy cập cơ sở dữ liệu và trả kết quả phù hợp. Bên cạnh đó, MongoDB và Redis là các thành phần hạ tầng hỗ trợ cho server, lần lượt phục vụ lưu trữ dữ liệu và kiểm soát truy cập.

2.4.2 REST API

REST API là phong cách thiết kế giao diện lập trình ứng dụng dựa trên giao thức HTTP, trong đó các tài nguyên được truy cập thông qua các URL định danh rõ ràng và được thao tác bằng các phương thức chuẩn như GET, POST, PATCH, DELETE. Trong mô hình này, client gửi yêu cầu đến đúng endpoint với dữ liệu phù hợp, còn server thực hiện xác thực, xử lý nghiệp vụ, truy xuất dữ liệu và trả về phản hồi chuẩn hóa. Cách tiếp cận này phù hợp với các hệ thống hiện đại vì rõ ràng, dễ mở rộng và thuận lợi cho tích hợp nhiều loại client. (Fielding, 2000)

Trong đề tài này, REST API là cơ chế giao tiếp thống nhất giữa các thành phần của hệ thống. ESP32 sử dụng REST qua HTTPS kết hợp X-Device-Token để gửi dữ liệu sức khỏe và nhận lệnh điều khiển, trong khi ứng dụng

Flutter/mobile/web/admin sử dụng REST qua HTTPS kết hợp Authorization: Bearer <JWT> để thực hiện đăng nhập, truy xuất dữ liệu, quản lý phiên và gửi các yêu cầu nghiệp vụ như đo ECG.

Các endpoint REST của hệ thống được tổ chức thành nhiều nhóm rõ ràng. Thứ nhất là nhóm API cho app và admin dưới tiền tố /api/v1/, bao gồm các chức năng xác thực, quản lý người dùng, quản lý thiết bị, truy vấn dữ liệu sức khỏe, xem cảnh báo và yêu cầu đo ECG. Thứ hai là nhóm API dành cho thiết bị ESP32 dưới tiền tố /api/v1/esp/, phục vụ các tác vụ gửi readings, nhận command tiếp theo và gửi ACK sau khi thực thi. Việc tách riêng nhóm endpoint cho thiết bị giúp kiểm soát xác thực, rate limit và logic xử lý phù hợp với đặc thù của client IoT.

Ngoài các API nghiệp vụ, hệ thống còn cung cấp các endpoint như /live, /ready, /health để kiểm tra trạng thái hoạt động của backend và các phụ thuộc liên quan. Bên cạnh đó, hệ thống cũng hỗ trợ /metrics để phục vụ giám sát theo kiểu Prometheus-style khi được cấu hình cho phép. Điều này cho thấy REST API không chỉ phục vụ trao đổi dữ liệu nghiệp vụ mà còn hỗ trợ quan sát và vận hành hệ thống.

Về nội dung chức năng, REST API hiện hỗ trợ các nhóm chính gồm xác thực (đăng ký, đăng nhập, refresh, logout, lấy thông tin người dùng), quản lý thiết bị và quyền truy cập (đăng ký thiết bị, liên kết thiết bị, quản lý viewers, cập nhật ngưỡng, xoay token ESP), dữ liệu sức khỏe (dữ liệu mới nhất, lịch sử, ECG, summary, alerts) và điều khiển tác vụ như yêu cầu đo ECG hoặc hủy lệnh. Với riêng thiết bị ESP32, các API chính gồm gửi readings, lấy command tiếp theo và gửi ACK, tạo thành một vòng giao tiếp REST khép kín giữa thiết bị và backend.

2.5 Xác thực và quản lý phiên bằng JWT, refresh token

2.5.1 JWT trong hệ thống

JWT là một loại token xác thực thường được sử dụng trong các hệ thống client-server để cho phép người dùng truy cập API. Trong hệ thống này, JWT được áp dụng cho ứng dụng Flutter, web app và admin tool, trong đó client gửi access token qua header Authorization: Bearer <access_token> khi gọi các endpoint nghiệp vụ của backend. (Jones et al., 2015)

Sau khi người dùng đăng nhập thành công, backend cấp access token để client sử dụng trong các request tiếp theo. Token này đóng vai trò là bằng chứng xác thực tạm thời, giúp backend nhận biết người dùng và kiểm soát quyền truy cập vào các tài nguyên của hệ thống. Cách tổ chức này phù hợp với mô hình REST API, vì client chỉ cần đính kèm token trong mỗi request mà không phải duy trì phiên theo cách truyền thống.

Việc sử dụng JWT mang lại ưu điểm như gọn nhẹ, thuận tiện cho giao tiếp HTTP, dễ tích hợp với ứng dụng di động và các route API hiện có. Tuy nhiên, do access token thường có thời gian sống ngắn để tăng tính an toàn, hệ thống không sử dụng JWT một cách độc lập mà kết hợp thêm refresh token để duy trì phiên đăng nhập.

2.5.2 Refresh token

Để khắc phục hạn chế của access token có thời gian sống ngắn, hệ thống sử dụng thêm refresh token nhằm duy trì phiên làm việc của người dùng theo hướng an toàn và thuận tiện hơn. Sau khi đăng nhập thành công, server trả về cả access token và refresh token; khi access token hết hạn, ứng dụng sẽ gọi API refresh để xin cấp lại cặp token mới. (Hardt, 2012; Sheffer et al., 2020)

Trong hệ thống hiện tại, refresh token là một token dạng opaque được sinh ngẫu nhiên, không phải JWT, và được lưu dưới dạng hash trong collection `auth_sessions`. Mỗi refresh token luôn gắn với một phiên đăng nhập cụ thể, giúp backend có thể quản lý, kiểm tra hiệu lực và kiểm soát vòng đời phiên làm việc.

Khi client gửi yêu cầu làm mới phiên, backend sẽ thực hiện xoay vòng refresh token bằng cách kiểm tra token cũ, tạo token mới và cập nhật lại thông tin phiên. Cơ chế này giúp tăng tính bảo mật, hạn chế rủi ro khi token cũ bị lộ hoặc bị tái sử dụng. Trong ứng dụng Flutter, luồng refresh cũng được tích hợp vào quá trình vận hành: nếu access token hết hạn, app sẽ tự động thử refresh; nếu thành công thì tiếp tục gửi lại request, còn nếu thất bại thì người dùng phải đăng nhập lại.

2.5.3 Quản lý session ở phía Flutter

Ở phía Flutter, session không chỉ là trạng thái “đã có token” mà được tổ chức như một trạng thái xác thực đầy đủ, bao gồm access token, refresh token, thông tin người dùng hiện tại, vai trò truy cập và các trạng thái phục vụ giao diện. Trong thiết kế hiện tại, `SessionProvider` là thành phần trung tâm quản lý phiên, phối hợp với `AuthApiService` để giao tiếp với backend, `AuthStorage` để lưu dữ liệu cục bộ và `AuthTokens` để biểu diễn cặp token xác thực.

Khi ứng dụng khởi động, session được khôi phục thông qua cơ chế bootstrap. Ứng dụng đọc token đã lưu, tải lại thông tin người dùng và đồng bộ dữ liệu hiện tại từ backend nếu phiên còn hiệu lực. Khi người dùng đăng nhập, app không chỉ nhận token mà còn lấy thêm thông tin người dùng để cập nhật đầy đủ trạng thái xác thực.

Ở tầng lưu trữ, session được duy trì qua các lần mở ứng dụng nhờ việc lưu access token và refresh token vào bộ nhớ cục bộ. Khi access token hết hạn, ứng dụng sẽ tự động dùng refresh token để xin cấp token mới; nếu thành công thì cập nhật lại phiên và tiếp tục request, còn nếu thất bại thì xóa toàn bộ session và đưa người dùng về trạng thái chưa đăng nhập.

Ngoài chức năng xác thực, session còn ảnh hưởng trực tiếp đến giao diện và quyền truy cập trong ứng dụng, chẳng hạn quyết định hiển thị màn hình đăng nhập, danh sách thiết bị hay các chức năng dành riêng cho quản trị viên. Như vậy, quản lý session ở phía Flutter được tổ chức khá đầy đủ, bảo đảm cả tính liên tục của đăng nhập, khả năng đồng bộ với backend và hỗ trợ điều khiển giao diện theo trạng thái người dùng.

2.5.4 Ý nghĩa của mô hình xác thực hiện tại

Bên cạnh cơ chế xác thực người dùng, hệ thống còn sử dụng Device Token để xác thực riêng cho thiết bị ESP32. Đây là cách tiếp cận phù hợp với bài toán IoT, vì thiết bị nhúng không thể áp dụng mô hình đăng nhập tương tác như người dùng trên ứng dụng. Trong hệ thống hiện tại, ESP32 gửi các request đến backend kèm header X-Device-Token, trong khi ứng dụng người dùng sử dụng JWT Bearer, cho thấy hai luồng xác thực được tách biệt rõ ràng.

Device Token được dùng khi thiết bị gửi dữ liệu sức khỏe, nhận lệnh điều khiển hoặc gửi phản hồi thực thi lệnh. Backend sẽ kiểm tra token này trước khi cho phép thiết bị truy cập các API dành riêng cho lớp IoT. Để tăng tính an toàn, token thiết bị không được lưu ở dạng thô trong cơ sở dữ liệu mà được băm (hash) trước khi lưu và đối chiếu khi xác thực. Cơ chế này giúp giảm rủi ro nếu dữ liệu lưu trữ bị lộ.

Việc xác thực bằng Device Token giúp bảo đảm chỉ các thiết bị hợp lệ mới có thể gửi dữ liệu hoặc nhận lệnh từ hệ thống, đồng thời ngăn chặn các client lạ giả mạo thiết bị. Bên cạnh đó, cơ chế này cũng thuận lợi cho việc xoay token theo từng thiết bị khi cần thay đổi cấu hình hoặc xử lý rủi ro bảo mật.

Nhìn chung, xác thực thiết bị bằng Device Token là một thành phần quan trọng trong kiến trúc bảo mật của hệ thống. Cách tổ chức này vừa phù hợp với đặc thù của thiết bị nhúng, vừa giúp backend kiểm soát đúng nguồn dữ liệu từ lớp IoT trong hệ thống theo dõi sức khỏe từ xa.

2.6 Xác thực thiết bị bằng Device Token

Ngoài xác thực người dùng, hệ thống còn phải xác thực thiết bị ESP32. Đây là một yêu cầu đặc thù của hệ thống IoT vì thiết bị không thể đăng nhập theo cùng mô hình như người dùng trên app.

Theo backend hệ thống, API cho ESP bắt buộc dùng header X-Device-Token, và token thiết bị được hash theo công thức sha256(secret:token) trước khi lưu vào cơ sở dữ liệu. Việc không lưu token thô là một điểm bảo mật tốt, giúp hạn chế rủi ro nếu database bị lộ.

Cơ chế xác thực thiết bị có vài ý nghĩa quan trọng:

- bảo đảm chỉ thiết bị hợp lệ mới được gửi readings,

- ngăn chặn client lạ giả mạo thiết bị,
- tách riêng luồng bảo mật của thiết bị với luồng user/admin,
- thuận lợi cho việc rotate token theo từng thiết bị.

So với JWT, device token đơn giản hơn, dễ triển khai cho firmware ESP32 và đủ phù hợp trong môi trường thiết bị nhúng. Đây là một quyết định thiết kế hợp lý với bài toán của đề tài.

2.7 Phân quyền RBAC trong hệ thống

RBAC là mô hình phân quyền dựa trên vai trò, trong đó quyền truy cập của người dùng được xác định theo vai trò được gán. Trong các hệ thống theo dõi sức khỏe từ xa, cơ chế này giúp kiểm soát truy cập phù hợp giữa các nhóm người dùng khác nhau. (Sandhu et al., 2000)

Tuy nhiên, trong hệ thống hiện tại của đề tài, cơ chế phân quyền không còn thuần túy dựa trên các vai trò như admin, caregiver, patient, mà đã chuyển sang mô hình device-centric. Theo đó, quyền truy cập dữ liệu hằng ngày được xác định chủ yếu dựa trên liên kết thiết bị (device_links), với hai mức quyền chính là owner và viewer. Trong khi đó, vai trò admin vẫn được giữ lại ở mức quản trị hệ thống, phục vụ các chức năng như khởi tạo người dùng, đăng ký thiết bị, xoay token ESP và vận hành hệ thống.

Cụ thể, owner là tài khoản sở hữu thiết bị, có quyền đọc dữ liệu, quản lý người được chia sẻ, chỉnh ngưỡng cảnh báo, gửi yêu cầu ECG, hủy lệnh và xoay token thiết bị. Viewer là tài khoản được cấp quyền theo dõi dữ liệu của thiết bị nhưng không có các quyền quản trị như owner. Cách tổ chức này phù hợp hơn với thực tế sử dụng, vì quyền được gán theo từng thiết bị thay vì chỉ dựa trên vai trò chung của tài khoản.

Hệ thống có yếu tố RBAC ở mức quản trị với vai trò admin, nhưng quyền truy cập dữ liệu sức khỏe trong vận hành hằng ngày được triển khai chủ yếu theo mô hình phân quyền dựa trên liên kết thiết bị. Cách tiếp cận này phản ánh đúng kiến trúc thực tế của hệ thống và bảo đảm kiểm soát truy cập linh hoạt hơn.

2.8 FastAPI và lý do lựa chọn

2.8.1 Tổng quan về FastAPI

FastAPI được lựa chọn làm nền tảng backend vì hỗ trợ xây dựng REST API nhanh, kiểm soát tốt dữ liệu đầu vào và phù hợp với hệ thống có nhiều loại client như ứng dụng Flutter, thiết bị ESP32 và công cụ quản trị. Trong phạm vi đề tài, FastAPI chủ yếu đóng vai trò lớp giao tiếp HTTP và điều phối nghiệp vụ, nên báo cáo chỉ trình bày ở mức vừa đủ để làm rõ vai trò của backend trong toàn hệ thống.

2.8.2 Vai trò của service layer

Lớp service được tổ chức để tách route API khỏi logic xử lý readings, cảnh báo, command và phân quyền. Cách tổ chức này giúp backend dễ bảo trì, thuận lợi cho kiểm thử và bảo đảm phần mềm phục vụ trực tiếp cho bài toán tích hợp phần cứng – phần mềm, thay vì sa đà vào chi tiết cài đặt của framework.

2.9 MongoDB và lý do lựa chọn

2.9.1 Tổng quan về MongoDB

MongoDB được dùng làm cơ sở dữ liệu chính vì phù hợp với dữ liệu readings có cấu trúc linh hoạt, nhiều trường lồng nhau và cần truy vấn theo thời gian. Việc lựa chọn MongoDB giúp hệ thống lưu trữ đồng thời thông tin người dùng, thiết bị, cảnh báo và bản ghi sức khỏe trong cùng một nền tảng.

2.9.2 Tổ chức dữ liệu trong MongoDB

Dữ liệu được tổ chức theo các collection chính như users, devices, device_links, health_readings, alerts, device_commands và auth_sessions. Cách phân tách này bám sát các luồng nghiệp vụ thật của hệ thống nhưng trong báo cáo chỉ trình bày ở mức đủ để làm rõ quan hệ giữa thiết bị, backend và ứng dụng.

2.9.3 Index và TTL

Hệ thống có áp dụng index cho truy vấn theo thiết bị và thời gian, đồng thời sử dụng TTL cho một số bản ghi tạm thời nhằm hỗ trợ dọn dữ liệu cũ. Các cấu hình chi tiết của chỉ mục không phải trọng tâm của báo cáo, nhưng chúng cho thấy backend đã được tổ chức có tính đến khả năng vận hành thực tế.

2.10 Redis và vai trò trong hệ thống

Redis được dùng như lớp hỗ trợ để rate limit và bảo vệ API, không giữ dữ liệu sức khỏe cốt lõi. Nhờ đó backend kiểm soát được tần suất truy cập của app và thiết bị mà không làm phức tạp luồng chính của hệ thống.

2.11 Docker và Docker Compose

2.11.1 Docker

Docker được dùng để đóng gói backend và chuẩn hóa môi trường chạy giữa phát triển, kiểm thử và triển khai, giúp nguyên mẫu không phụ thuộc vào cấu hình riêng của từng máy.

2.11.2 Docker Compose

Docker Compose giúp khởi động đồng thời backend, MongoDB, Redis và các dịch vụ phụ trợ theo cùng một cấu hình, tạo điều kiện để nguyên mẫu được dựng lại và kiểm thử nhất quán.

2.11.3 Vai trò trong đề tài

Trong đề tài, Docker Compose có giá trị chủ yếu ở góc độ triển khai nguyên mẫu và minh chứng khả năng vận hành, thay vì là phần công nghệ cần phân tích quá sâu.

2.11.4 Cloudflare Tunnel và Nginx

Hệ thống có phương án công bố dịch vụ ra ngoài thông qua tunnel hoặc reverse proxy để app và thiết bị truy cập khi kiểm thử từ xa. Phần này chỉ được trình bày ở mức khái quát vì không phải trọng tâm chính của luận văn.

2.12 Flutter và lý do lựa chọn

2.12.1 Tổng quan về Flutter

Flutter được chọn để xây dựng ứng dụng di động vì hỗ trợ giao diện thống nhất, kết nối tốt với REST API và phù hợp cho việc theo dõi dữ liệu sức khỏe trên điện thoại.

2.12.2 Vai trò của Flutter trong hệ thống

Trong hệ thống, Flutter đóng vai trò lớp tương tác người dùng: đăng nhập, đồng bộ thiết bị, xem dữ liệu mới nhất, dữ liệu lịch sử, cảnh báo và gửi yêu cầu ECG. Báo cáo tập trung vào vai trò và luồng sử dụng, không đi quá sâu vào từng chi tiết triển khai giao diện.

2.12.3 Kiến trúc ứng dụng Flutter, quản lý trạng thái

Ứng dụng được tổ chức theo hướng tách lớp giao tiếp API, lưu trữ cục bộ và quản lý trạng thái để thuận lợi cho bảo trì. Mức mô tả trong báo cáo nhằm làm rõ sự phối hợp với backend, không nhằm trình bày toàn bộ chi tiết framework.

2.13 Cơ chế command queue cho ECG on-demand

Cơ chế ECG on-demand được hiện thực bằng collection `device_commands` trong MongoDB. Đây là giải pháp phù hợp cho nguyên mẫu vì dễ tích hợp với backend hiện có, đủ phục vụ luồng app → backend → thiết bị → phản hồi kết quả mà không cần bổ sung thêm hạ tầng hàng đợi phức tạp.

2.14 Các nguyên tắc bảo mật cơ bản được áp dụng

Hệ thống áp dụng các nguyên tắc bảo mật cơ bản gồm tách xác thực người dùng và thiết bị, băm token trước khi lưu, giới hạn truy cập và cấu hình CORS. Những biện pháp này giúp bảo vệ dữ liệu sức khỏe ở mức phù hợp với một nguyên mẫu học thuật.

2.15 Logging, health check và giám sát hệ thống

Backend có health check, logging và chỉ số vận hành cơ bản để phục vụ kiểm thử và chẩn đoán lỗi. Mức trình bày này cho thấy hệ thống có khả năng vận hành thử nghiệm ổn định mà không cần sa đà vào các công nghệ observability chuyên sâu.

2.16 Nghiên cứu liên quan

Bên cạnh cơ sở lý thuyết và công nghệ sử dụng, việc đặt đề tài vào bối cảnh các nghiên cứu liên quan là cần thiết để làm rõ điểm kế thừa và phần đóng góp riêng của hệ thống. Các công trình gần đây về theo dõi sức khỏe từ xa có thể chia thành ba hướng chính: (i) hệ thống chi phí thấp dựa trên ESP32 và web server; (ii) các framework RPM tích hợp cloud/edge và cơ chế mở rộng; và (iii) các nền tảng tập trung mạnh vào trực quan hóa và trải nghiệm sử dụng cho người theo dõi hoặc nhân viên y tế.

2.16.1 Nhóm hệ thống chi phí thấp dựa trên ESP32

Trong nhóm hệ thống chi phí thấp, Ferdous và cộng sự (2023) xây dựng một hệ thống theo dõi sức khỏe dựa trên ESP32 web server để thu thập nhịp tim, nhiệt độ và SpO₂, truyền dữ liệu qua Wi-Fi và hiển thị trên giao diện web. Công trình này cho thấy ưu điểm lớn của kiến trúc gọn nhẹ, dễ triển khai và phù hợp với môi trường giám sát tại nhà. Tuy nhiên, cách tổ chức vẫn thiên về web server cục bộ và chưa đi sâu vào backend dịch vụ tách biệt, quản lý phiên đăng nhập, phân quyền truy cập theo thiết bị hay luồng điều khiển hai chiều giữa ứng dụng và thiết bị (Ferdous et al., 2023).

Rahman và cộng sự (2023) tiếp cận bài toán ở mức cao hơn khi tích hợp ECG, độ bão hòa oxy và nhịp tim với ESP32, sử dụng Firebase làm cơ sở dữ liệu thời gian thực và một ứng dụng di động để nhân viên y tế theo dõi bệnh nhân. Điểm mạnh của công trình là chứng minh được đường đi dữ liệu cảm biến → cloud realtime → mobile app đối với các tín hiệu sức khỏe quan trọng. Tuy nhiên, hệ thống vẫn chưa nhấn mạnh các vấn đề như kiểm soát quyền owner/viewer theo thiết bị, observability, khả năng triển khai có cấu trúc bằng Docker hay command queue cho tác vụ đo ECG theo yêu cầu (Rahman et al., 2023).

2.16.2 Nhóm hệ thống RPM tích hợp cloud/edge và AI

Palanisamy và cộng sự (2023) đề xuất một hệ thống theo dõi hoạt động bệnh nhân từ xa dựa trên wearable IoT, cloud database, Android app và GPS,

đồng thời tích hợp mô hình CNN-UUGRU cho bài toán nhận dạng hoạt động. Công trình này nổi bật ở việc kết hợp cả dữ liệu sinh hiệu, dữ liệu vị trí và kỹ thuật AI, với độ chính xác được báo cáo ở mức 97,7%, precision 96,8% và F-measure 97,75% cho tác vụ nhận dạng hoạt động. Dù vậy, bối cảnh nghiên cứu nghiêng mạnh về giám sát bệnh nhân liên quan COVID/quarantine và phân tích activity hơn là xây dựng một nguyên mẫu cân bằng giữa firmware thiết bị, backend API và ứng dụng chăm sóc gia đình theo device scope (Palanisamy et al., 2023).

Alasmary (2024) đưa ra ScalableDigitalHealth (SDH), một framework theo dõi bệnh nhân từ xa có khả năng mở rộng, hỗ trợ ECG, nhiệt độ, huyết áp và SpO2, đồng thời kết hợp edge computing với cơ chế autoscaling theo độ trễ cho các dịch vụ container hóa. Đây là hướng tiếp cận mạnh về kiến trúc cloud/edge và vận hành ở quy mô lớn. Tuy nhiên, so với phạm vi một đồ án kỹ thuật phần mềm, SDH sử dụng hạ tầng nặng hơn đáng kể và không đặt trọng tâm vào mô hình phân quyền owner/viewer, quản lý thiết bị theo device-centric scope hay command queue ECG on-demand ở mức nguyên mẫu tích hợp giữa ESP32, backend FastAPI và ứng dụng Flutter (Alasmary, 2024).

2.16.3 Nhóm hệ thống tập trung vào trực quan hóa và trải nghiệm giám sát

Arun, Sykes và Tanbeer (2024) giới thiệu RemoteHealthConnect, một nền tảng web tích hợp wearable Vitaliti để theo dõi năm chỉ số sống còn cùng ECG, PPG và accelerometer theo thời gian thực. Công trình này đáng chú ý ở các dashboard chuyên biệt cho bác sĩ, khả năng trực quan hóa waveform, radial chart 24 giờ và kết quả đánh giá khả dụng SUS đạt 71,5. Điểm hạn chế khi so với đề tài là hệ thống phụ thuộc vào wearable thương mại và bối cảnh sử dụng thiên về clinician dashboard, trong khi đề tài hiện tại theo đuổi định hướng low-cost ESP32 + backend FastAPI + app Flutter để phù hợp hơn với mô hình gia đình và người chăm sóc tại nhà (Arun et al., 2024).

2.17 Bảng so sánh đề tài với các hệ thống liên quan

Bảng 2.1. So sánh một số hệ thống theo dõi sức khỏe từ xa tiêu biểu với đề tài.

Công trình / hệ thống	Kiến trúc và dữ liệu chính	Điểm mạnh nổi bật	Khoảng trống so với đề tài
Ferdous et al. (2023)	ESP32 web server; nhịp tim, nhiệt độ, SpO2; giao diện web.	Chi phí thấp, cấu trúc gọn, dễ triển khai nhanh trong môi trường thử nghiệm.	Chưa tách backend dịch vụ; chưa có quản lý phiên, phân quyền theo thiết bị hoặc cơ chế điều khiển hai chiều.
Rahman et al. (2023)	ESP32 ECG/SpO2/HR; Firebase + realtime	Thể hiện tốt luồng cảm biến → cloud realtime → ứng dụng di động.	Chưa đi sâu vào observability, device-centric access control,

	database; mobile app cho nhân viên y tế.		Docker deployment và ECG on-demand có command queue.
Palanisamy et al. (2023)	Wearable IoT + cloud + Android app + GPS; AI nhận dạng hoạt động.	Mở rộng sang phân tích activity và cảnh báo theo vị trí, có báo cáo chỉ số accuracy/precision/F-measure.	Bối cảnh nghiêng về COVID/quarantine và activity AI; chưa cân bằng giữa firmware IoT, backend API và app chăm sóc gia đình.
Alasmary (2024)	Framework cloud/edge có autoscaling; theo dõi ECG, nhiệt độ, huyết áp, SpO2.	Mạnh về khả năng mở rộng, latency-aware autoscaling và triển khai container hóa.	Hạ tầng nặng hơn phạm vi đồ án; không tập trung vào device scope owner/viewer và ECG on-demand mức nguyên mẫu.
Arun et al. (2024)	Wearable thương mại + web dashboard; 5 vital signs + ECG/PPG/accelerometer.	Trực quan hóa mạnh, có đánh giá khả dụng SUS 71,5 và hỗ trợ xem waveform thời gian thực.	Phụ thuộc wearable thương mại; thiên về clinician dashboard hơn là mô hình low-cost ESP32 + FastAPI + Flutter.
Đề tài hiện tại	ESP32 + FastAPI + MongoDB/Redis + Flutter; HR, SpO2, nhiệt độ, nhịp thở, ECG; Docker Compose.	Cân bằng giữa chi phí, khả năng triển khai, phân quyền theo thiết bị, observability cơ bản và ECG on-demand.	Chưa có benchmark tải lớn, chưa kiểm định sai số y khoa chuyên sâu và command queue vẫn ở mức Mongo polling.

2.18 Khoảng trống nghiên cứu và định hướng đóng góp của đề tài

Từ các công trình liên quan có thể thấy một khoảng trống đáng chú ý: nhiều hệ thống hoặc tập trung vào phần cứng chi phí thấp nhưng backend còn đơn giản, hoặc tập trung vào cloud/visualization ở quy mô lớn nhưng khó áp dụng cho một nguyên mẫu triển khai nhanh, chi phí thấp và phù hợp với bối cảnh chăm sóc gia đình. Khoảng trống này đặc biệt rõ ở bốn điểm: thiếu phân quyền theo thiết bị, thiếu backend tách biệt có quản lý phiên bản, thiếu cơ chế điều khiển tác vụ đo on-demand, và thiếu sự kết nối chặt chẽ giữa firmware – API – mobile app trong cùng một nguyên mẫu.

Đề tài hiện tại được định vị để lấp khoảng trống đó ở mức học thuật – thực nghiệm. Cụ thể, hệ thống lựa chọn ESP32 làm nền tảng thiết bị, FastAPI làm backend trung tâm, MongoDB và Redis cho dữ liệu – vận hành, Flutter làm ứng dụng người dùng, đồng thời triển khai mô hình device-centric với owner/viewer, command queue cho ECG on-demand, health check/metrics/rate limit và khả năng dựng hệ thống bằng Docker Compose. Cách tiếp cận này không cạnh tranh trực tiếp với các nền tảng cloud-scale hay wearable thương mại, nhưng tạo ra một nguyên mẫu cân bằng, có chiều sâu kỹ thuật và phù hợp với phạm vi của đồ án tốt nghiệp kỹ thuật phần mềm.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU, THIẾT KẾ VÀ KIẾN TRÚC HỆ THỐNG

3.1 Mục tiêu và định hướng của chương

Chương này trình bày toàn bộ phần phân tích, thiết kế và kiến trúc của hệ thống Eldercare theo một cấu trúc thống nhất, trong đó phần cứng và phần mềm được xem là hai nửa không thể tách rời của cùng một bài toán. Nếu chỉ mô tả backend, cơ sở dữ liệu và ứng dụng di động mà không làm rõ cách thiết bị thật thu thập dữ liệu, báo cáo sẽ lệch sang phía công nghệ phần mềm. Ngược lại, nếu chỉ mô tả sơ đồ mạch, cảm biến và bo ESP32 mà thiếu kiến trúc xử lý trung tâm, dữ liệu sức khỏe thu được cũng không thể hình thành nên một hệ thống theo dõi từ xa hoàn chỉnh. Vì vậy, chương này được tổ chức theo hướng cân bằng: cùng một lúc giải thích thiết bị đo hoạt động như thế nào, dữ liệu đi qua backend ra sao, và ứng dụng Flutter tiêu thụ dữ liệu đó như thế nào.

Về phạm vi, chương không đi quá sâu vào từng chi tiết cài đặt mã nguồn hoặc từng thành phần giao diện nhỏ lẻ. Thay vào đó, chương tập trung vào các quyết định kiến trúc, các sơ đồ tổng quát, các quan hệ dữ liệu chính, các luồng nghiệp vụ lớn và cách các khối thiết bị được lắp ghép thành một nguyên mẫu hoàn chỉnh. Những sơ đồ được lựa chọn trong chương cũng theo tinh thần đó: use case, activity và sequence chỉ giữ ở mức tổng quát để làm rõ bản chất hệ thống, không sa đà vào chi tiết quá mịn gây dần trải nội dung.

Mục tiêu cuối cùng của chương là chứng minh rằng đề tài Eldercare không phải là tập hợp rời rạc của một bo ESP32, một REST API và một ứng dụng di động, mà là một hệ thống IoT theo dõi sức khỏe từ xa được thiết kế có chủ đích. Từ các chỉ số sinh hiệu đo được ở bo đeo tay, qua quá trình hợp nhất dữ liệu ở bo gateway, đến bước lưu trữ – cảnh báo tại backend và hiển thị ở ứng dụng, toàn bộ chuỗi xử lý được tổ chức như một kiến trúc thống nhất. Đây chính là trọng tâm học thuật của chương và cũng là cơ sở để các chương sau nói về triển khai, kiểm thử và đánh giá hệ thống.

3.2 Bối cảnh bài toán và yêu cầu hệ thống

Bài toán mà đề tài đặt ra xuất phát từ nhu cầu theo dõi sức khỏe người cao tuổi theo hướng liên tục, thuận tiện và có khả năng quan sát từ xa. Trong thực tế chăm sóc tại nhà, các chỉ số như nhịp tim, độ bão hòa oxy trong máu, nhiệt độ cơ thể, chuyển động bất thường và điện tim đều có giá trị cảnh báo sớm. Tuy nhiên, việc đo thủ công theo từng thời điểm riêng lẻ thường làm dữ liệu bị rời rạc, khó theo dõi xu hướng và không tạo được cơ chế phản hồi kịp thời. Một hệ thống theo dõi sức khỏe đúng nghĩa phải giải quyết đồng thời ba nhu cầu: thu nhận tín hiệu vật lý, xử lý – lưu trữ – cảnh báo dữ liệu, và cung cấp cho người dùng một giao diện đủ trực quan để hiểu tình trạng hiện tại lẫn lịch sử vận hành.

Trong khuôn khổ đề tài này, bài toán được cụ thể hóa thành một hệ thống nhiều lớp. Ở lớp thiết bị, nguyên mẫu sử dụng hai bo ESP32 phối hợp: bo ESP32-C3 làm nhiệm vụ đọc nhịp tim, SpO2 và nhiệt độ, còn bo ESP32-S3 làm gateway nhận BLE, đọc thêm tín hiệu ECG và gia tốc, hiển thị trạng thái qua màn hình TFT, đồng thời gửi readings tổng hợp lên backend qua Wi-Fi. Ở lớp trung tâm, backend FastAPI chịu trách nhiệm xác thực, tiếp nhận dữ liệu, lưu trữ vào MongoDB, áp dụng Redis cho

rate limit và xử lý các nghiệp vụ như sinh cảnh báo hay điều phối ECG on-demand. Ở lớp người dùng, ứng dụng Flutter đóng vai trò truy cập dữ liệu, xem latest/history/alerts và gửi yêu cầu tác vụ.

Để hệ thống hoạt động đúng với bối cảnh chăm sóc sức khỏe từ xa, yêu cầu đặt ra không chỉ là “đọc được dữ liệu”. Hệ thống còn phải cho phép theo dõi dữ liệu theo đúng thiết bị được cấp quyền, duy trì phiên đăng nhập đủ ổn định, bảo đảm đường đi dữ liệu từ phần cứng sang phần mềm không bị đứt gãy và cho phép người dùng chủ động tạo yêu cầu đo ECG từ ứng dụng thay vì chỉ chờ thiết bị gửi readings định kỳ. Đó là lý do chương này xem xét đồng thời cả yêu cầu chức năng lẫn yêu cầu phi chức năng.

3.2.1 Yêu cầu chức năng của hệ thống

Nhóm yêu cầu chức năng đầu tiên thuộc về lớp thiết bị. Thiết bị phải có khả năng đọc dữ liệu sinh hiệu từ cảm biến quang học MAX30102, nhiệt độ từ DS18B20, dữ liệu ECG từ AD8232 và chuyển động từ MPU6050. Ngoài nhiệm vụ đo, thiết bị còn phải đóng gói dữ liệu theo cấu trúc thống nhất để backend có thể tiếp nhận. Đề tài cũng yêu cầu bo ESP32-S3 hiển thị tình trạng hiện tại lên màn hình TFT để người vận hành có thể kiểm tra nhanh ngay trên phần cứng mà không cần lúc nào cũng mở ứng dụng.

Nhóm yêu cầu chức năng thứ hai thuộc về backend. Backend phải tiếp nhận readings, xác thực danh tính thiết bị, lưu dữ liệu sức khỏe, cung cấp dữ liệu mới nhất và dữ liệu lịch sử cho ứng dụng, đồng thời tạo cảnh báo khi readings vượt ngưỡng cấu hình. Hệ thống còn phải cho phép tạo lệnh điều khiển từ phía ứng dụng xuống thiết bị, cụ thể là yêu cầu đo ECG on-demand. Điều này khiến backend đóng vai trò điều phối hai chiều thay vì chỉ là nơi lưu dữ liệu một chiều.

Nhóm yêu cầu chức năng thứ ba thuộc về ứng dụng Flutter. Ứng dụng phải hỗ trợ người dùng đăng nhập, duy trì phiên làm việc, đồng bộ danh sách thiết bị, chọn thiết bị hiện tại, hiển thị latest, history, alerts và ECG. Ứng dụng còn phải cho phép gửi yêu cầu ECG on-demand và phản ánh các trạng thái như chưa có thiết bị, chưa có dữ liệu, không có quyền truy cập hoặc phiên đăng nhập hết hạn. Đây là nhóm chức năng giúp hệ thống trở thành công cụ giám sát thực sự đối với người dùng cuối.

Bảng 3.1. Ma trận yêu cầu chức năng theo thành phần.

Thành phần	Nhóm chức năng chính	Ý nghĩa trong hệ thống
Thiết bị ESP32-C3	Đọc MAX30102 và DS18B20; gửi BLE	Tạo dữ liệu sinh hiệu đầu vào cho toàn hệ thống
Thiết bị ESP32-S3	Nhận BLE; đọc AD8232, MPU6050; hiển thị TFT; gửi HTTPS	Hợp nhất dữ liệu phần cứng và đóng vai trò gateway
Backend FastAPI	Xác thực; nhận readings; lưu trữ; cảnh báo; điều phối ECG	Lớp trung tâm xử lý nghiệp vụ và tích hợp

MongoDB / Redis	Lưu dữ liệu sức khỏe, session, queue; hỗ trợ rate limit	Duy trì trạng thái dữ liệu và ổn định hệ thống
Ứng dụng Flutter	Đăng nhập; đồng bộ thiết bị; xem latest/history/alerts/ECG	Giao diện giám sát và thao tác cho người dùng

3.2.2 Yêu cầu phi chức năng của hệ thống

Với một hệ thống sức khỏe từ xa, yêu cầu phi chức năng có ý nghĩa không kém yêu cầu chức năng. Trước hết là tính đúng đắn dữ liệu. Thiết bị có thể phải gửi lại gói tin khi mạng chập chờn, do đó backend cần có cơ chế chống trùng lặp readings dựa trên device_id và seq. Tiếp theo là tính bảo mật: quyền đọc dữ liệu không thể chỉ dựa vào việc biết endpoint, mà phải được ràng buộc bởi cơ chế JWT cho ứng dụng, X-Device-Token cho thiết bị và quyền sở hữu hoặc chia sẻ thiết bị trong device_links.

Một yêu cầu phi chức năng khác là tính sẵn sàng và khả năng quan sát vận hành. Backend phải cung cấp các đầu mối như /live, /ready, /health hoặc /metrics để kiểm tra runtime. Ở phía thiết bị, màn hình TFT và log nối tiếp đóng vai trò chỉ báo nhanh trong lúc chạy thử phần cứng. Ở phía ứng dụng, hệ thống cần phân biệt các trạng thái dữ liệu online và dữ liệu cache để người dùng hiểu tình trạng hiện tại của readings.

Cuối cùng, hệ thống phải có khả năng mở rộng ở mức hợp lý đối với một nguyên mẫu học thuật. Điều đó thể hiện ở việc backend được tách thành API, service, db và utils; cơ sở dữ liệu dùng document model linh hoạt; app được tách thành provider, service và storage; còn phần cứng tách thành hai bo để tránh quá tải một node duy nhất. Đây là cơ sở để sau này hệ thống có thể mở rộng thêm loại readings, chính sách command queue hoặc khả năng giám sát vận hành mà không phá vỡ toàn bộ cấu trúc ban đầu.

Bảng 3.2. Yêu cầu phi chức năng của hệ thống.

Yêu cầu	Biểu hiện hiện tại	Giá trị đối với đề tài
Tính đúng đắn dữ liệu	Dedupe readings; truy vấn latest và history theo device	Giảm sai lệch do retransmission và đồng bộ trễ
Bảo mật	JWT, refresh token, X-Device-Token, ownership check	Bảo vệ dữ liệu sức khỏe và phân quyền truy cập
Tính sẵn sàng	Health check, metrics, logging, TFT trạng thái	Hỗ trợ vận hành và kiểm thử nguyên mẫu
Khả năng mở rộng	MongoDB document model, command queue, provider graph	Dễ mở rộng tính năng phần cứng và phần mềm

Tính khả dụng	Empty state, cache, thông báo lỗi, giao diện tại chỗ	Tăng trải nghiệm sử dụng và khả năng kiểm chứng
---------------	--	---

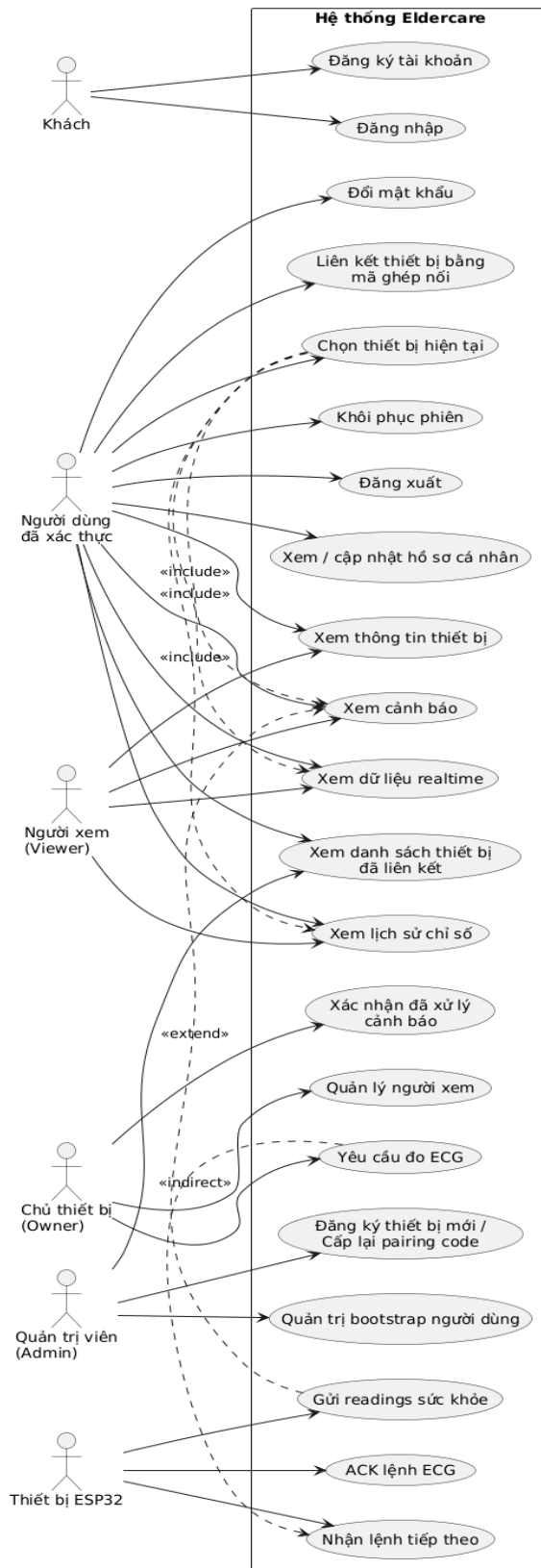
3.2.3 Tác nhân tham gia trong hệ thống

Nếu nhìn ở góc độ tác nhân, hệ thống có bốn nhóm chủ thể chính. Nhóm thứ nhất là người dùng cuối hoặc người chăm sóc, tức người tương tác với ứng dụng Flutter để xem dữ liệu và gửi yêu cầu nghiệp vụ. Nhóm thứ hai là quản trị viên hoặc chủ thiết bị, có thêm trách nhiệm đăng ký thiết bị, liên kết, xoay token hoặc chia sẻ quyền xem cho người khác. Nhóm thứ ba là thiết bị ESP32, được xem như tác nhân kỹ thuật có khả năng gửi dữ liệu và nhận lệnh. Nhóm cuối cùng là backend – cơ sở dữ liệu, tức lớp trung tâm điều phối và duy trì trạng thái của toàn bộ hệ thống.

Điểm đáng chú ý là trong hệ thống này, thiết bị không chỉ là phần cứng bị động. Khi người dùng gửi yêu cầu đo ECG, thiết bị trở thành tác nhân thực thi lệnh. Điều đó làm cho sơ đồ tác nhân của hệ thống không phải kiểu ứng dụng – server truyền thống, mà là mô hình ba chiều giữa người dùng, backend và thiết bị vật lý. Đây cũng là lý do use case tổng quát của đề tài cần thể hiện đồng thời vai trò của người dùng, quản trị viên và thiết bị ESP32.

Bảng 3.3. Tác nhân chính và phạm vi tương tác.

Tác nhân	Vai trò	Tương tác chính
Người dùng / người chăm sóc	Theo dõi sức khỏe từ ứng dụng	Đăng nhập, xem latest/history/alerts, gửi ECG
Quản trị viên / chủ thiết bị	Quản lý thiết bị và quyền truy cập	Đăng ký thiết bị, liên kết, chia sẻ quyền, giám sát
ESP32-C3 / ESP32-S3	Đo dữ liệu và thực thi tác vụ	Gửi readings, nhận lệnh, ACK, hiển thị tại chỗ
Backend + MongoDB/Redis	Điều phối trung tâm	Xác thực, lưu trữ, cảnh báo, trả dữ liệu, queue command



Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống Eldercare.

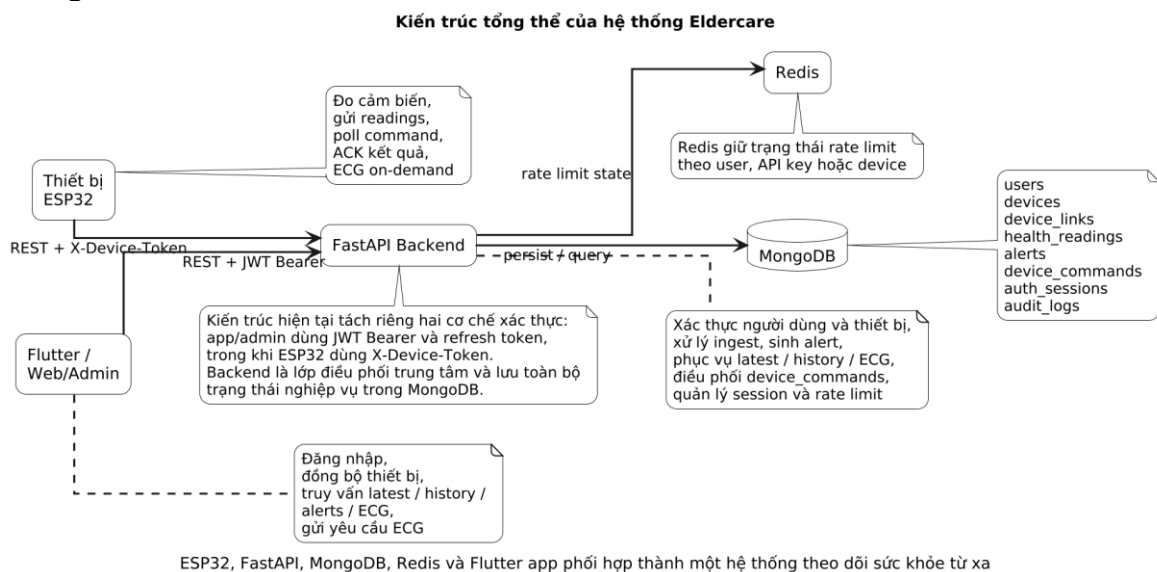
3.3 Kiến trúc tổng thể và nguyên tắc phân lớp

Kiến trúc tổng thể của Eldercare được xây dựng theo hướng trung tâm hóa tại backend. Thiết bị phần cứng không truy cập trực tiếp cơ sở dữ liệu; ứng dụng người dùng cũng không giao tiếp trực tiếp với bo ESP32. Tất cả readings, request và trạng thái đều đi qua backend FastAPI để được xác thực, kiểm tra quyền, chuẩn hóa và lưu trữ. Cách tổ chức này giúp hệ thống duy trì được một trục điều phối thống nhất, đồng thời làm cho dữ liệu sức khỏe không bị phân tán ở nhiều điểm khó kiểm soát.

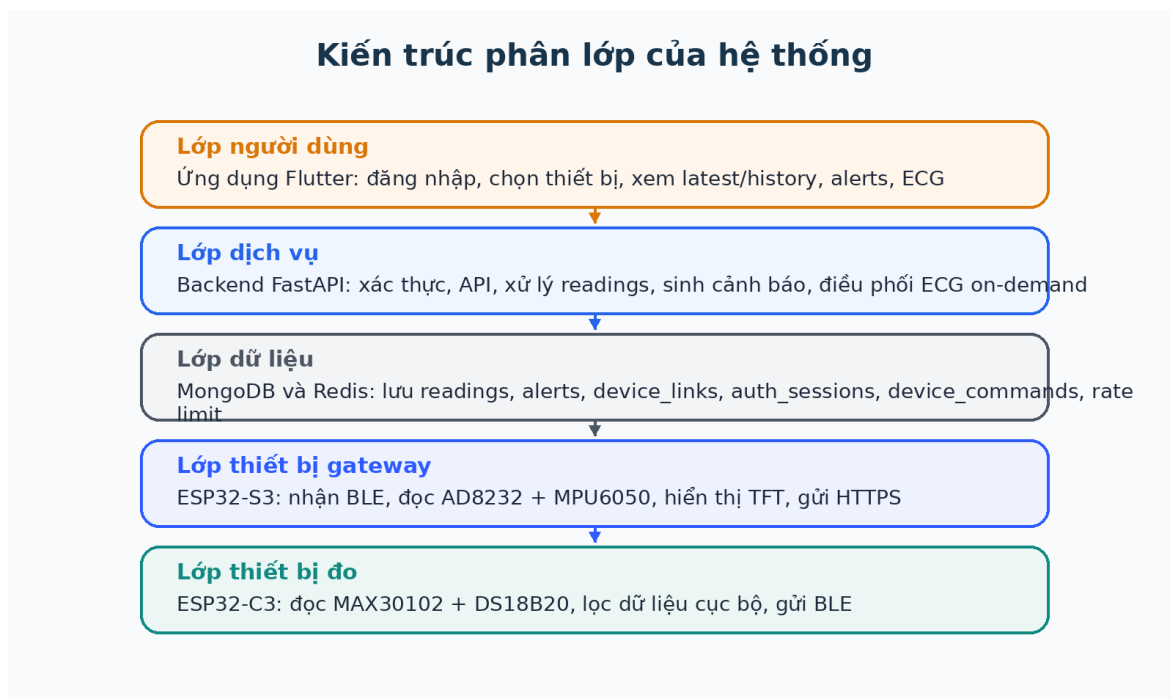
Ở mức giao tiếp, hệ thống có ba kênh chính. Kênh thứ nhất là BLE giữa bo ESP32-C3 và ESP32-S3. Kênh này phục vụ lớp thiết bị cục bộ, nơi bo C3 đóng vai trò nút đo sinh hiệu còn bo S3 là gateway tổng hợp. Kênh thứ hai là Wi-Fi/HTTPS giữa bo ESP32-S3 và backend, bảo đảm dữ liệu đi từ thiết bị vật lý lên lớp dịch vụ trung tâm. Kênh thứ ba là REST API giữa Flutter và backend, nơi người dùng thực hiện đăng nhập, lấy dữ liệu và gửi yêu cầu ECG. Việc phân tách kênh như vậy giúp mỗi kênh được tối ưu theo đúng ngữ cảnh: BLE cho khoảng cách ngắn và tiêu thụ thấp, Wi-Fi cho đồng bộ với server, REST API cho truy cập có kiểm soát từ ứng dụng.

Về logic phân lớp, kiến trúc của Eldercare có thể chia thành năm lớp liên hoàn. Lớp dưới cùng là lớp cảm biến và thiết bị đo, nơi các hiện tượng vật lý như mạch đập, nhiệt độ, chuyển động và điện tim được chuyển thành tín hiệu điện. Lớp thứ hai là lớp gateway và giao tiếp, nơi dữ liệu từ nhiều mô-đun được hợp nhất, kiểm tra trạng thái mạng và chuyển đi qua HTTPS. Lớp thứ ba là lớp dịch vụ backend, chịu trách nhiệm xác thực, lưu trữ, cảnh báo và điều phối lệnh. Lớp thứ tư là lớp dữ liệu, nơi MongoDB và Redis duy trì trạng thái dài hạn hoặc ngắn hạn. Lớp trên cùng là lớp ứng dụng người dùng, chịu trách nhiệm tổ chức giao diện và hành vi sử dụng.

Nguyên tắc phân lớp này giúp báo cáo và cả hệ thống tránh được hai cực đoan thường gặp. Cực đoan thứ nhất là coi mọi thứ đều thuộc về backend, làm phần cứng bị xem nhẹ. Cực đoan thứ hai là chỉ tập trung vào bo mạch và cảm biến mà bỏ qua giá trị của hạ tầng dịch vụ. Ở Eldercare, thiết bị và phần mềm được thiết kế theo quan hệ hỗ trợ lẫn nhau: phần cứng chỉ có ý nghĩa khi backend hiệu và quản lý được dữ liệu, còn backend cũng chỉ có giá trị khi readings phản ánh trạng thái thực của đối tượng theo dõi.



Hình 3.2. Kiến trúc tổng thể phần cứng – phần mềm của hệ thống Eldercare.



Hình 3.3. Kiến trúc phân lớp của hệ thống Eldercare.

3.3.1 Kiến trúc giao tiếp giữa các thành phần

Tách biệt cơ chế giao tiếp là một quyết định quan trọng trong hệ thống. Ứng dụng Flutter sử dụng HTTPS REST cùng Authorization: Bearer <JWT> và refresh token để duy trì phiên. Thiết bị ESP32-S3 cũng sử dụng HTTPS REST, nhưng xác thực bằng X-Device-Token. Trong khi đó, dữ liệu giữa C3 và S3 không đi theo HTTP mà dùng BLE để giảm độ phức tạp ở nút đo. Nhờ vậy, mỗi lớp client được đối xử đúng với đặc thù riêng của nó. Ứng dụng người dùng có thể chấp nhận luồng phiên phức tạp hơn để đổi lấy trải nghiệm đăng nhập mượt mà, trong khi firmware nhúng chỉ cần một cách xác thực gọn nhẹ, ít trạng thái và ít chi phí xử lý.

Cách tổ chức này còn giúp kiểm soát phạm vi lỗi tốt hơn. Nếu BLE giữa hai bo gặp vấn đề, lỗi chủ yếu nằm trong lớp phần cứng cục bộ. Nếu Wi-Fi hoặc HTTPS lên server gián đoạn, bo S3 vẫn có thể hiển thị dữ liệu tại chỗ và cho phép chẩn đoán cục bộ. Nếu ứng dụng Flutter gặp lỗi phiên hoặc lỗi giao diện, backend và thiết bị vẫn có thể tiếp tục chạy readings độc lập. Nói cách khác, mỗi lớp giao tiếp vừa nối các thành phần lại với nhau, vừa đóng vai trò ranh giới cô lập sự cố.

Bảng 3.4. Ma trận giao tiếp và xác thực giữa các thành phần.

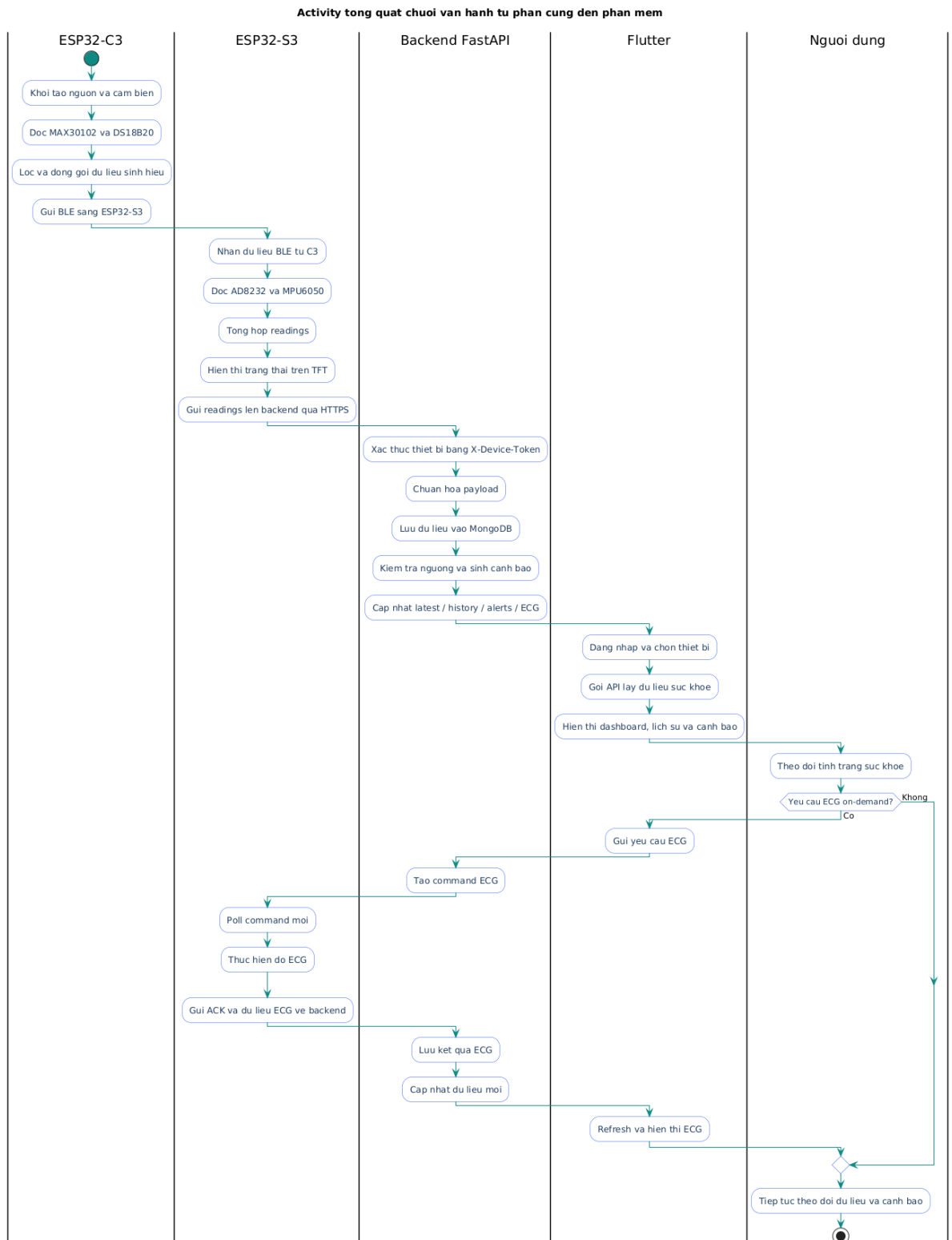
Nguồn	Đích	Giao thức / kênh	Xác thực	Mục đích
ESP32-C3	ESP32-S3	BLE	Ghép cặp cục bộ	Gửi dữ liệu HR,

				SpO2, nhiệt độ
ESP32-S3	Backend	Wi-Fi / HTTPS REST	n X-Device-Token	Gửi readings, nhận command, ACK
Flutter App	Backend	HTTP S REST	JWT + refresh token	Đăng nhập, truy vấn dữ liệu, yêu cầu ECG
Backend	MongoDB / Redis	Kết nối nội bộ	Nội bộ hệ thống	Lưu trữ, queue, session, rate limit

3.3.2 Activity tổng quát của hệ thống

Nếu rút gọn toàn bộ hệ thống thành một activity tổng quát, luồng vận hành cơ bản luôn bắt đầu từ thiết bị. Bo C3 đọc sinh hiệu cơ bản, sau đó gửi BLE sang bo S3. Bo S3 vừa tổng hợp dữ liệu này, vừa thu thêm ECG và chuyển động, hiển thị tại chỗ rồi gửi gói readings lên backend. Backend xác thực, lưu dữ liệu, sinh cảnh báo khi cần và cung cấp dữ liệu cho ứng dụng. Ứng dụng cuối cùng là nơi người dùng quan sát kết quả và tạo thêm yêu cầu nghiệp vụ như ECG on-demand.

Điểm cốt lõi ở activity tổng quát không phải là số lượng bước, mà là vòng kín dữ liệu được giữ nguyên từ đầu đến cuối. Cảm biến không dừng ở mức đo; readings không dừng ở mức lưu; ứng dụng không dừng ở mức hiển thị. Mỗi bước đều nối tiếp bước trước và tạo đầu vào cho bước sau. Đây là logic thiết kế trung tâm của đề tài và cũng là cách cân bằng giữa phần cứng và phần mềm trong chương này.



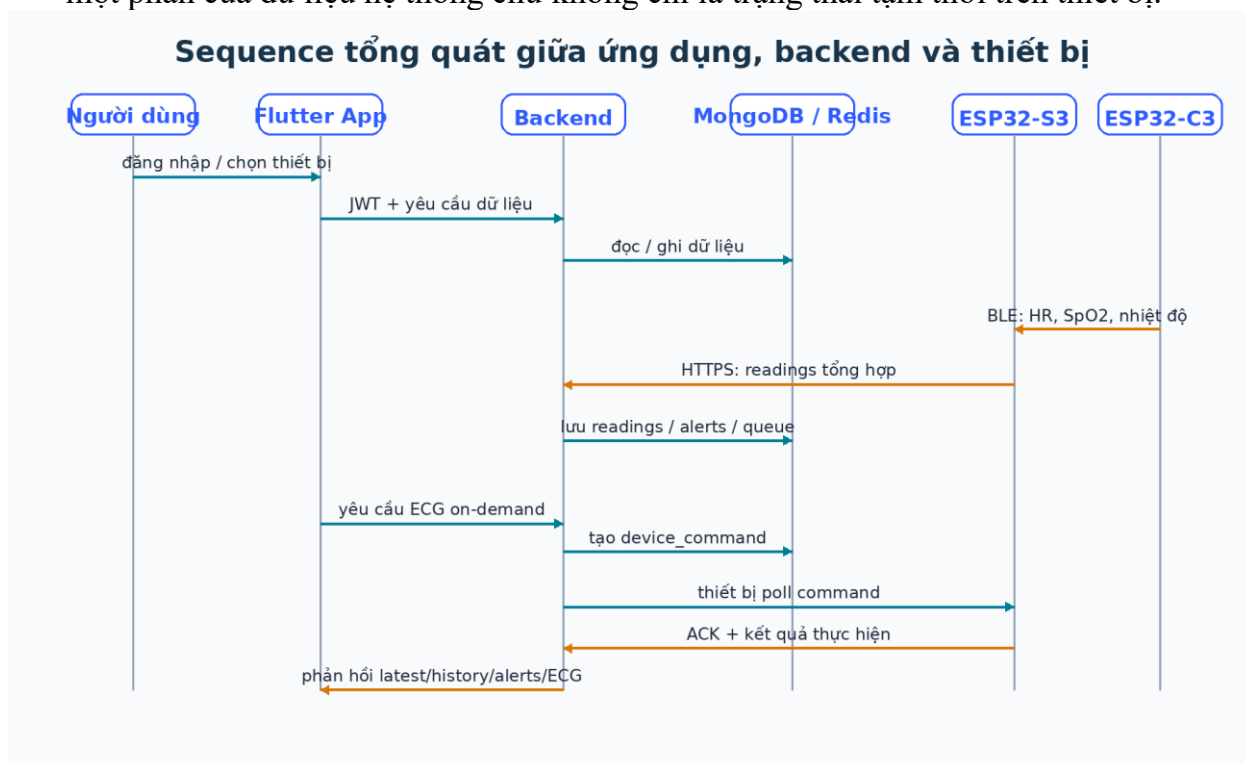
Hình 3.4. Activity tổng quát của chuỗi vận hành hệ thống.

3.3.3 Sequence tổng quát giữa ứng dụng, backend và thiết bị

Sơ đồ sequence tổng quát của Eldercare cho thấy hệ thống có hai dòng tương tác lớn diễn ra song song. Dòng thứ nhất là dòng dữ liệu từ thiết bị lên

backend rồi đến ứng dụng: bo C3 gửi BLE sang bo S3, bo S3 gửi readings lên backend, backend lưu MongoDB/Redis và sau đó cung cấp dữ liệu cho Flutter khi app truy vấn latest, history, alerts hoặc ECG. Dòng thứ hai là dòng điều khiển từ ứng dụng xuống thiết bị: người dùng gửi yêu cầu ECG, backend tạo command, thiết bị poll command tiếp theo, thực thi rồi ACK kết quả trở lại backend. Chính dòng điều khiển này làm cho hệ thống vượt khỏi mô hình giám sát thụ động thông thường.

Trong sequence tổng quát, backend luôn đóng vai trò điểm hội tụ. Ứng dụng không gửi lệnh trực tiếp đến phần cứng; thiết bị cũng không đẩy readings trực tiếp đến giao diện. Mọi tương tác đều đi qua backend để được chuẩn hóa và lưu trữ. Điều này làm tăng độ an toàn và tính nhất quán của dữ liệu, đồng thời bảo đảm rằng lịch sử sức khỏe, cảnh báo và kết quả ECG đều tồn tại như một phần của dữ liệu hệ thống chứ không chỉ là trạng thái tạm thời trên thiết bị.



Hình 3.5. Sequence tổng quát giữa người dùng, ứng dụng, backend và thiết bị.

3.4 Thiết kế tổng thể phần cứng thiết bị Eldercare

Trong cấu trúc mới của Chương 3, phần cứng không còn được tách riêng thành một chương độc lập ở cuối báo cáo, mà được đặt ngang hàng với backend, dữ liệu và Flutter như một phần của cùng một kiến trúc hệ thống. Điều này phù hợp với bản chất của Eldercare hơn, bởi nếu không có thiết bị vật lý thì toàn bộ lớp backend – app chỉ còn là khung xử lý rỗng. Mặt khác, phần cứng của đề tài không phải phần cứng tượng trưng. Nó đã được hiện thực ở mức nguyên mẫu, có pin map cụ thể, có giao tiếp rõ ràng giữa hai bo, có thiết kế PCB và có ảnh minh chứng chạy thử thật.

Mục tiêu thiết kế phần cứng được xác định theo ba hướng. Hướng thứ nhất là tạo được cụm đo sinh hiệu đủ gọn để đặt gần vùng tiếp xúc cơ thể. Hướng thứ hai là

tách riêng cụm gateway để nhận dữ liệu, hiển thị trạng thái và truyền Wi-Fi mà không làm bo đo bị quá tải. Hướng thứ ba là cho phép kiểm thử từng mô-đun tương đối độc lập để thuận tiện cho việc lắp ráp, sửa lỗi và đối chiếu với firmware. Đây là cơ sở dẫn đến quyết định tách hệ thống thành hai bo ESP32 khác nhau.

Sự phân tách phần cứng này đồng thời giải quyết bài toán cân bằng tài nguyên. ESP32-C3 thích hợp làm bo đo nhờ kích thước gọn, tiêu thụ hợp lý và đủ khả năng giao tiếp với MAX30102, DS18B20. ESP32-S3 lại phù hợp làm gateway nhờ có nhiều chân hơn, khả năng xử lý giao diện TFT tốt hơn và đủ tài nguyên để vừa nhận BLE, vừa đọc AD8232, MPU6050 và gửi HTTPS. Việc phân vai như vậy làm cho từng bo có nhiệm vụ rõ ràng, tránh tình trạng một bo duy nhất phải gánh quá nhiều mô-đun với bản đồ chân chằng chéo, dẫn đến tăng độ phức tạp trong cả firmware lẫn lắp ráp.

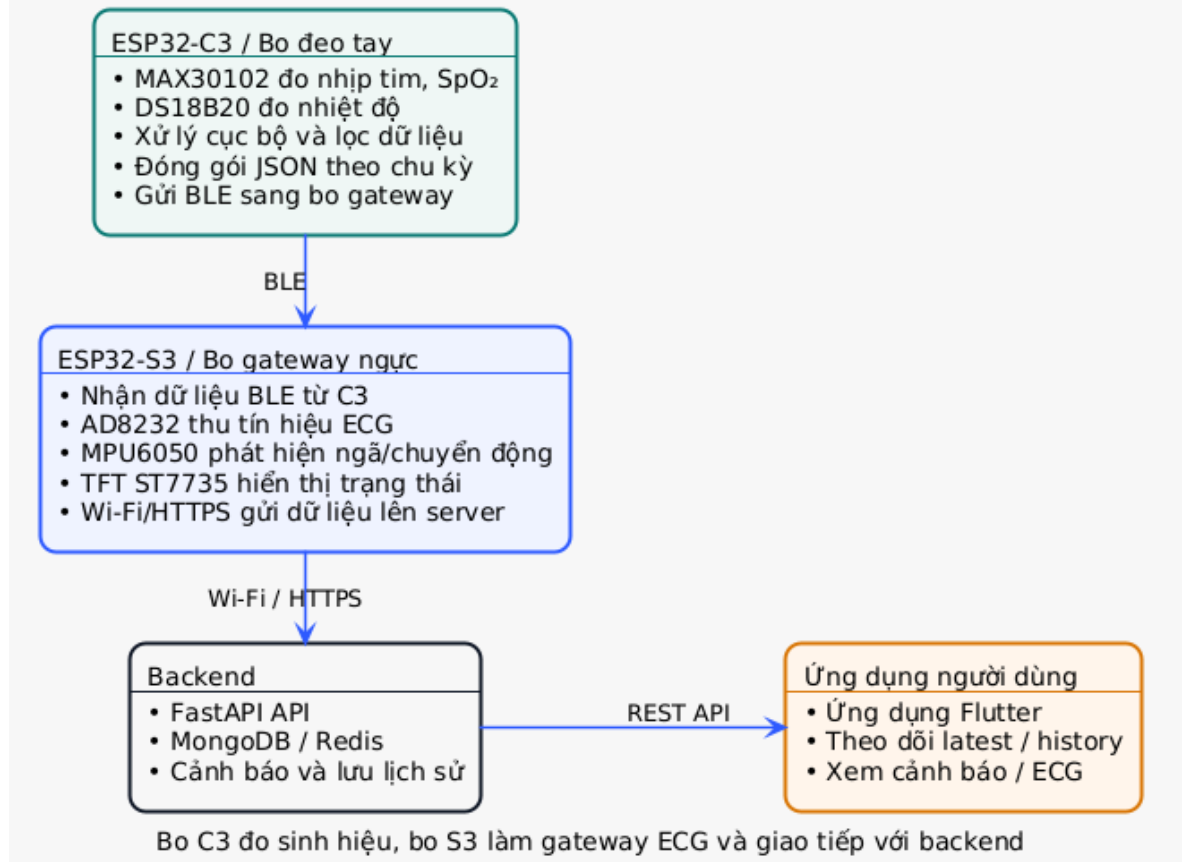
3.4.1 Kiến trúc phần cứng hai bo

Kiến trúc phần cứng của Eldercare được tổ chức như một hệ thống hai nút. Nút thứ nhất là bo ESP32-C3 đặt gần vị trí đeo, có nhiệm vụ đọc cảm biến quang học MAX30102 để suy ra nhịp tim và SpO₂, đồng thời đọc DS18B20 để lấy nhiệt độ bề mặt. Nút thứ hai là bo ESP32-S3 đóng vai trò gateway, nhận gói BLE từ bo C3, đọc thêm tín hiệu ECG từ AD8232 và dữ liệu chuyển động từ MPU6050, sau đó hiển thị nhanh lên TFT ST7735 và đồng bộ với backend qua Wi-Fi. Cách phân tách này tạo ra sự liên mạch về mặt chức năng: C3 là nơi lấy mẫu đầu vào, S3 là nơi hợp nhất và đưa dữ liệu ra ngoài.

Ưu điểm nổi bật của kiến trúc hai bo là khả năng tách biệt chức năng theo đặc thù tín hiệu. Tín hiệu quang học và nhiệt độ cần vị trí đo thuận lợi, trong khi ECG và màn hình hiển thị lại đòi hỏi không gian lắp ráp cũng như số chân kết nối lớn hơn. Nếu dùng chung một bo, đường dây và nguồn cấp sẽ trở nên phức tạp, đặc biệt khi phải đồng thời vận hành giao tiếp I²C, 1-Wire, SPI, BLE và Wi-Fi. Khi tách hai bo, bo C3 có thể tối ưu cho vùng đo; bo S3 tối ưu cho gateway, hiển thị và truyền thông mạng.

Một lợi thế khác là khả năng kiểm thử độc lập. Trong giai đoạn thử nghiệm, người thực hiện có thể chạy bo C3 để kiểm tra MAX30102, DS18B20 và gói BLE trước khi tích hợp với bo S3. Tương tự, bo S3 có thể được kiểm tra TFT, MPU6050, AD8232 và kết nối mạng ngay cả khi chưa ghép đủ readings từ bo C3. Điều này làm giảm thời gian khoanh vùng lỗi và rất phù hợp với nguyên tắc phát triển nguyên mẫu trong môi trường học thuật.

Kiến trúc phần cứng hai bo của hệ thống Eldercare



Hình 3.6. Kiến trúc phần cứng hai bo của thiết bị Eldercare.

Bảng 3.5. Các khối phần cứng chính và vai trò trong hệ thống.

Khối phần cứng	Vai trò chính	Giao tiếp
ESP32-C3	Đo HR, SpO2, nhiệt độ; gửi BLE	I ² C, 1-Wire, BLE
MAX30102	Thu tín hiệu quang học nhịp tim và SpO2	I ² C
DS18B20	Đo nhiệt độ tiếp xúc/bề mặt	1-Wire
ESP32-S3	Gateway, đọc ECG/MPU, hiển thị TFT, gửi HTTPS	BLE, Wi-Fi, GPIO, SPI, I ² C
AD8232	Thu tín hiệu ECG analog	Analog / GPIO
MPU6050	Theo dõi gia tốc, góc nghiêng, phát hiện ngã	I ² C
TFT ST7735	Hiển thị trạng thái tại chỗ	SPI

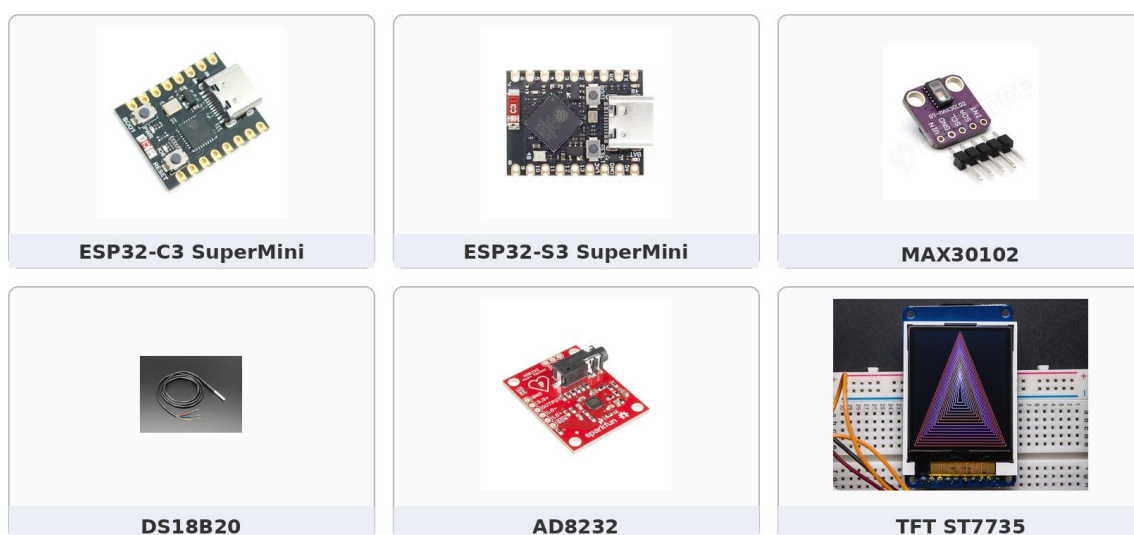
3.4.2 Lựa chọn mô-đun và lý do kỹ thuật

MAX30102 được lựa chọn vì đây là mô-đun phổ biến cho các nguyên mẫu đo HR và SpO2, tích hợp LED đỏ – hồng ngoại và photodiode, giao tiếp I²C đơn giản và có tài liệu hỗ trợ rộng. Với yêu cầu của đề tài, MAX30102 đáp ứng tốt mục tiêu theo dõi sinh hiệu nên tăng mà không làm tăng quá nhiều độ phức tạp của phần cứng. Tuy nhiên, để mô-đun hoạt động ổn định, cảm biến cần được đặt sát da, giảm ánh sáng ngoại lai và hạn chế rung cơ học.

DS18B20 được chọn để đo nhiệt độ vì mô-đun này dễ triển khai, dùng 1-Wire nên chỉ cần một chân dữ liệu, đồng thời cho phép kéo dây tương đối linh hoạt trong nguyên mẫu. Mặc dù không phải cảm biến y sinh chuyên biệt, DS18B20 phù hợp với mục tiêu học thuật của đề tài: minh chứng khả năng tích hợp dữ liệu nhiệt độ vào chuỗi readings chung của hệ thống.

AD8232 được chọn như một front-end ECG đơn giản và phổ biến. Điểm mạnh của AD8232 là cho phép đọc tín hiệu điện tim analog ở mức nguyên mẫu mà không đòi hỏi phần cứng quá phức tạp. Dù vậy, mô-đun này nhạy với nhiễu nguồn và cách bố trí dây điện cực, nên việc tách khỏi Wi-Fi/TFT và cố định dây cơ khí là yêu cầu rất quan trọng. MPU6050 được chọn để bổ sung ngữ cảnh chuyển động và tư thế, giúp hệ thống không chỉ biết chỉ số sinh hiệu mà còn nhận biết tình huống ngã hoặc thay đổi trạng thái vận động.

Màn hình TFT ST7735 được đưa vào vì nhu cầu kiểm tra tại chỗ của một nguyên mẫu phần cứng là rất lớn. Trong thực tế chạy thử, việc nhìn thấy ngay tình trạng BLE, Wi-Fi, HR, SpO2, nhiệt độ, phát hiện ngã và sóng ECG tạm đo trên màn hình giúp chẩn đoán lỗi nhanh hơn rất nhiều so với chỉ dựa vào log. Nó cũng làm cho bo S3 trở thành một gateway có giá trị tự thân, thay vì chỉ là nút chuyển tiếp vô hình.



Hình 3.7. Các mô-đun phần cứng chính sử dụng trong nguyên mẫu Eldercare.

Bảng 3.6. Lý do lựa chọn các mô-đun phần cứng chính.

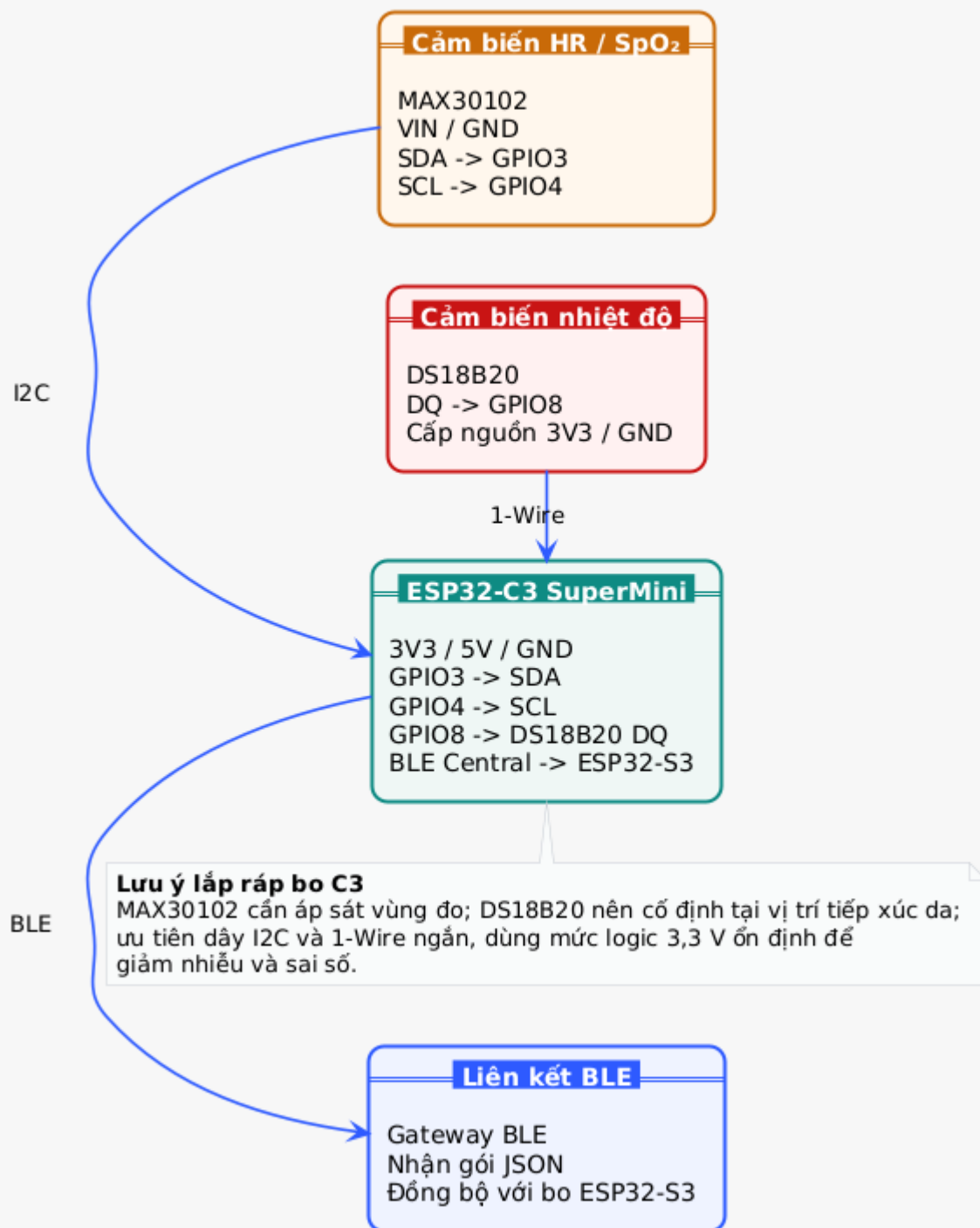
Mô-đun	Lý do lựa chọn	Lưu ý khi lắp ráp / sử dụng
ESP32-C3 SuperMini	Nhỏ gọn, đủ BLE và I/O cho bo đo	Đặt gần vùng đo để giảm dây cảm biến
ESP32-S3 SuperMini	Nhiều GPIO, xử lý hiển thị và Wi-Fi tốt	Tách tương đối khỏi cụm analog ECG
MAX30102	Đo HR/SpO2 bằng quang học, giao tiếp I ² C đơn giản	Cần áp sát da và hạn chế ánh sáng ngoài
DS18B20	1-Wire đơn giản, chi phí thấp, dễ tích hợp	Đảm bảo đầu dò tiếp xúc ổn định với bề mặt đo
AD8232	Thu ECG ở mức nguyên mẫu, phổ biến	Cần dây điện cực ngắn, cố định chắc, giảm nhiễu
MPU6050	Bổ sung chuyển động và phát hiện ngã	Cần hiệu chỉnh vị trí gắn để dữ liệu ổn định
TFT ST7735	Hiển thị trực tiếp trạng thái hệ thống	Ưu tiên đọc rõ, không che khuất kết nối

3.4.3 Sơ đồ kết nối nguyên lý bo ESP32-C3

Ở bo ESP32-C3, bản đồ chân được giữ ở mức tối giản để giảm rủi ro khi lắp ráp. MAX30102 dùng giao tiếp I²C qua GPIO3 và GPIO4, trong khi DS18B20 dùng chân dữ liệu GPIO8 theo giao tiếp 1-Wire. C3 không gánh nhiệm vụ hiển thị hay giao tiếp mạng với backend; nhiệm vụ của nó là thu đọc cảm biến, lọc dữ liệu cục bộ và định kỳ đóng gói JSON gửi qua BLE. Chính vì phạm vi chức năng được giữ gọn, bo C3 có thể được bố trí như một nút đo thuận tiện cho quá trình đeo thử.

Về kỹ thuật lắp ráp, bo C3 cần ưu tiên chiều dài dây cảm biến ngắn, đặc biệt là với MAX30102 để giảm nhiễu và giảm mất ổn định trong quá trình đo. DS18B20 nên được cố định ở vị trí tiếp xúc tương đối ổn định với da hoặc bề mặt đo để hạn chế sai lệch do môi trường xung quanh. Trong trường hợp thử nghiệm trên tay hoặc ngón tay, mô-đun quang học cần được che tương đối để giảm nhiễu ánh sáng bên ngoài.

Sơ đồ kết nối nguyên lý bo ESP32-C3



Dựa trên firmware esp32c3.ino và đầu nối của bo thiết kế

Hình 3.8. Sơ đồ kết nối nguyên lý bo ESP32-C3.

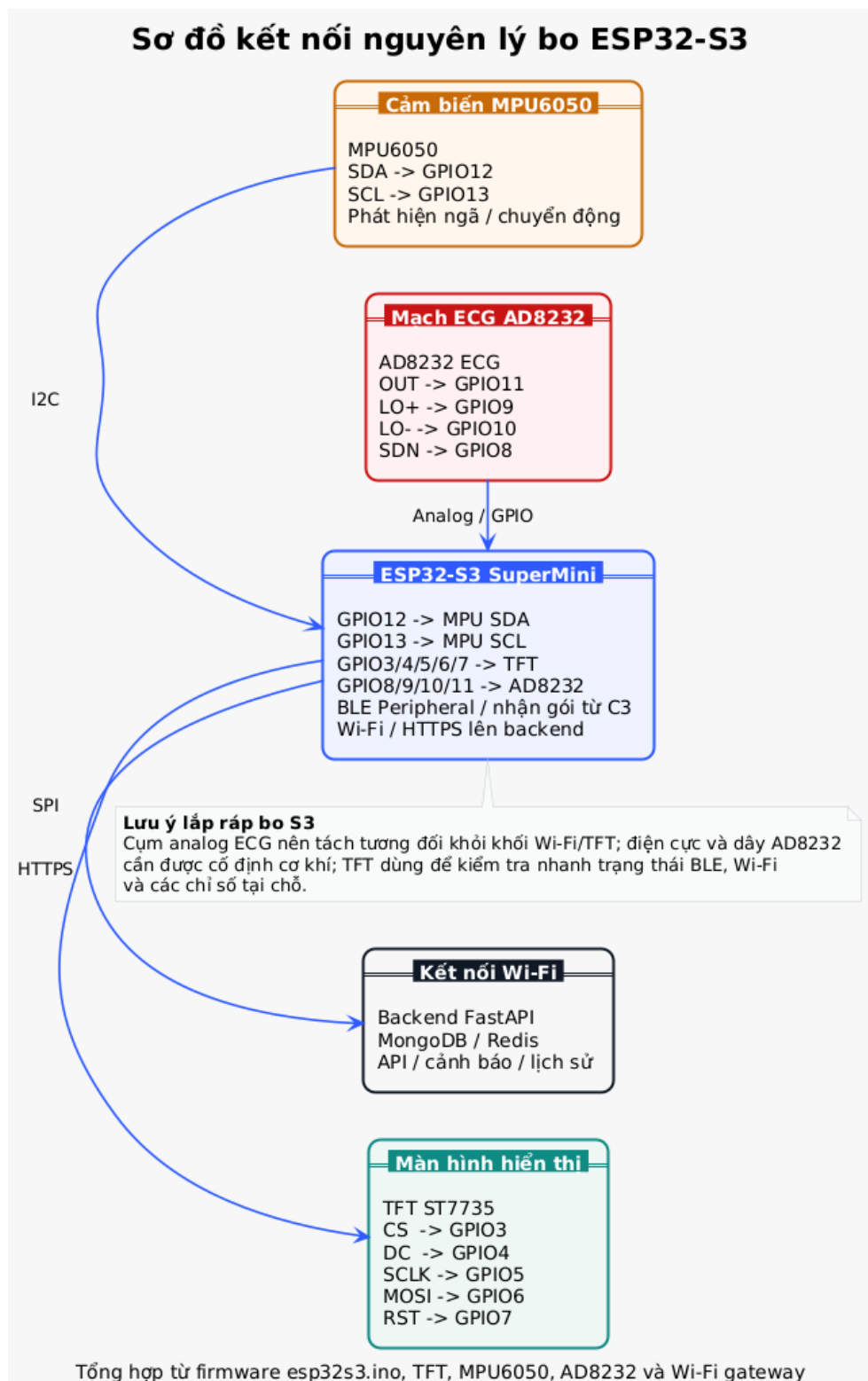
Bảng 3.7. Bảng chân kết nối chính của bo ESP32-C3.

Thành phần	Chân kết nối	Giao tiếp	Chức năng
MAX30102	SDA = GPIO3; SCL = GPIO4	I ² C	Đo nhịp tim và SpO ₂
DS18B20	DQ = GPIO8	1-Wire	Đo nhiệt độ
BLE tích hợp	Radio nội bộ	BLE	Gửi gói dữ liệu sang ESP32-S3

3.4.4 Sơ đồ kết nối nguyên lý bo ESP32-S3

Bo ESP32-S3 là nút phức tạp hơn về phần cứng vì đồng thời hội tụ nhiều nhóm tín hiệu. MPU6050 nối qua I²C với GPIO12 và GPIO13. TFT ST7735 dùng các chân GPIO3, GPIO4, GPIO5, GPIO6 và GPIO7 cho CS, DC, SCLK, MOSI và RST. Mạch ECG AD8232 dùng GPIO8, GPIO9, GPIO10 và GPIO11 cho SDN, LO+, LO- và OUT. Bên cạnh đó, bo S3 còn phải duy trì hai kênh không dây: BLE để nhận dữ liệu từ bo C3 và Wi-Fi/HTTPS để gửi readings lên backend.

Do phải xử lý cả tín hiệu analog ECG lẫn hoạt động mạng – màn hình, bo S3 là nơi cần quan tâm nhất đến bố trí linh kiện và chống nhiễu. Vùng AD8232 nên được đặt tương đối tách khỏi phần Wi-Fi/TFT để tránh nhiễu xuyên. Điện cực ECG và dây dẫn cần được cố định chắc về cơ khí. Màn hình TFT nên đặt ở vị trí dễ quan sát để thuận tiện chẩn đoán nhanh khi chạy thử phần cứng, đặc biệt trong các trường hợp chưa chắc lỗi thuộc BLE, Wi-Fi hay chính cảm biến.



Hình 3.9. Sơ đồ kết nối nguyên lý bo ESP32-S3.

Bảng 3.8. Bảng chân kết nối chính của bo ESP32-S3.

Thành phần	Chân kết nối	Giao tiếp	Chức năng
MPU6050	SDA = GPIO12; SCL = GPIO13	I ² C	Đo gia tốc và góc nghiêng
TFT ST7735	CS = GPIO3; DC = GPIO4; SCLK = GPIO5; MOSI = GPIO6; RST = GPIO7	SPI	Hiển thị trạng thái tại chỗ
AD8232	SDN = GPIO8; LO+ = GPIO9; LO- = GPIO10; OUT = GPIO11	Analog / GPIO	Thu tín hiệu ECG
BLE tích hợp	Radio nội bộ	BLE	Nhận dữ liệu từ ESP32-C3
Wi-Fi tích hợp	Radio nội bộ	Wi-Fi / HTTPS	Gửi readings lên backend

3.4.5 Nguồn cấp, chống nhiễu và nguyên tắc lắp ráp

Đối với thiết bị đeo y sinh ở mức nguyên mẫu, nguồn cấp và chống nhiễu quyết định rất lớn đến độ ổn định của readings. Toàn bộ hệ thống được định hướng dùng đầu vào 5V, sau đó các chân logic ESP32 và các mô-đun tương thích làm việc ở mức 3,3V. Yêu cầu đầu tiên là tất cả mô-đun phải dùng chung mass tham chiếu để tránh trôi mức logic. Yêu cầu thứ hai là tuyệt đối không cấp sai mức điện áp vào chân dữ liệu của ESP32, đặc biệt với các mô-đun giao tiếp trực tiếp qua I²C, 1-Wire hoặc GPIO analog.

Với AD8232, vấn đề chính không nằm ở việc “đọc được tín hiệu” mà là đọc được tín hiệu đủ sạch. Vì thế, dây điện cực cần ngắn, tránh đi song song gần cụm nguồn hoặc cụm Wi-Fi, và nên được cố định cơ khí để không rung trong lúc người dùng cử động. MAX30102 cũng cần được xem là mô-đun nhạy nhiễu: rung cơ học, ánh sáng ngoại lai và áp lực tiếp xúc không ổn định đều có thể làm readings dao động. Điều này lý giải vì sao phần cứng của đề tài được tách thành hai bo và tại sao các bước lắp ráp – chạy thử phải đi từ nguồn, đến từng mô-đun đơn lẻ, rồi mới tích hợp toàn hệ thống.

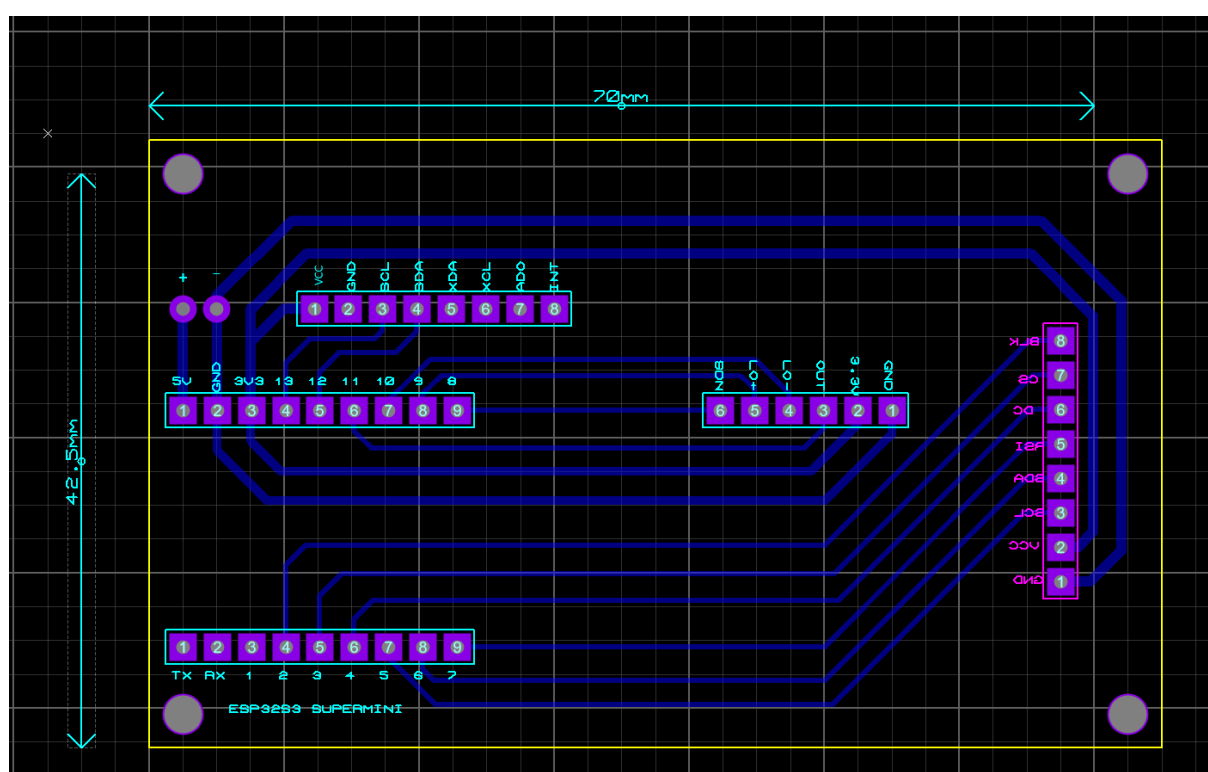
Quy trình lắp ráp hợp lý gồm sáu bước. Bước đầu kiểm tra mạch nguồn khi chưa gắn cảm biến. Bước hai hàn hai bo ESP32 và kiểm tra nạp firmware. Bước ba đấu nối MAX30102, DS18B20, TFT và MPU6050 để xác nhận I²C và SPI. Bước bốn mới gắn AD8232 và điện cực ECG. Bước năm kiểm tra BLE giữa hai bo. Bước cuối cùng là thử Wi-Fi và HTTPS đến backend. Trình tự này không chỉ giảm nguy cơ làm hỏng mô-đun mà còn giúp người thực hiện xác định lỗi nhanh theo từng lớp.

3.4.6 Thiết kế PCB, bố trí linh kiện và minh chứng phần cứng

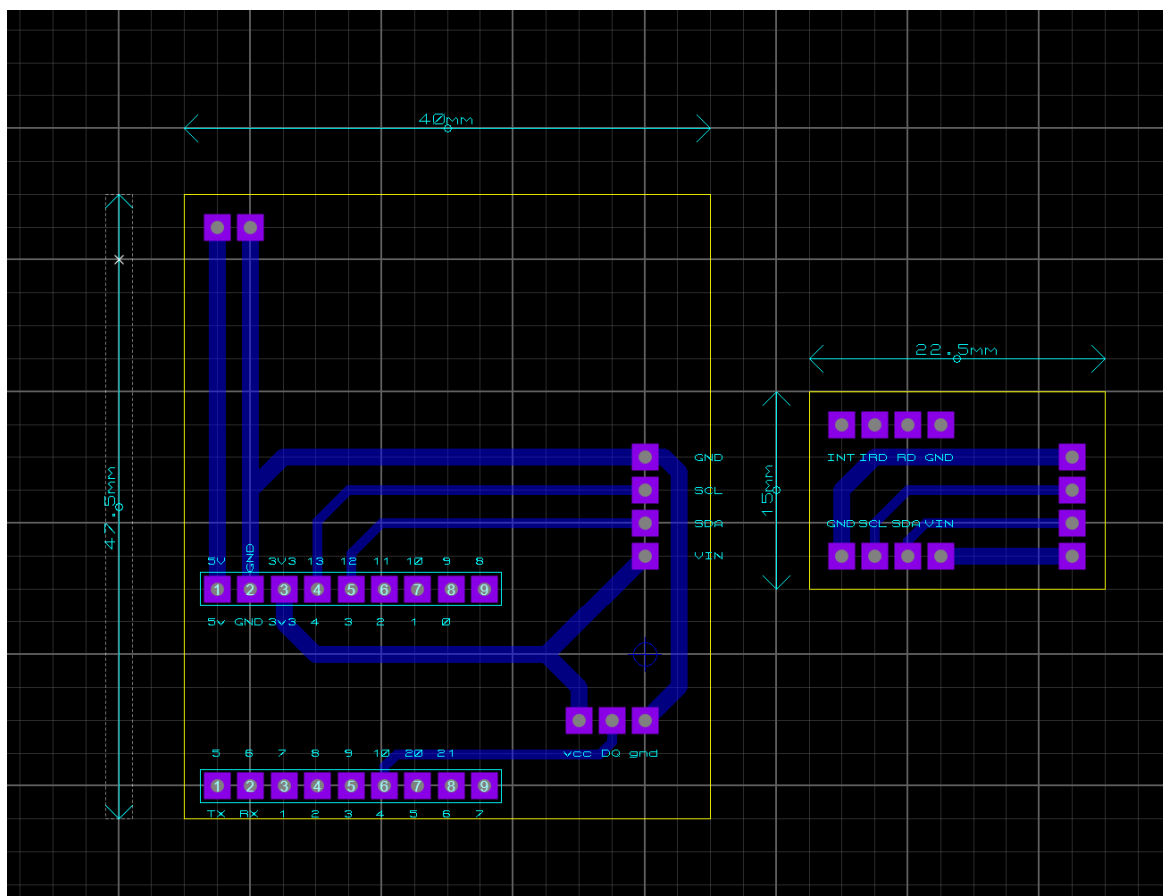
Hai bản vẽ PCB của đề tài cho thấy phần cứng đã được đưa đến mức triển khai cụ thể, vượt khỏi sơ đồ khối đơn thuần. Bo ESP32-S3 được bố trí với không gian đủ cho đầu nối cảm biến, màn hình và các khối chức năng chính. Bo ESP32-C3 được bố trí gọn hơn, tập trung vào cụm đầu nối cảm biến và đường truyền sang gateway. Trong cả hai trường hợp, các đầu nối đều được đưa tương đối ra mép bo để thuận tiện cho việc đo điện áp, thay dây và sửa lỗi trong quá trình thử nghiệm.

Về nguyên tắc bố trí, nhóm cảm biến được đặt theo cụm chức năng thay vì phân tán ngẫu nhiên. Điều này có lợi cho cả đường đi tín hiệu lẫn khả năng bảo trì. Với bo S3, vùng analog ECG cần được tách tương đối khỏi cụm Wi-Fi và TFT. Với bo C3, cụm cảm biến được ưu tiên đường nối ngắn. Dù nguyên mẫu chưa hướng đến tối ưu hóa cơ khí như sản phẩm thương mại, cách bố trí hiện tại đủ để chứng minh rằng phần cứng của đề tài được thiết kế có cơ sở chứ không phải ghép dây ngẫu hứng trên breadboard.

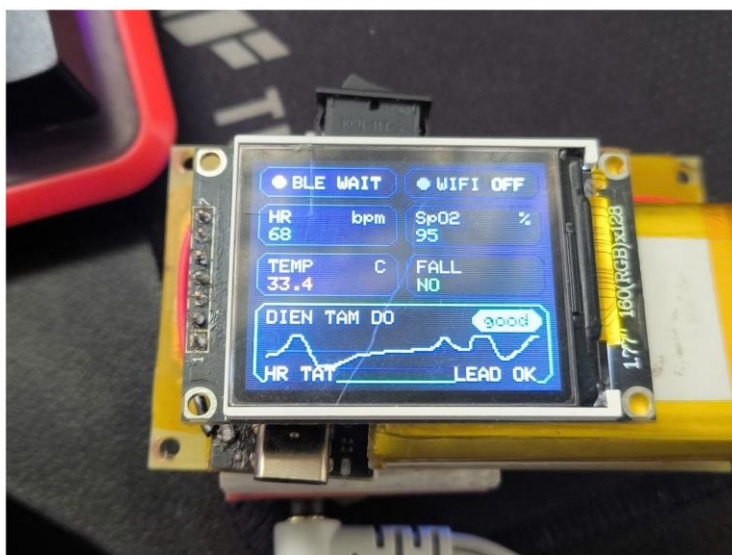
Ngoài sơ đồ và PCB, phần cứng còn được minh chứng bằng ảnh nguyên mẫu thực tế. Các ảnh chụp cho thấy bo gateway S3 có màn hình TFT hoạt động, bo đo C3 được gắn kèm cảm biến đeo tay và toàn bộ nguyên mẫu có thể vận hành trong điều kiện thử nghiệm. Việc đưa ảnh thực tế vào chương có ý nghĩa rất lớn về học thuật: nó chuyển báo cáo từ mức “mô tả thiết kế” sang “mô tả nguyên mẫu đã lắp ráp và chạy thử”.



Hình 3.10. Thiết kế PCB bo ESP32-S3 kích thước 70 mm × 42,5 mm.



Hình 3.12. Thiết kế PCB bo ESP32-C3 và cụm đầu nối cảm biến/gateway.



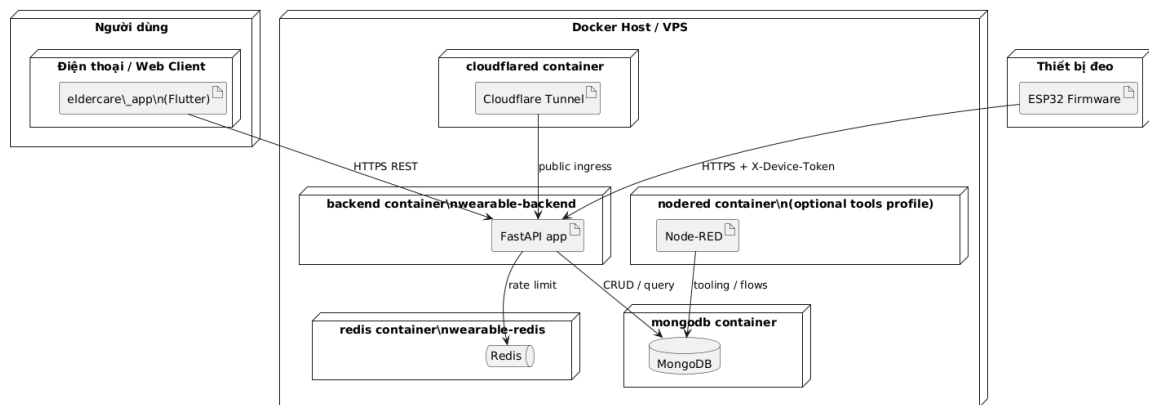
Hình 3.13. Hình ảnh phân cứng thực tế của cụm thiết bị sau khi lắp ráp.

3.5 Thiết kế tổng thể backend và lớp dữ liệu

Backend là lớp trung tâm của toàn hệ thống Eldercare. Nếu coi thiết bị phần cứng là nơi phát sinh readings và ứng dụng Flutter là nơi tiêu thụ dữ liệu, thì backend chính là điểm hội tụ của cả hai phía. Nó tiếp nhận request từ app, request từ thiết bị, xác thực đúng loại client, xử lý nghiệp vụ, truy xuất cơ sở dữ liệu và trả về kết quả theo đúng ngữ cảnh. Chính vì vậy, trong cấu trúc chương mới, backend không được xem như một chương kỹ thuật riêng lẻ mà là một thành phần lớn của kiến trúc hệ thống chung.

Mục tiêu thiết kế backend có thể tóm lược thành bốn hướng. Hướng thứ nhất là ingest readings đúng đắn và ổn định. Hướng thứ hai là phục vụ đồng thời nhiều loại client: app/admin và thiết bị ESP32. Hướng thứ ba là cho phép luồng tương tác hai chiều thông qua ECG on-demand. Hướng thứ tư là hỗ trợ vận hành ở mức nguyên mẫu tương đối hoàn chỉnh thông qua logging, metrics, health check và rate limit. Những mục tiêu này cho thấy backend của Eldercare không dừng ở mức tập hợp endpoint trả JSON, mà là một lớp điều phối nghiệp vụ thực sự.

Về tổ chức mã, backend được cấu trúc theo mô hình phân lớp gồm main, config, api, services, db, models và utils. Từ góc nhìn kiến trúc, đây là lựa chọn hợp lý vì route handler không bị dồn quá nhiều logic nghiệp vụ, service không phải biết quá nhiều về chi tiết HTTP, còn db.py tập trung xử lý truy cập MongoDB và tạo index. Cách tổ chức này làm cho backend dễ bảo trì hơn, đồng thời tạo ra một ngôn ngữ chung để các lớp khác như app và thiết bị có thể tương tác nhất quán thông qua contract API.



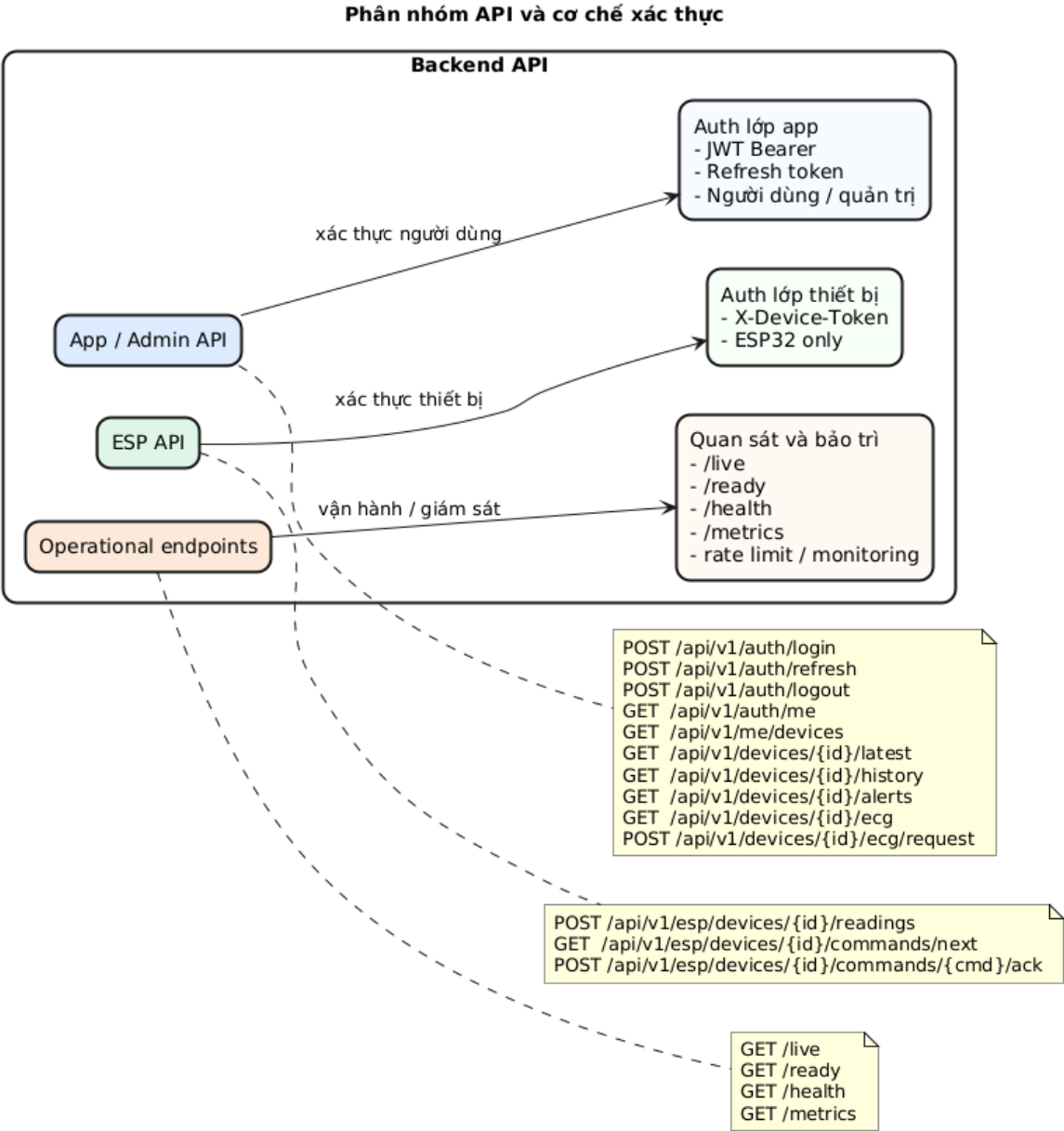
Hình 3.14. Deployment diagram tổng quan Docker/Production

3.5.1 Nhóm API và cơ chế xác thực

Một quyết định kiến trúc rất quan trọng của Eldercare là tách riêng API cho ứng dụng và API cho thiết bị. Nhóm route `/api/v1/*` phục vụ app và công cụ quản trị với cơ chế Authorization: Bearer <JWT> cùng refresh token. Nhóm route `/api/v1/esp/*` phục vụ firmware và dùng X-Device-Token. Sự phân tách này làm giảm đáng kể nguy cơ trộn lẫn logic xác thực của hai loại client có bản chất hoàn toàn khác nhau.

Ở phía ứng dụng, access token đóng vai trò bằng chứng xác thực ngắn hạn, còn refresh token giữ cho phiên hoạt động ổn định qua nhiều lần mở ứng dụng. Ở phía thiết bị, device token được chọn vì đơn giản, đủ nhẹ cho firmware và ít

trạng thái hơn nhiều so với mô hình session của mobile app. Nhờ vậy, cả hai bên đều có được cơ chế phù hợp với đặc thù riêng, trong khi backend vẫn giữ vai trò điều phối thống nhất.



Hình 3.16. Phân nhóm API và cơ chế xác thực

Bảng 3.9. Nhóm API và lớp xác thực tương ứng.

Nhóm API	Đường dẫn tiêu biểu	Xác thực	Vai trò
Auth	/api/v1/auth/login, /refresh, /me	JWT + refresh token	Tạo và duy trì phiên người dùng

Devices	/api/v1/me/devices, /latest, /history, /ecg	JWT + ownership	Phục vụ truy vấn dữ liệu theo thiết bị
ESP	/api/v1/esp/devices/{id}/readings, /commands/next, /ack	X-Device-Token	Ingest readings và xử lý command
Operational	/live, /ready, /health, /metrics	Nội bộ hệ thống	Giám sát và kiểm tra runtime

3.5.2 Pipeline xử lý readings, cảnh báo và ECG on-demand

Pipeline xử lý readings là trục xương sống của backend. Khi bo ESP32-S3 gửi readings lên, route ESP của backend đầu tiên kiểm tra device token và tính hợp lệ của payload. Sau đó health_service tiếp nhận reading, db.py lưu document vào health_readings và alert_service kiểm tra ngưỡng để tạo alert nếu cần. Đây là thiết kế quan trọng vì nó làm cho cảnh báo không phải là bước xử lý tách rời về sau, mà được gắn trực tiếp với luồng ingest dữ liệu.

Một đặc điểm đáng chú ý khác là cơ chế dedupe dựa trên cặp device_id và seq. Trong hệ thống IoT, mạng yếu hoặc việc gửi lại cùng một gói là tình huống bình thường. Vì vậy, backend không được coi resend là lỗi, mà phải có chiến lược loại bỏ trùng lặp nhưng vẫn bảo toàn readings hợp lệ. Việc dùng unique partial index và logic duplicate giúp giải bài toán này theo cách phù hợp với môi trường phần cứng thực tế.

Cùng nằm trong pipeline dữ liệu là cơ chế ECG on-demand. Khi người dùng bấm yêu cầu đo ECG từ app, backend không kết nối trực tiếp vào thiết bị theo kiểu push mà tạo một document command trong device_commands. Thiết bị ESP32-S3 định kỳ poll command tiếp theo, nhận lệnh, thực thi và ACK trở lại backend. Cơ chế này đơn giản, dễ debug và tận dụng MongoDB làm nơi giữ trạng thái queue trong phạm vi của một nguyên mẫu học thuật.

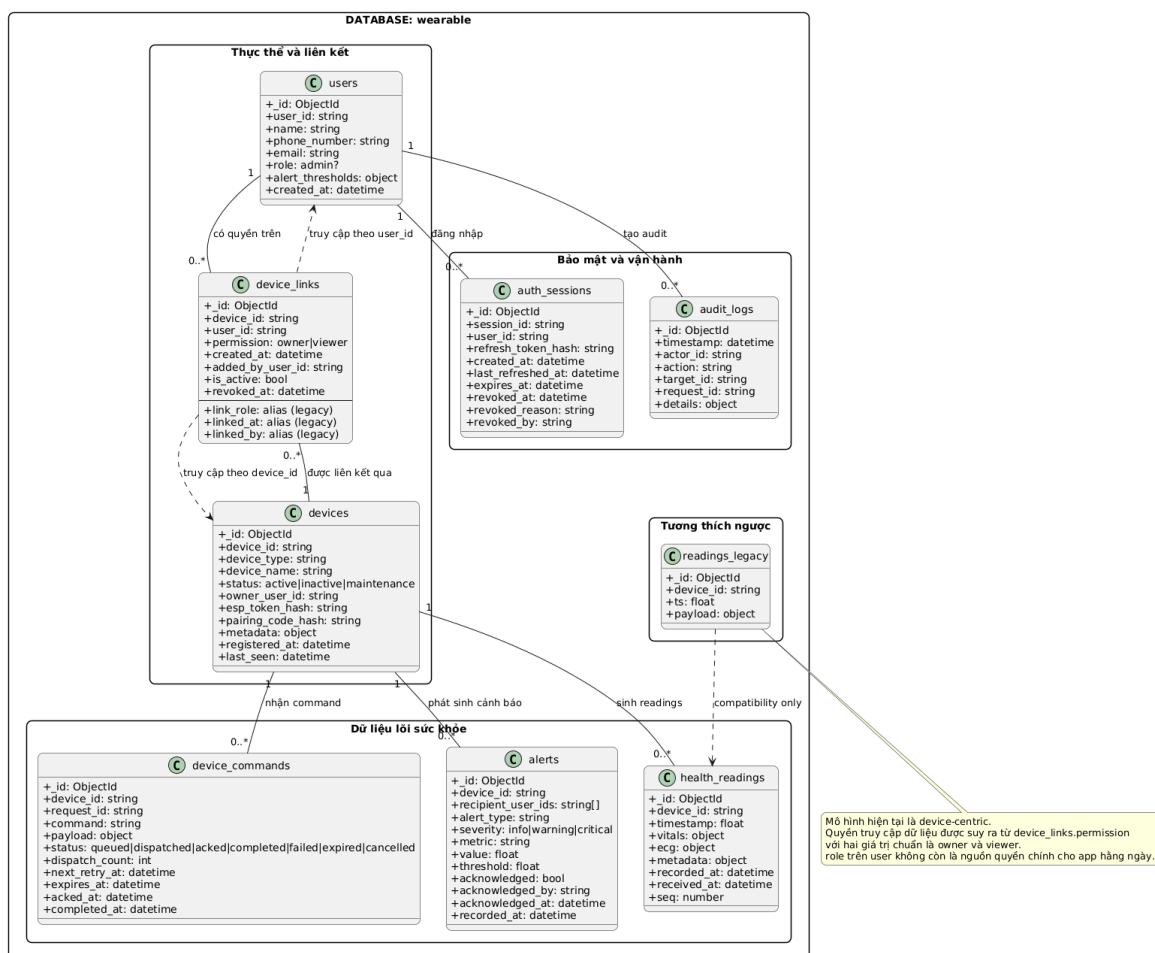
3.5.3 Thiết kế dữ liệu MongoDB và Redis

Cơ sở dữ liệu của hệ thống được xây dựng theo hướng document-centric để phù hợp với readings sức khỏe vốn có cấu trúc bán linh hoạt. Collection health_readings lưu dữ liệu sức khỏe, alerts lưu cảnh báo vượt ngưỡng, device_commands lưu lệnh điều phối như ECG on-demand, users và devices lưu thực thể nền tảng, device_links biểu diễn quan hệ owner/viewer giữa người dùng và thiết bị, còn auth_sessions duy trì refresh token và vòng đời phiên.

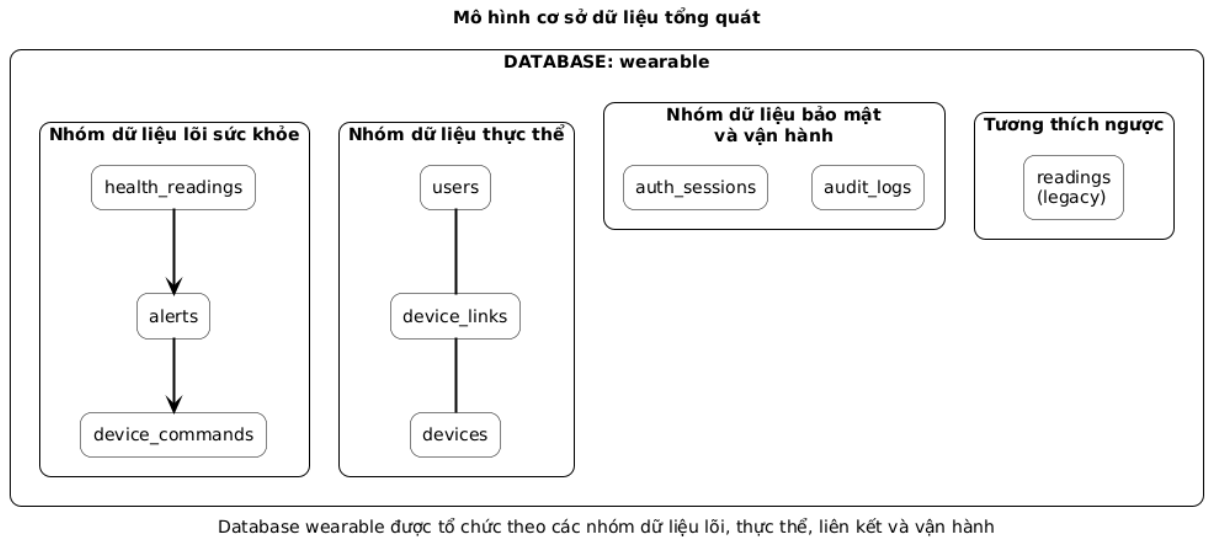
MongoDB được chọn vì phù hợp với readings có cấu trúc lồng nhau, hỗ trợ tốt việc truy xuất latest, history hoặc ECG theo device_id và timestamp. Redis không giữ dữ liệu nghiệp vụ lỗi mà phục vụ cho rate limit và một số chức năng trạng thái ngắn hạn. Cách phân vai này giúp dữ liệu sức khỏe nằm gọn

trong một lớp persistence chính, trong khi Redis bảo vệ API khỏi tình trạng bị spam hoặc gọi quá dày.

Một ưu điểm rõ của lớp dữ liệu là có ý thức về vòng đời dữ liệu. Health readings, alerts, command và session không phải lúc nào cũng cần được giữ vô thời hạn, vì vậy hệ thống áp dụng TTL và các index phù hợp để vừa tối ưu truy vấn vừa giới hạn kích thước database. Điều này cho thấy cơ sở dữ liệu của Eldercare không chỉ được dựng để “chạy được” mà còn có tư duy vận hành từ sớm.



Hình 3.17. Các nhóm dữ liệu logic của hệ thống



Hình 3.18. Mô hình tổng quát của database wearable và các nhóm collection chính.

Bảng 3.10. Các collection dữ liệu chính của hệ thống.

Collection	Vai trò nghiệp vụ	Ghi chú thiết kế
health_readings	Lưu readings và ECG gửi từ thiết bị	Truy vấn latest/history; có TTL và dedupe
alerts	Lưu cảnh báo vượt ngưỡng	Theo severity, acknowledged, cửa sổ lặp
device_commands	Lưu lệnh điều phối ECG on-demand	Polling queue cho thiết bị
users	Lưu hồ sơ và thông tin người dùng	Liên kết mềm với device_links
devices	Lưu định danh thiết bị, token hash, metadata, last_seen	Thực thể cốt lõi cho mô hình device-centric
device_links	Gắn owner / viewer với thiết bị	Cơ sở phân quyền dữ liệu hằng ngày
auth_sessions	Lưu refresh token hash và trạng thái phiên	Hỗ trợ rotate và revoke session

3.5.4 Quan sát vận hành và độ sẵn sàng của backend

Với một hệ thống sức khỏe từ xa, việc backend chỉ xử lý đúng nghiệp vụ là chưa đủ; hệ thống còn phải cho phép quan sát khi có sự cố. Vì vậy backend

cung cấp các endpoint /live, /ready, /health và /metrics. Những endpoint này giúp kiểm tra service còn chạy hay không, phụ thuộc như MongoDB và Redis có sẵn sàng hay không, đồng thời cung cấp số liệu để theo dõi hành vi runtime nếu cần.

Ở góc độ học thuật, phần observability của Eldercare có giá trị vì nó chứng minh đề tài không chỉ dừng API để minh họa. Logging, monitor script, backup/restore và rate limit là những yếu tố gắn hệ thống với bối cảnh vận hành thực tế. Nhờ đó, khi ghép với phần cứng thật, backend trở thành một lớp dịch vụ đủ trưởng thành để chống đỡ readings, queue command và phiên người dùng ở mức nguyên mẫu hoàn chỉnh.

3.6 Thiết kế tổng thể ứng dụng Flutter

Ứng dụng Flutter là điểm tiếp xúc trực tiếp giữa hệ thống và người dùng. Nếu backend chịu trách nhiệm nghiệp vụ và thiết bị tạo ra dữ liệu, thì app chịu trách nhiệm chuyển toàn bộ dữ liệu đó thành trải nghiệm sử dụng thực tế. Điều này nghe có vẻ đơn giản, nhưng trên thực tế app phải giải quyết cùng lúc nhiều vấn đề: đăng nhập, phục hồi phiên, đồng bộ thiết bị, quản lý device scope, tải latest, history, alerts, ECG, phản ánh trạng thái cache và tạo yêu cầu ECG on-demand. Vì thế, ứng dụng không thể chỉ được xây dựng như một tập hợp màn hình rời rạc.

Kiến trúc tổng thể của app được tổ chức theo hướng tách lớp giữa UI, state management, API service và local storage. SessionProvider, DeviceProvider, RealtimeProvider, HistoryProvider, EcgProvider và AlertsProvider chia nhau các trách nhiệm lớn, trong khi ApiClient, AuthStorage, VitalsCacheStorage và các service domain làm nhiệm vụ giao tiếp hoặc lưu trữ nền. Chính sự phân lớp này giúp app bám chặt contract backend mà vẫn giữ được giao diện mạch lạc.

Một điểm rất quan trọng là mô hình device-centric ở phía app. Người dùng không bị gán cứng với một thiết bị duy nhất; sau khi đăng nhập, app phải tải danh sách thiết bị liên kết, nhớ current device và tái bind các provider dữ liệu theo đúng thiết bị đang được chọn. Điều đó làm cho Flutter của Eldercare không chỉ là ứng dụng hiển thị số đo, mà là lớp điều phối trải nghiệm xoay quanh thiết bị thực tế.

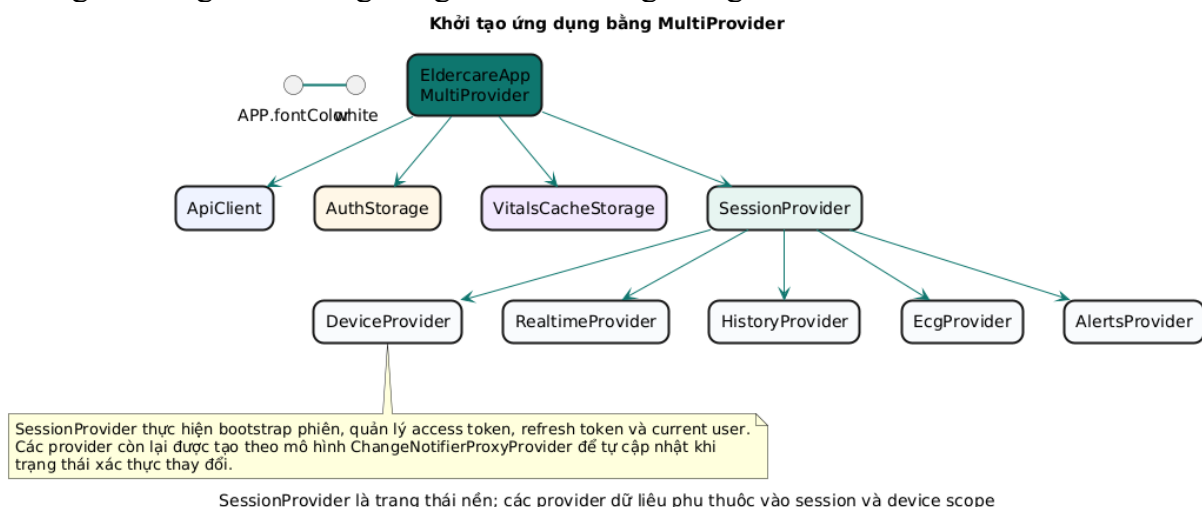
3.6.1 Kiến trúc provider, service và storage

Ở cấp khởi tạo, EldercareApp dùng MultiProvider làm điểm lắp ráp mọi dependency. SessionProvider được tạo đầu tiên để chịu trách nhiệm bootstrap phiên. DeviceProvider, RealtimeProvider, HistoryProvider, EcgProvider và AlertsProvider được tạo sau dưới dạng các trạng thái phụ thuộc. Cách tổ chức này giúp mọi trạng thái cấp cao đều gắn với session và current device, thay vì mỗi màn hình tự xử lý logic kết nối riêng của nó.

Tầng giao tiếp API được chuẩn hóa bởi ApiClient. Thành phần này gắn JWT vào header, quản lý callback refresh khi token hết hạn và chuyển lỗi HTTP sang một dạng lỗi có ý nghĩa hơn cho tầng state. Nhờ vậy, AuthApiService, DeviceApiService hay HealthApiService chỉ cần tập trung vào endpoint và payload thay vì lặp lại logic xác thực hoặc xử lý lỗi ở nhiều nơi.

Ở tầng lưu trữ cục bộ, AuthStorage giữ access token, refresh token và current user; SharedPreferences lưu current device và một phần danh sách thiết bị; VitalsCacheStorage lưu latest và history theo scope. Đây là lớp giúp app vẫn

duy trì được trải nghiệm sử dụng tương đối mượt khi mạng không ổn định hoặc khi người dùng mở lại ứng dụng sau một thời gian ngắn.



Hình 3.19. Sơ đồ dependency graph khi khởi tạo ứng dụng bằng MultiProvider.

Bảng 3.11. Vai trò của các provider chính trong ứng dụng Flutter.

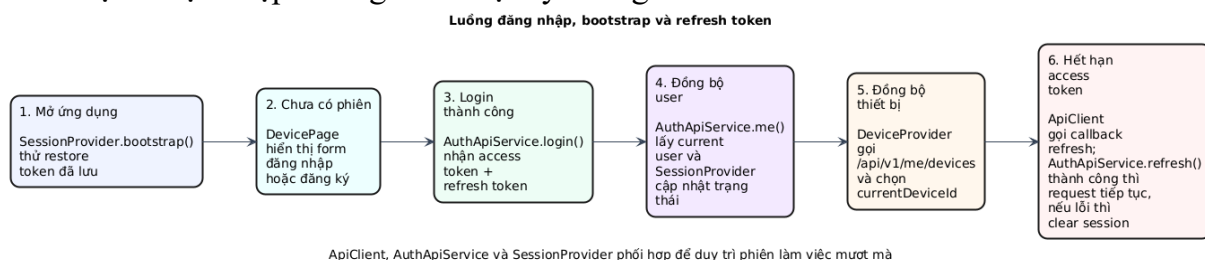
Provider	Trách nhiệm	Trạng thái nổi bật
SessionProvider	Quản lý phiên đăng nhập và bootstrap	access token, refresh token, current user
DeviceProvider	Đồng bộ danh sách thiết bị và current device	devices, current device
RealtimeProvider	Tải dữ liệu latest và phản ánh trạng thái online/cache	latest, live points, cached state
HistoryProvider	Tải lịch sử theo ngày và theo thiết bị	selected day, history points
EcgProvider	Điều phối yêu cầu ECG on-demand và kết quả chờ	status, last result, error
AlertsProvider	Tải và lọc danh sách cảnh báo	alert list, filter state

3.6.2 Quản lý phiên, device scope và trạng thái lỗi

Trong một ứng dụng sức khỏe từ xa, việc người dùng chỉ “đăng nhập được” là chưa đủ. App còn phải duy trì được phiên, phục hồi phiên sau khi mở lại ứng dụng, phát hiện khi access token hết hạn và thử refresh token trước khi buộc người dùng đăng nhập lại. SessionProvider và ApiClient là hai thành phần then chốt cho luồng này. SessionProvider giữ trạng thái xác thực, còn ApiClient là lớp thực thi request, tự gắn JWT và kích hoạt refresh callback khi cần.

Sau khi phiên hợp lệ, ứng dụng chuyển sang bài toán thứ hai là chọn đúng thiết bị. DeviceProvider tải danh sách thiết bị liên kết, lưu current device và dùng lựa chọn này làm scope cho latest, history, alerts và ECG. Từ góc độ kiến trúc, đây là điểm nối trực tiếp giữa mô hình device_links ở backend và mô hình hiển thị ở Flutter. Nếu phần này không được thiết kế rõ, app rất dễ rơi vào tình trạng trộn dữ liệu giữa nhiều thiết bị hoặc hiển thị readings sai ngữ cảnh.

Ứng dụng cũng phải xử lý tốt các trạng thái rỗng và lỗi. Trường hợp chưa đăng nhập, chưa có thiết bị, chưa có readings, phiên hết hạn hoặc không có quyền xem thiết bị đều phải được phản ánh rõ bằng giao diện. Cách xử lý này giúp app gần với ngữ cảnh vận hành thật hơn rất nhiều so với các giao diện chỉ hiển thị dữ liệu “đẹp” trong điều kiện lý tưởng.



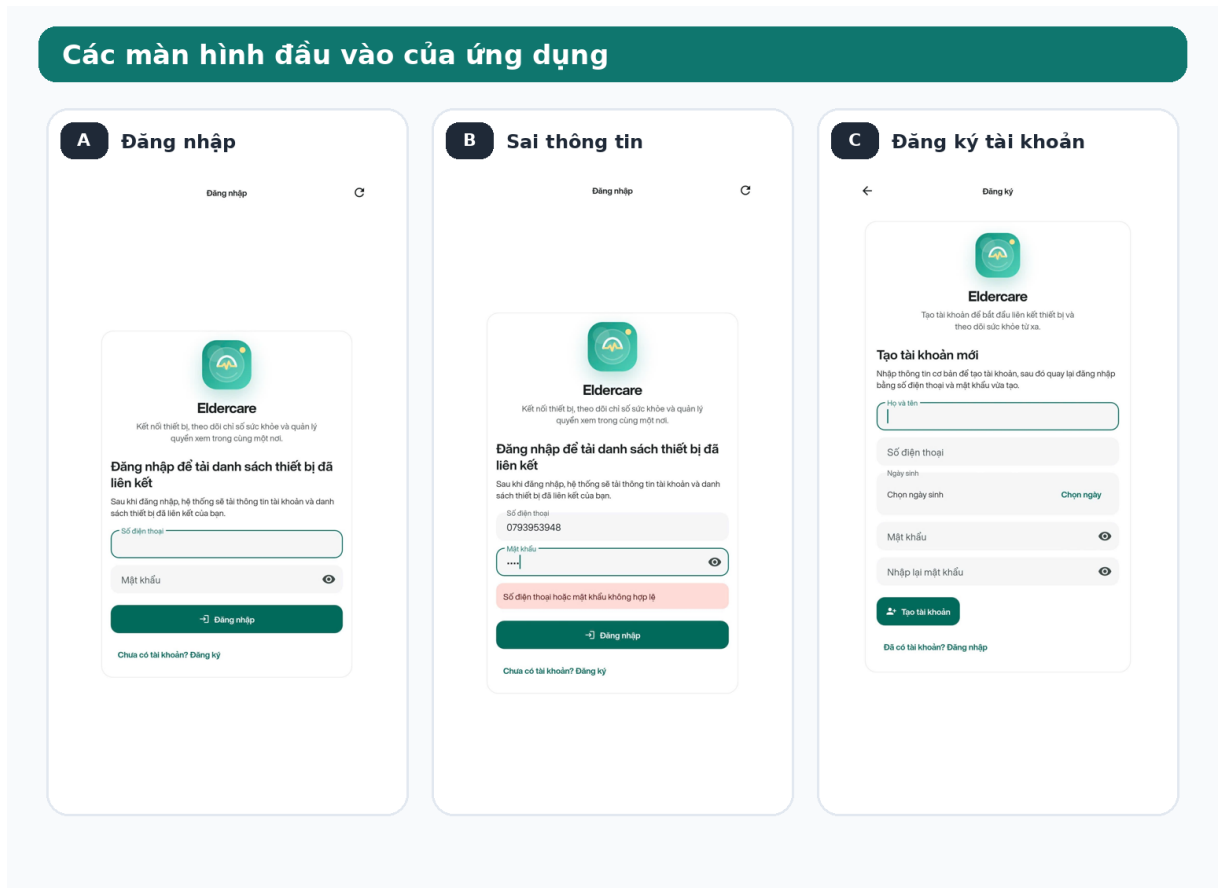
Hình 3.20. Luồng bootstrap, đăng nhập và refresh token của ứng dụng.

3.6.3 Tổ chức giao diện và luồng sử dụng chính

Về tổ chức giao diện, hệ thống xoay quanh bốn khu vực chính. Khu vực thứ nhất là đăng nhập – đăng ký – thông tin tài khoản, phục vụ điểm vào của người dùng. Khu vực thứ hai là danh sách thiết bị và liên kết thiết bị, nơi người dùng xác lập current device. Khu vực thứ ba là dashboard sức khỏe, tập trung hiển thị latest, cảnh báo và các hành động nhanh. Khu vực cuối cùng là lịch sử, ECG và các màn hình chức năng mở rộng như danh sách người xem hay hồ sơ thiết bị.

Thiết kế giao diện của app hướng đến sự trực quan và nhất quán hơn là cố gắng “nhồi” quá nhiều thông tin vào cùng một màn hình. Dashboard đóng vai trò trung tâm vì đây là nơi người dùng thường quay lại nhất để xem tình trạng sức khỏe hiện tại. Lịch sử theo ngày, cảnh báo và ECG được chuyển thành các màn hình chuyên biệt để tránh làm giao diện chính trở nên dày đặc. Đây là một quyết định hợp lý vì đối tượng sử dụng hệ thống có thể không phải là người dùng kỹ thuật cao; họ cần bố cục dễ hiểu trước khi cần chi tiết sâu hơn.

Từ góc nhìn của chương thiết kế, điều đáng chú ý không chỉ là app có những màn hình nào, mà là mỗi màn hình phản ánh đúng một lớp nghiệp vụ của toàn hệ thống. Màn hình danh sách thiết bị phản ánh device_links ở backend. Dashboard phản ánh latest readings. Màn hình cảnh báo phản ánh collection alerts. Màn hình ECG phản ánh cả readings lẫn command queue. Nhờ vậy, Flutter đóng vai trò cầu nối rất rõ giữa ngôn ngữ dữ liệu của backend và trải nghiệm sử dụng của người chăm sóc.



Hình 3.21. Cụm màn hình đầu vào của ứng dụng: đăng nhập, lỗi xác thực và đăng ký tài khoản.

Mô tả: Cụm màn hình đầu vào của ứng dụng được thiết kế theo phong cách tối giản, trực quan và đồng bộ với nhận diện thương hiệu Eldercare thông qua tông màu xanh ngọc chủ đạo. Giao diện gồm ba màn hình chính: đăng nhập, thông báo sai thông tin và đăng ký tài khoản. Ở màn hình đăng nhập, người dùng được yêu cầu nhập số điện thoại đã liên kết với danh sách thiết bị và mật khẩu để truy cập hệ thống; các trường nhập liệu được bố trí rõ ràng, nút chức năng nổi bật giúp thao tác nhanh chóng. Khi người dùng nhập sai thông tin xác thực, hệ thống hiển thị thông báo lỗi ngay bên dưới ô nhập, đồng thời làm nổi bật trường dữ liệu chưa hợp lệ để hỗ trợ nhận biết và chỉnh sửa kịp thời. Đối với màn hình đăng ký tài khoản, ứng dụng cung cấp biểu mẫu gồm các thông tin cơ bản như họ và tên, số điện thoại, ngày sinh, mật khẩu và xác nhận mật khẩu, giúp người dùng tạo tài khoản mới một cách đầy đủ và thuận tiện. Nhìn chung, cụm giao diện này thể hiện quy trình tiếp cận ban đầu của người dùng với hệ thống, đảm bảo tính dễ sử dụng, nhất quán và hỗ trợ tốt cho trải nghiệm đăng nhập, xử lý lỗi và đăng ký tài khoản.

Các màn hình quản lý thiết bị và phân quyền

A Chưa có thiết bị

Thiết bị đã liên kết    

Thông tin tài khoản

Tên: Đặng Thành Tài
Mã tài khoản: user-532aab58
SDT: +84793963498
Ngày sinh: 20/03/2002
Quyền trên thiết bị hiện tại: Chưa chọn thiết bị
Số thiết bị đã liên kết: 0
Chưa chọn thiết bị nào.

 **Mở trang tài khoản**

 Tìm theo tên thiết bị, mã thiết bị, tài khoản liên kết...

**Bạn chưa có thiết bị nào**

Bạn có thể thêm thiết bị bằng mã thiết bị để liên kết thiết bị. Nếu bạn chỉ cần quyền xem, vui lòng liên hệ chủ thiết bị để được cấp quyền người xem.

Liên kết thiết bị

[Xem hướng dẫn liên kết](#)

B Liên kết thiết bị

15:11   2 devices   

← Liên kết thiết bị

**Eldercare**

Xác thực thiết bị bằng mã ghép nối để hoàn tất quyền truy cập an toàn.

Liên kết thiết bị bằng mã ghép nối

Nhập mã thiết bị và mã ghép nối để nhận quyền trên thiết bị, sau đó ứng dụng sẽ tải lại danh sách thiết bị của bạn.

Mã thiết bị

Mã ghép nối

 **Liên kết thiết bị**

C Lỗi ghép nối

← Liên kết thiết bị

**Eldercare**

Xác thực thiết bị bằng mã ghép nối để hoàn tất quyền truy cập an toàn.

Liên kết thiết bị bằng mã ghép nối

Nhập mã thiết bị và mã ghép nối để nhận quyền trên thiết bị, sau đó ứng dụng sẽ tải lại danh sách thiết bị của bạn.

Mã thiết bị

Mã ghép nối

Mã ghép nối không đúng hoặc dữ liệu gửi lên chưa hợp lệ.

Liên kết thiết bị

D Quản lý người xem

← Quản lý người xem

Shared Wrist 401

Mã thiết bị: dev-shared-401
Quyền trên thiết bị hiện tại: Chủ thiết bị

Thêm người xem

Nhập mã tài khoản của người cần được cấp quyền xem thiết bị này.

Mã tài khoản

 **Thêm người xem**

Viewer Demo 401

+84922200401 | Quyền trên thiết bị này: Người xem 

Hình 3.22. Cụm màn hình quản lý thiết bị, liên kết thiết bị và phân quyền người xem.

Mô tả: Cụm màn hình quản lý thiết bị và phân quyền của ứng dụng được thiết kế nhằm hỗ trợ người dùng theo dõi trạng thái thiết bị, thực hiện liên kết và quản lý quyền truy cập một cách thuận tiện. Ở màn hình “Chưa có thiết bị”, hệ thống hiển thị trạng thái chưa phát hiện thiết bị nào được kết nối, đồng thời cung cấp nút thao tác để người dùng bắt đầu quá trình liên kết thiết bị mới. Tại màn hình “Liên kết thiết bị”, người dùng được yêu cầu nhập mã ghép nối để kết nối thiết bị với tài khoản, giao diện được bố trí đơn giản, tập trung vào thao tác chính nhằm giảm nhầm lẫn khi sử dụng. Khi quá trình ghép nối gặp sự cố, màn hình “Lỗi ghép nối” sẽ hiển thị thông báo lỗi ngay bên dưới ô nhập, giúp người dùng dễ dàng nhận biết nguyên nhân và thử lại. Bên cạnh đó, màn hình “Quản lý người xem” cho phép chủ tài khoản thêm và kiểm soát danh sách những người được chia sẻ quyền theo dõi thiết bị, qua đó hỗ trợ việc phân quyền truy cập rõ ràng và phù hợp với nhu cầu sử dụng thực tế. Tổng thể, cụm giao diện này thể hiện quy trình quản lý thiết bị từ khi chưa kết nối, thực hiện ghép nối, xử lý lỗi đến chia sẻ quyền truy cập, góp phần nâng cao tính trực quan và hiệu quả trong quá trình vận hành hệ thống.

Các màn hình theo dõi và điều hướng dữ liệu sức khỏe

A Danh sách thiết bị

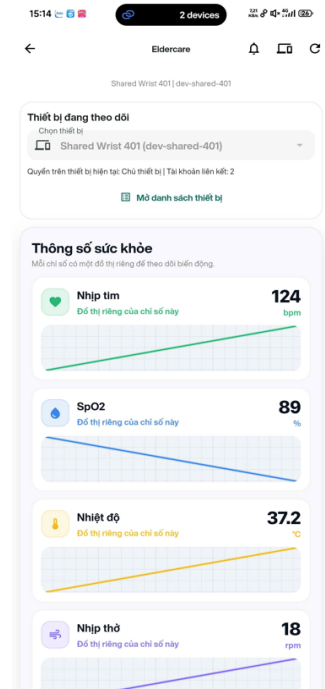
Thiết bị đã liên kết

Thông tin tài khoản
 Tên: Đặng Thành Tài
 Mã tài khoản: user-owner-401
 SĐT: +84911100401
 Ngày sinh: 14/06/1955
 Quyền trên thiết bị hiện tại: Chủ thiết bị
 Số thiết bị đã liên kết: 1
 Đang theo dõi: Shared Wrist 401 (dev-shared-401)
[Mở trang tài khoản](#)

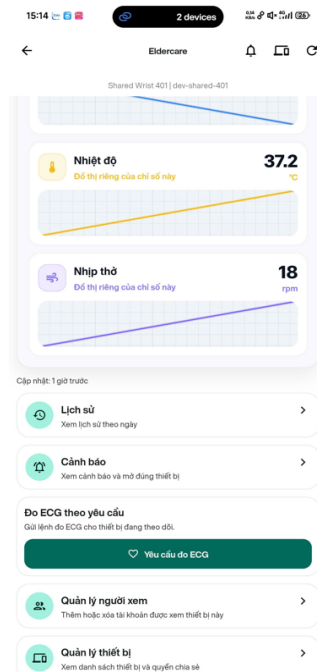
Tìm theo tên thiết bị, mã thiết bị, tài khoản liên kết...

Shared Wrist 401
 Mã thiết bị: dev-shared-401 [Đang theo dõi](#)
 Quyền trên thiết bị này: Chủ
 Người xem đang được chia sẻ
 Viewer Demo 401 | Quyền trên thiết bị này: Người xem | +8492222
[Theo dõi thiết bị này](#) [Quản lý người xem](#)

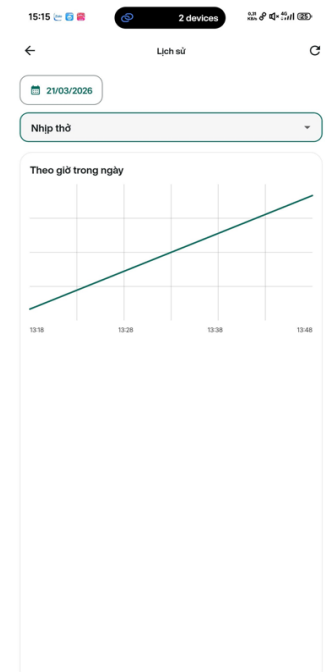
B HomePage - chỉ số chính



C Hành động nhanh

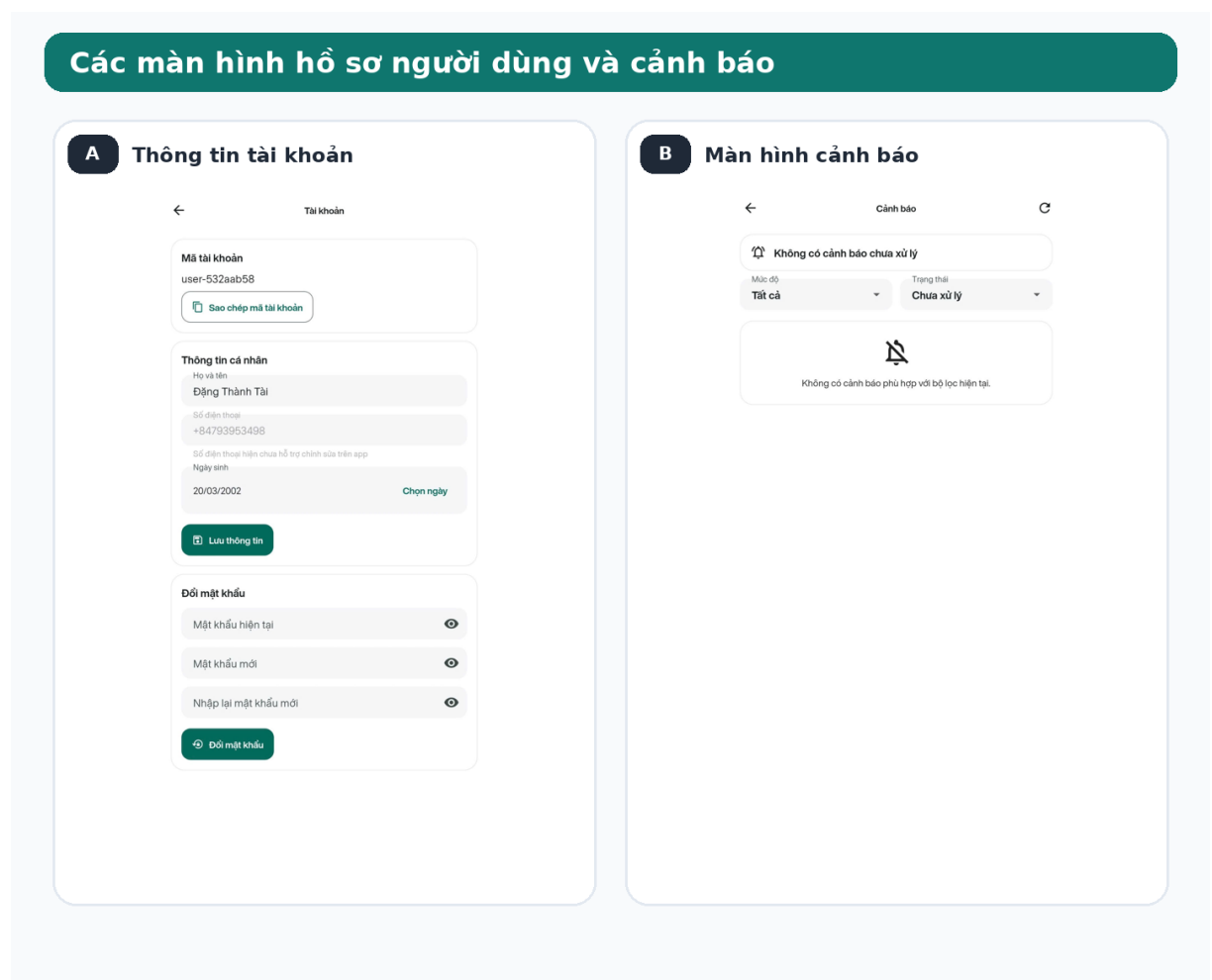


D Lịch sử theo ngày



Hình 3.23. Cụm màn hình theo dõi thiết bị, dashboard sức khỏe và lịch sử theo ngày.

Mô tả: Cụm màn hình theo dõi và điều hướng dữ liệu sức khỏe được thiết kế nhằm hỗ trợ người dùng quan sát nhanh tình trạng thiết bị và các chỉ số sức khỏe quan trọng của đối tượng theo dõi. Ở màn hình danh sách thiết bị, hệ thống hiển thị thông tin cơ bản của thiết bị đã liên kết, trạng thái chia sẻ và các nút chức năng để truy cập nhanh đến phần theo dõi hoặc quản lý quyền xem. Màn hình trang chủ đóng vai trò như một dashboard tổng quan, trình bày các chỉ số sức khỏe chính dưới dạng thẻ thông tin kết hợp đồ thị xu hướng, giúp người dùng dễ dàng nắm bắt sự thay đổi của dữ liệu theo thời gian. Bên cạnh đó, màn hình hành động nhanh cung cấp các lối tắt đến những chức năng thường dùng như xem thông tin liên quan, xử lý cảnh báo, theo dõi thiết bị hoặc quản lý người xem, từ đó rút ngắn thao tác và tăng tính thuận tiện khi sử dụng. Cuối cùng, màn hình lịch sử theo ngày cho phép người dùng lựa chọn từng loại chỉ số và xem biểu đồ biến thiên trong ngày, hỗ trợ việc theo dõi chi tiết, đối chiếu dữ liệu và phát hiện các bất thường sức khỏe một cách trực quan. Nhìn chung, cụm giao diện này thể hiện rõ vai trò của ứng dụng trong việc tập trung hóa dữ liệu, hỗ trợ điều hướng nhanh và nâng cao hiệu quả giám sát sức khỏe từ xa.



Hình 3.24. Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng.

Mô tả: Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng được xây dựng nhằm hỗ trợ người dùng quản lý thông tin cá nhân và theo dõi các thông báo bất thường một cách thuận tiện. Ở màn hình thông tin tài khoản, hệ thống hiển thị mã tài khoản cùng các thông tin cá nhân cơ bản như họ và tên, số điện thoại, ngày sinh, đồng thời cho phép người dùng cập nhật dữ liệu và thực hiện thay đổi mật khẩu ngay trong cùng một giao diện. Cách bố trí các trường thông tin rõ ràng, dễ thao tác, giúp nâng cao tính thuận tiện trong quá trình quản lý hồ sơ cá nhân. Bên cạnh đó, màn hình cảnh báo được thiết kế để tổng hợp các thông báo liên quan đến trạng thái sức khỏe hoặc thiết bị, cho phép người dùng lọc theo mức độ và tình trạng xử lý nhằm tra cứu thông tin nhanh chóng hơn. Trong trường hợp chưa có cảnh báo phù hợp, hệ thống hiển thị trạng thái rỗng để thông báo rằng hiện không có sự kiện cần xử lý. Nhìn chung, cụm giao diện này góp phần hoàn thiện chức năng quản lý tài khoản và hỗ trợ người dùng theo dõi cảnh báo một cách trực quan, từ đó nâng cao hiệu quả sử dụng và khả năng giám sát trong hệ thống.

3.7 Thiết kế tích hợp phần cứng – phần mềm

Thiết kế tích hợp của Eldercare được xây dựng trên một vòng kín dữ liệu. Vòng kín này bắt đầu từ sự kiện vật lý trên cơ thể, đi vào cảm biến, được số hóa ở thiết bị, được hợp nhất và truyền lên backend, được lưu và phân tích ngưỡng, rồi được trình bày cho người dùng. Sau đó, từ chính ứng dụng, người dùng có thể phát ra một yêu cầu mới như ECG on-demand; yêu cầu này quay ngược trở lại backend, đi xuống thiết bị và tái tạo nên một chu trình mới. Chính đặc điểm vòng kín đó làm cho hệ thống mang bản chất IoT rõ ràng hơn rất nhiều so với mô hình gửi dữ liệu một chiều đơn giản.

Một hệ quả quan trọng của thiết kế tích hợp là ranh giới giữa phần cứng và phần mềm không nằm ở “bo mạch” hay “API”, mà nằm ở hợp đồng dữ liệu và trạng thái. Chỉ khi readings được chuẩn hóa ổn định, command có vòng đời rõ ràng, session người dùng được quản lý đúng và app bind đúng device scope thì hệ thống mới vận hành trơn tru. Vì vậy, phân tích hợp là nơi có thể nhìn thấy rõ nhất mức độ hài hòa giữa phần cứng và phần mềm của đề tài.

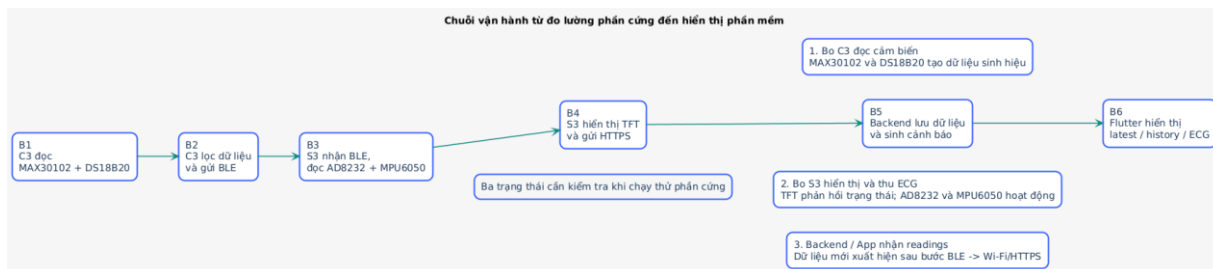
3.7.1 Chuỗi vận hành từ đo lường đến hiển thị

Trong một chu kỳ đo điển hình, bo ESP32-C3 là nút hoạt động đầu tiên. Nó đọc dữ liệu từ MAX30102 và DS18B20, có thể áp dụng bước lọc hoặc đóng gói sơ bộ, sau đó gửi gói BLE sang bo ESP32-S3. Bo S3 nhận gói BLE, đọc thêm AD8232 và MPU6050, đồng thời cập nhật màn hình TFT để phản ánh tức thời trạng thái BLE, Wi-Fi, nhịp tim, SpO2, nhiệt độ và sự kiện ngã. Đây là giai đoạn tích hợp phần cứng ở cấp cục bộ.

Khi readings đã được hợp nhất, bo S3 trở thành cầu nối sang lớp phần mềm. Nó gửi dữ liệu bằng HTTPS đến backend, nơi readings được xác thực, lưu trữ và kiểm tra ngưỡng. Nếu có bất thường, hệ thống sinh cảnh báo trong alerts. Sau đó, ứng dụng Flutter truy vấn latest, history, alerts hoặc ECG để hiển thị lại cho người dùng. Nhìn toàn bộ chuỗi, có thể thấy bo C3 và bo S3 phụ trách

“tạo dữ liệu thật”, backend phụ trách “tổ chức dữ liệu thật”, còn Flutter phụ trách “diễn giải dữ liệu thật cho người dùng”.

Việc biểu diễn chuỗi vận hành theo các bước khối như B1 đến B6 có ý nghĩa lớn về kiểm thử. Khi gặp lỗi, người thực hiện có thể đối chiếu xem hệ thống dừng ở bước nào: C3 không đọc được cảm biến, BLE không tới S3, S3 không gửi được HTTPS, backend không lưu được readings hay Flutter không hiển thị đúng current device. Chính khả năng truy vết theo bước như vậy làm cho kiến trúc của đề tài có tính khả kiểm cao.



Hình 3.25. Chuỗi vận hành từ đo lường phần cứng đến hiển thị phần mềm.

3.7.2 Đồng bộ trạng thái, lỗi và cơ chế phục hồi

Một hệ thống IoT thực tế luôn phải giả định rằng một trong các lớp có thể trễ, mất kết nối hoặc gửi lại dữ liệu. Ở phía thiết bị, BLE giữa C3 và S3 có thể bị gián đoạn ngắn; ở phía S3, Wi-Fi có thể không kết nối; ở backend, MongoDB hoặc Redis có thể tạm thời chưa sẵn sàng; ở app, session người dùng có thể hết hạn. Thiết kế tích hợp của Eldercare giải quyết vấn đề này bằng cách để mỗi lớp có cơ chế phản ánh trạng thái riêng nhưng vẫn dựa trên cùng một logic dữ liệu chung.

Bo S3 dùng TFT để hiển thị nhanh tình trạng BLE và Wi-Fi. Backend cung cấp /live, /ready, /health và logging để hỗ trợ kiểm tra runtime. Ứng dụng Flutter duy trì cơ chế refresh token, cache và empty state để phản ánh các tình huống lỗi mà không làm người dùng hiểu sai rằng hệ thống “không hoạt động”. Đối với ECG on-demand, collection device_commands còn lưu trạng thái queued, dispatched, acked, completed, failed hoặc expired, giúp chu kỳ lệnh có vòng đời rõ ràng ngay cả khi thiết bị hoặc mạng gặp trục trặc.

Cách tổ chức như vậy làm cho hệ thống không dựa vào một lớp duy nhất để phản ánh lỗi. Thay vào đó, mỗi lớp nói rõ phân trách nhiệm của mình: C3 và S3 chỉ báo lỗi cục bộ, backend chỉ báo lỗi ingest hoặc readiness, app chỉ báo lỗi phiên và hiển thị. Khi ghép lại, người vận hành có thể xác định lỗi thuộc phần cứng, thuộc lớp mạng hay thuộc phần mềm ứng dụng một cách tương đối nhanh.

3.7.3 Phần cứng và phần mềm trong thiết kế

Tiêu chí đầu tiên là phân vai rõ ràng. Phần cứng chịu trách nhiệm đo và hợp nhất tín hiệu ở mức gần thời gian thực; backend chịu trách nhiệm bảo mật, lưu trữ, điều phối và cảnh báo; Flutter chịu trách nhiệm trải nghiệm và trực quan hóa. Không có lớp nào “nuốt” vai trò của lớp còn lại. Nhờ đó, báo cáo và cả hệ

thống tránh được tình trạng backend gánh mọi thứ còn phần cứng chỉ làm nhiệm vụ tượng trưng.

Tiêu chí thứ hai là dữ liệu được thiết kế như hợp đồng chung giữa hai phía. Dù readings đến từ MAX30102, DS18B20, AD8232 hay MPU6050, chúng đều phải đi vào cùng một cấu trúc logic để backend hiểu, MongoDB lưu và app truy vấn. Điều này bảo đảm phần mềm được xây trên dữ liệu thật của phần cứng, chứ không phải trên dữ liệu giả lập tách rời sản phẩm.

Tiêu chí thứ ba là khả năng kiểm chứng. Một kiến trúc chỉ đẹp trên sơ đồ nhưng không thể kiểm thử theo chuỗi thì chưa phải kiến trúc tốt cho đồ án. Ở Eldercare, người thực hiện có thể theo dõi trạng thái C3, S3, TFT, backend và app theo từng bước. Chính yếu tố khả kiểm này làm cho sự cân bằng phần cứng – phần mềm trở nên thuyết phục về mặt học thuật hơn bất kỳ lời mô tả chung chung nào.

3.7.4 Phân vai tài nguyên

Về phần cứng, bo ESP32-C3 được giao nhiệm vụ hẹp nhưng rõ: đọc sinh hiệu cơ bản và gửi BLE. Bo ESP32-S3 gánh các tác vụ nặng hơn như gateway, hiển thị TFT, kết nối Wi-Fi, đọc ECG và MPU6050. Cách phân vai này cho phép tối ưu tài nguyên của từng bo theo đúng ưu thế phần cứng, thay vì cố ép một vi điều khiển duy nhất làm mọi việc.

Ở phần mềm, backend cũng được phân vai tương tự. API layer chỉ tiếp nhận request; service layer xử lý nghiệp vụ; db layer giữ persistence; app chỉ tập trung vào trải nghiệm hiển thị. Điều này tạo ra sự tương đồng thú vị giữa hai phía: cũng giống như phần cứng tách thành node đo và node gateway, phần mềm cũng tách thành các lớp có trách nhiệm riêng. Nhờ vậy, toàn hệ thống tránh được tình trạng dồn tải hoặc dồn trách nhiệm vào một điểm duy nhất.

Sự tương thích giữa hai cách phân vai đó là một biểu hiện rõ của sự hài hòa. Khi bo S3 đóng vai gateway, backend có thể coi nó như client thiết bị chính thay vì phải xử lý nhiều luồng cảm biến rời. Khi app bám theo device scope, dữ liệu từ hai bo được nhìn như một thiết bị giám sát thống nhất thay vì như hai board tách biệt. Đây là cách các quyết định phân vai ở hai phía tự nâng đỡ lẫn nhau.

3.7.5 Dữ liệu và giao tiếp

Một hệ thống tích hợp chỉ ổn định khi phần cứng và phần mềm nói cùng một “ngôn ngữ dữ liệu”. Trong Eldercare, readings gửi từ thiết bị phải chứa được các trường mà backend hiểu và có thể lưu, đồng thời backend phải trả về cấu trúc mà Flutter có thể hiển thị ổn định. Điều này có vẻ hiển nhiên, nhưng trên thực tế nhiều nguyên mẫu thất bại vì firmware, API và app phát triển tách rời, mỗi bên hiểu dữ liệu theo một kiểu.

Ở dự án này, hợp đồng dữ liệu được giữ ở mức đủ chặt để bảo toàn nghĩa nghiệp vụ nhưng vẫn đủ mở để mở rộng. Các chỉ số như HR, SpO2, nhiệt độ, ECG hay trạng thái ngã có thể cùng tồn tại trong readings, trong khi app vẫn truy vấn latest/history theo device. Với command queue, chính sách trạng thái

queued, dispatched, acked, completed, failed hoặc expired tạo ra một ngôn ngữ chung cho cả app, backend và thiết bị khi nói về tác vụ ECG.

Sự hài hòa ở mức giao tiếp còn thể hiện ở việc không áp đặt cùng một cơ chế xác thực cho mọi client. Thiết bị dùng X-Device-Token vì phù hợp với nhúng; người dùng dùng JWT + refresh token vì phù hợp với mobile app. Cả hai cơ chế cùng đi vào backend như một điểm hội tụ chung. Đây là ví dụ tiêu biểu cho một thiết kế vừa linh hoạt vừa có kỷ luật giữa phần cứng và phần mềm.

3.7.6 Khả năng mở rộng tương lai

Hệ thống hiện tại được xây như một nguyên mẫu, nhưng nhiều quyết định kiến trúc đã tính đến khả năng mở rộng về sau. Ở phần cứng, tách hai bo cho phép thay đổi cảm biến hoặc điều chỉnh vai trò từng node mà không phải đập bỏ toàn bộ nguyên mẫu. Ở backend, MongoDB document model và command queue theo collection mở ra khả năng bổ sung thêm loại readings và loại command. Ở app, provider graph cho phép thêm màn hình hoặc trạng thái mới mà không cần phá cấu trúc nền.

Điều quan trọng là các hướng mở rộng này không làm mất đi sự cân bằng hiện tại. Chẳng hạn, nếu sau này thêm cảm biến huyết áp hoặc tăng số thiết bị, ta vẫn có thể giữ logic rằng thiết bị tạo dữ liệu, backend điều phối dữ liệu và app hiển thị dữ liệu. Nói cách khác, hệ thống có thể phình to về quy mô nhưng vẫn giữ đúng nguyên tắc phân vai đã được xác lập trong kiến trúc ban đầu.

Khả năng mở rộng không chỉ nằm ở việc thêm tính năng, mà còn nằm ở việc nâng cấp chất lượng vận hành. Command queue hiện dùng Mongo polling có thể được thay bằng message broker chuyên dụng; app hiện thiên về refresh có thể được tăng cường bằng push hoặc realtime stream; phần cứng hiện còn thô về cơ khí có thể được tối ưu đóng gói. Việc những hướng nâng cấp đó có thể diễn ra mà không làm sai tinh thần kiến trúc hiện tại là dấu hiệu của một thiết kế cân bằng tốt.

3.7.7 bối cảnh sử dụng cho người cao tuổi

Bối cảnh chăm sóc người cao tuổi đặt ra một yêu cầu rất thực tế: hệ thống phải đủ đơn giản ở phía sử dụng nhưng đủ đầy ở phía kỹ thuật. Phần cứng cần gọn, dễ đeo thử và có phản hồi tại chỗ; phần mềm cần rõ ràng, ít thao tác thừa và phản ánh trạng thái đủ trực quan cho người chăm sóc. Nếu chỉ chạy theo sự phức tạp kỹ thuật mà bỏ quên tính dễ dùng, hệ thống sẽ khó phát huy giá trị thực tế.

Trong Eldercare thể hiện ở nhiều điểm nhỏ nhưng quan trọng. TFT trên bo S3 giúp xác nhận readings và mạng ngay cả khi chưa cầm điện thoại. Flutter tổ chức dashboard theo các khối thông tin dễ đọc thay vì dồn thành màn hình kỹ thuật dày đặc. Backend duy trì quyền owner/viewer theo thiết bị, phù hợp với ngữ cảnh một người cao tuổi được nhiều người thân cùng theo dõi. Những chi tiết ấy cho thấy thiết kế không chỉ xuất phát từ công nghệ mà còn từ hoàn cảnh sử dụng.

Đặt hệ thống vào đúng bối cảnh người dùng mục tiêu cũng làm cho báo cáo thuyết phục hơn. Nó chứng minh rằng phần cứng được chọn không chỉ vì

rẻ hay phổ biến, phần mềm được chọn không chỉ vì quen tay lập trình, mà cả hai phía đều phục vụ cho cùng một bài toán chăm sóc sức khỏe từ xa theo cách vừa khả thi vừa dễ tiếp cận.

Bảng 3.12. Đối chiếu đóng góp của phần cứng và phần mềm trong kiến trúc Eldercare.

Tiêu chí	Đóng góp từ phần cứng	Đóng góp từ phần mềm
Sinh dữ liệu	Thu nhận HR, SpO2, nhiệt độ, ECG, chuyển động	Chuẩn hóa, lưu trữ và trả dữ liệu có ngữ nghĩa
Điều phối	BLE giữa C3 – S3; hiển thị cục bộ trên TFT	API, session, command queue, cảnh báo
Quan sát lỗi	Serial log, TFT, tín hiệu mô-đun	health check, logging, cache/error state
Mở rộng	Thay mô-đun, tối ưu PCB, đổi cảm biến	Thêm endpoint, collection, provider, dashboard
Tính khả dụng	Nguyên mẫu có thể lắp ráp, đo và chạy thử	Người dùng có thể xem dữ liệu và gửi yêu cầu từ xa

3.8 Đánh giá phương án thiết kế

Nhìn tổng thể, phương án thiết kế của Eldercare có một số điểm mạnh nổi bật. Thứ nhất, hệ thống được tổ chức theo một trục dữ liệu nhất quán từ thiết bị đến backend và ứng dụng. Thứ hai, kiến trúc hai bo giúp phần cứng có cơ sở kỹ thuật rõ ràng, giảm gánh nặng cho từng node và thuận tiện cho kiểm thử. Thứ ba, backend được xây dựng ở mức đủ trưởng thành để không chỉ lưu dữ liệu mà còn điều phối cảnh báo, phiên người dùng và ECG on-demand. Thứ tư, Flutter không chỉ đóng vai trò hiển thị mà được tổ chức như một client đúng nghĩa, có session, cache, device scope và trạng thái lỗi.

Bên cạnh điểm mạnh, kiến trúc hiện tại vẫn còn những giới hạn. Về phần cứng, nguyên mẫu chưa đạt mức hoàn thiện cơ khí như sản phẩm thương mại; nhiễu của ECG và độ ổn định của readings quang học vẫn phụ thuộc nhiều vào cách đeo và lắp ráp. Về backend, command queue vẫn dựa trên Mongo polling nên chưa tối ưu cho quy mô lớn. Về ứng dụng, dữ liệu vẫn thiên về refresh/polling hơn là cập nhật đầy thời gian thực. Tuy nhiên, những giới hạn này không phủ nhận giá trị của phương án thiết kế; trái lại, chúng làm rõ phạm vi của đề tài như một nguyên mẫu tích hợp có khả năng mở rộng về sau.

Từ góc nhìn báo cáo, cấu trúc chương mới giúp làm nổi bật đúng bản chất của hệ thống: đây là một giải pháp IoT theo dõi sức khỏe người cao tuổi, trong đó phần cứng và phần mềm phải được trình bày như một chỉnh thể. Phần cứng cung cấp readings thật, backend tạo nên hạ tầng xử lý – lưu trữ – cảnh báo, còn Flutter cung cấp điểm tiếp xúc cho người dùng. Sự phối hợp giữa ba lớp này là giá trị cốt lõi của Eldercare và cũng là đóng góp chính của đề tài ở mức nguyên mẫu.

Có thể xem Chương 3 như phần “bản thiết kế đầy đủ” của toàn bộ hệ thống. Những quyết định được trình bày ở đây sẽ là cơ sở trực tiếp cho các nội dung triển khai, kiểm thử và đánh giá ở chương sau. Đồng thời, chương cũng cho thấy đề tài đã được xây dựng theo tư duy hệ thống, chứ không phải theo tư duy ghép nối rời rạc giữa một vài công nghệ độc lập. Đây chính là nền tảng quan trọng nhất để đánh giá mức độ hoàn thiện của đồ án tốt nghiệp.

3.9 Các kịch bản vận hành điển hình và đối chiếu với kiến trúc hệ thống

Một cách đánh giá tốt mức độ hợp lý của kiến trúc là đặt hệ thống vào các kịch bản vận hành điển hình thay vì chỉ mô tả các khối chức năng trong điều kiện lý tưởng. Khi làm như vậy, có thể thấy rõ hơn vì sao một số quyết định như tách hai bo, tách cơ chế xác thực, dùng device-centric scope hay duy trì command queue lại là các lựa chọn có ý nghĩa thực tế. Các kịch bản dưới đây được xây dựng không nhằm mô phỏng lỗi ngẫu nhiên, mà nhằm đối chiếu trực tiếp giữa yêu cầu ban đầu của hệ thống với hành vi của kiến trúc đã chọn.

Ở mỗi kịch bản, điều quan trọng không chỉ là hệ thống “có chạy hay không”, mà là lớp nào chịu trách nhiệm phản ứng, lớp nào bị ảnh hưởng và lớp nào còn có thể tiếp tục vận hành độc lập. Chính khả năng phân rã này cho phép đánh giá một kiến trúc có cân bằng phần cứng – phần mềm hay không. Một kiến trúc tốt phải cho phép phát hiện điểm đứt, giới hạn phạm vi lỗi và duy trì mức dịch vụ tối thiểu ở các lớp còn lại thay vì sụp đổ đồng loạt.

3.9.1 Kịch bản vận hành bình thường của hệ thống

Trong điều kiện vận hành bình thường, người dùng hoặc người chăm sóc mở ứng dụng, phiên đăng nhập được phục hồi hoặc tạo mới thành công, danh sách thiết bị liên kết được đồng bộ và current device được chọn chính xác. Song song với đó, bo ESP32-C3 đọc MAX30102 và DS18B20 theo chu kỳ định sẵn, gửi gói BLE sang bo ESP32-S3. Bo S3 tiếp tục nhận gói này, hợp nhất cùng ECG và dữ liệu chuyển động, cập nhật TFT và gửi readings lên backend qua HTTPS. Backend xác thực thiết bị, lưu readings, kiểm tra ngưỡng cảnh báo và sẵn sàng trả latest/history/alerts/ECG cho ứng dụng.

Kịch bản vận hành bình thường cho thấy rõ sự phân vai tự nhiên giữa các lớp. C3 không cần biết gì về database hay giao diện người dùng; nó chỉ cần tạo readings đúng cấu trúc. S3 không phải gánh trách nhiệm xác thực người dùng; nó chỉ cần đóng vai gateway và hiển thị tình trạng kết nối. Backend không phải xử lý tín hiệu analog gốc; nó chỉ tiếp nhận, lưu và điều phối. Flutter không trực tiếp can thiệp vào cảm biến; nó chỉ làm việc với dữ liệu đã được chuẩn hóa. Khi mỗi lớp giữ đúng vai của mình, toàn hệ thống đạt được độ mượt cần thiết cho một nguyên mẫu.

Từ góc độ đánh giá kiến trúc, đây là kịch bản chứng minh rõ nhất rằng việc cân bằng phần cứng và phần mềm đã được thực hiện hợp lý. Nếu hệ thống được thiết kế lệch về phía phần mềm, dữ liệu thật từ thiết bị sẽ khó đi trọn vòng. Nếu hệ thống lệch về phía phần cứng, readings dù đo được cũng không thể biến thành dữ liệu theo dõi có cấu trúc trong ứng dụng. Trong kịch bản bình thường,

sự liên mạch giữa các lớp cho thấy kiến trúc hiện tại đã giải được bài toán trung tâm của đề tài.

3.9.2 Kịch bản gián đoạn BLE giữa bo C3 và bo S3

Gián đoạn BLE giữa bo C3 và bo S3 là một kịch bản hoàn toàn có thể xảy ra trong quá trình chạy thử hoặc khi người dùng thay đổi vị trí đeo. Khi đó, readings sinh hiệu nền tảng từ MAX30102 và DS18B20 không còn được chuyển đều sang bo gateway. Nếu hệ thống không có sự phân lớp rõ, lỗi này rất dễ bị hiểu nhầm là lỗi của backend hoặc ứng dụng. Tuy nhiên, trong kiến trúc hiện tại, TFT trên bo S3 phản ánh được tình trạng BLE, còn backend và app có thể được quan sát để xác nhận rằng readings mới không đi lên server trong giai đoạn tương ứng.

Sự tồn tại của bo S3 như một gateway độc lập giúp kịch bản này được cô lập tốt hơn. Dù BLE gián đoạn, bo S3 vẫn có thể tiếp tục hoạt động với các mô-đun cục bộ của nó như AD8232, MPU6050 và TFT. Điều này cho thấy quyết định tách hai bo không chỉ phục vụ phân chia tài nguyên, mà còn có giá trị lớn về khả năng cô lập lỗi. Nếu dùng một bo duy nhất, việc mất tín hiệu từ một mô-đun dễ kéo theo toàn bộ hệ thống rơi vào trạng thái khó chẩn đoán hơn.

Ở góc độ báo cáo, kịch bản BLE gián đoạn giúp nhấn mạnh rằng lớp phân cứng của Eldercare không bị che khuất bởi lớp phần mềm. Những lỗi riêng của lớp thiết bị vẫn có cách quan sát và lý giải độc lập trước khi dữ liệu chạm đến backend. Điều đó làm cho báo cáo cân bằng hơn, vì phần cứng được nhìn nhận như một lớp có quy luật và trạng thái riêng chứ không chỉ là “nguồn dữ liệu mặc định” cho phần mềm.

3.9.3 Kịch bản mất Wi-Fi hoặc backend chưa sẵn sàng

Trong điều kiện mạng Wi-Fi gián đoạn hoặc backend chưa sẵn sàng, luồng HTTPS từ bo S3 lên server bị ngắt. Đây là kịch bản quan trọng vì nó kiểm tra xem kiến trúc của hệ thống có bắt người dùng và người phát triển phụ thuộc hoàn toàn vào server hay không. Với Eldercare, khi mất Wi-Fi, bo S3 vẫn có thể duy trì hiển thị tại chỗ trên TFT để người vận hành biết rằng readings cục bộ vẫn đang được đọc, trong khi lớp đồng bộ lên backend chỉ tạm thời bị đình lại.

Ở phía backend, các endpoint /live, /ready và /health giúp xác nhận server có thực sự mất sẵn sàng hay chỉ có vấn đề tạm thời với phụ thuộc như MongoDB hoặc Redis. Ở phía Flutter, ứng dụng có thể phản ánh việc latest chưa đổi hoặc đang hiển thị dữ liệu cache. Khi ghép ba góc nhìn này lại, người vận hành có thể hiểu đây là sự cố lớp mạng hoặc lớp backend, chứ không phải cảm biến ngừng hoạt động. Điều này đặc biệt quan trọng trong các hệ thống sức khỏe vì một kết luận sai nguyên nhân dễ dẫn đến việc xử lý sai lớp kỹ thuật.

Kiến trúc hiện tại cho thấy lợi ích của việc mỗi lớp đều có cơ chế chỉ báo riêng. TFT của bo S3 là chỉ báo cho trạng thái cục bộ; health check là chỉ báo cho service; cache và empty state là chỉ báo cho ứng dụng. Chính sự đa tầng của chỉ báo giúp kịch bản mất Wi-Fi không biến thành một trạng thái “đen toàn bộ”, mà trở thành một lỗi có thể phân tích được theo tầng.

3.9.4 Kịch bản phiên ứng dụng hết hạn và cơ chế phục hồi session

Với ứng dụng di động, một tình huống phổ biến là access token hết hạn khi người dùng mở lại app sau một thời gian. Nếu ứng dụng không có cơ chế refresh hợp lý, người dùng sẽ thường xuyên bị buộc đăng nhập lại, làm giảm mạnh giá trị sử dụng của hệ thống. Trong Eldercare, SessionProvider và ApiClient phối hợp để tự động dùng refresh token xin access token mới trước khi coi phiên là không còn hợp lệ. Đây là một yếu tố phần mềm thuần túy, nhưng nó ảnh hưởng trực tiếp đến cảm nhận của người dùng đối với toàn hệ thống.

Kịch bản này cho thấy phần mềm không thể bị xem nhẹ trong một đề tài IoT. Dù thiết bị có đo dữ liệu ổn định đến đâu, nếu ứng dụng không phục hồi phiên tốt thì trải nghiệm giám sát vẫn bị gián đoạn. Mặt khác, cơ chế session này cũng phải tương thích với mô hình device-centric: sau khi phục hồi phiên, ứng dụng cần đồng bộ lại current device và ràng buộc lại các provider latest/history/alerts/ECG. Nếu không, người dùng có thể vẫn đăng nhập được nhưng nhìn thấy dữ liệu sai scope.

Việc giải quyết tốt kịch bản hết hạn phiên giúp cân bằng lại trọng số của phần mềm trong toàn bộ kiến trúc. Nó cho thấy Eldercare không chỉ mạnh ở lớp thiết bị mà còn đầu tư cho chất lượng vận hành ở lớp client. Đây là điểm quan trọng trong báo cáo vì nó phản ánh đúng tính chất của một hệ thống theo dõi từ xa: giá trị cuối cùng nằm ở khả năng người dùng tiếp cận dữ liệu đúng lúc, chứ không chỉ ở việc readings tồn tại trong server.

3.9.5 Kịch bản readings trùng lặp và bảo đảm tính đúng đắn dữ liệu

Thiết bị IoT trong điều kiện mạng không ổn định thường có xu hướng gửi lại gói tin khi chưa chắc backend đã nhận. Nếu hệ thống không có cơ chế nhận diện readings trùng lặp, dữ liệu lịch sử sẽ bị phồng lên sai sự thật, các cảnh báo có thể bị lặp và dashboard latest có thể phản ánh readings không nhất quán. Trong Eldercare, cặp khóa device_id – seq và các unique partial index đóng vai trò xương sống cho chiến lược dedupe.

Kịch bản này là ví dụ rõ ràng cho thấy quyết định thiết kế dữ liệu không thể tách khỏi bản chất của phần cứng. Dedupe không phải là “tối ưu cơ sở dữ liệu” thuần túy; nó là phản ứng của backend trước hành vi rất thực của thiết bị ngoài đời. Khi nhìn theo cách đó, có thể thấy phần mềm của hệ thống được xây trên logic thực nghiệm của phần cứng chứ không phải trên giả định lý tưởng. Đây là biểu hiện rõ của một kiến trúc hài hòa giữa hai phía.

Ngoài lợi ích về dữ liệu, cơ chế dedupe còn giúp việc đánh giá hệ thống đáng tin cậy hơn. Khi nhìn vào history và alert, người đọc có thể tin rằng hệ thống đang phản ánh trạng thái sức khỏe chứ không phản ánh số lần gói tin bị gửi lại. Điều này rất quan trọng đối với đồ án, vì mọi đánh giá triển khai và kiểm thử về sau đều dựa trên giả định rằng readings trong database là đủ sạch để có ý nghĩa phân tích.

3.9.6 Kịch bản thay thế mô-đun, bảo trì và mở rộng thiết bị

Một nguyên mẫu tốt không chỉ chạy được một lần trong điều kiện lý tưởng mà còn có khả năng thay thế mô-đun hoặc mở rộng theo nhu cầu. Trong Eldercare, các mô-đun phần cứng được nối theo những giao tiếp phổ biến như I²C, 1-Wire, SPI và GPIO, khiến việc thay cảm biến hoặc điều chỉnh vị trí lắp ráp có tính khả thi. Ở phía backend, dữ liệu readings được giữ trong document model đủ linh hoạt để thêm trường mới mà không phải phá vỡ toàn bộ lược đồ. Ở phía app, mô hình provider cho phép mở rộng thêm dữ liệu hoặc widget theo scope thiết bị.

Kịch bản bảo trì này cho thấy giá trị của việc không dồn quá nhiều logic vào một lớp. Nếu muốn thay đổi cảm biến nhiệt độ hoặc chỉnh thuật toán đọc quang học, ta chủ yếu tác động đến bo C3 và cấu trúc readings. Nếu muốn mở rộng loại command, ta chủ yếu tác động đến backend và một phần app. Nếu muốn cải thiện dashboard, ta có thể tác động đến Flutter mà không làm thay đổi phần cứng. Tính tách lớp này là bằng chứng rõ cho chất lượng thiết kế hệ thống.

Từ góc nhìn tương lai, kịch bản thay thế và mở rộng còn là cơ sở để đề tài có thể phát triển sau khi kết thúc đồ án. Một thiết kế chỉ có giá trị khi nó mở ra khả năng kế thừa. Trong Eldercare, sự kế thừa đó được tạo nên không phải bằng những lời hứa chung chung, mà bằng các điểm móc kiến trúc rất cụ thể ở cả phần cứng và phần mềm.

Bảng 3.13. Đối chiếu các kịch bản vận hành với lớp chịu trách nhiệm chính.

Kịch bản	Lớp chịu trách nhiệm chính	Cơ chế quan sát / phản hồi
Vận hành bình thường	Toàn bộ hệ thống	TFT, backend readings, dashboard app
Gián đoạn BLE	Phần cứng C3 – S3	Trạng thái BLE trên TFT, readings thiếu ở backend
Mất Wi-Fi / backend chưa sẵn sàng	Gateway + backend	TFT Wi-Fi, /ready, cache trên app
Phiên app hết hạn	Flutter + backend auth	Refresh token, trạng thái session
ECG timeout / failed	Backend queue + S3	device_commands, ACK, thông báo trạng thái
Readings trùng lặp	Backend + MongoDB	Dedupe theo seq, lịch sử không bị nhân bản
Bảo trì / mở rộng mô-đun	Thiết bị + backend + app	Tính mô-đun của từng lớp

3.10 Phân tích bổ sung về vận hành, tính khả dụng và khả năng kế thừa

Sau khi xem xét kiến trúc ở mức chức năng, cần có thêm một lớp phân tích về vận hành và khả năng kế thừa để làm rõ hơn giá trị của phương án thiết kế. Đây là lớp nội dung giúp liên kết các quyết định kỹ thuật với bối cảnh sử dụng thực tế. Một nguyên mẫu chỉ dừng ở việc chạy được trong buổi demo là chưa đủ. Đối với hệ thống theo dõi sức khỏe từ xa, cần nhìn thêm ba khía cạnh: hệ thống có thân thiện với quá trình chạy thử và bảo trì hay không, có tạo điều kiện để người dùng cuối tiếp cận một cách tự nhiên hay không, và có mở ra con đường nâng cấp sau đó hay không.

Phần bổ sung này đặc biệt cần thiết vì Eldercare là một đề tài giao thoa giữa phần cứng và phần mềm. Một số giá trị của hệ thống không hiện ra ngay trong sơ đồ use case hay bảng collection dữ liệu, mà hiện ra ở cách nguyên mẫu được chạy thử, cách lỗi được nhận biết, cách người dùng hiểu trạng thái hệ thống và cách các thành phần có thể được thay thế hoặc mở rộng trong tương lai. Nói cách khác, đây là lớp phân tích nhằm trả lời câu hỏi: kiến trúc hiện tại có thực sự “sống được” ngoài trang báo cáo hay không.

3.10.1 Khả năng vận hành và chạy thử theo từng lớp

Một trong những điểm mạnh của Eldercare là người thực hiện có thể chạy thử theo từng lớp chứ không buộc phải tích hợp toàn bộ ngay từ đầu. Ở lớp phần cứng, bo C3 có thể được kiểm tra riêng với MAX30102 và DS18B20, trong khi bo S3 có thể được kiểm tra riêng với TFT, AD8232, MPU6050 và kết nối Wi-Fi. Ở lớp backend, có thể kiểm tra /live, /ready, API auth và đường ghi readings độc lập với app. Ở lớp Flutter, có thể kiểm tra đăng nhập, danh sách thiết bị và dashboard bằng dữ liệu đã có sẵn trong backend mà không nhất thiết phải chạy đủ phần cứng ở mọi thời điểm.

Khả năng chạy thử theo lớp như vậy giúp giảm mạnh chi phí tích hợp. Khi hệ thống gặp lỗi, người thực hiện không bị đẩy vào tình huống “mọi thứ đều có thể sai”. Thay vào đó, từng lớp có thể được xác minh hoặc loại trừ dần. Đây là một ưu điểm rất thực dụng đối với đề án tốt nghiệp, vì thời gian và nguồn lực luôn hữu hạn, trong khi hệ thống tích hợp giữa điện tử – firmware – backend – mobile vốn có xác suất lỗi chéo rất cao.

Từ góc nhìn kiến trúc, khả năng chạy thử theo lớp là dấu hiệu cho thấy sự phân tách trách nhiệm được thực hiện tốt. Nếu phần cứng và phần mềm bị buộc chặt quá mức, gần như không thể có bước kiểm thử cục bộ có ý nghĩa. Eldercare tránh được vấn đề đó nhờ cách chia hai bo, chia API cho app và thiết bị, đồng thời tổ chức Flutter theo provider độc lập tương đối với backend thật.

3.10.2 Tính khả dụng và mức độ thân thiện với người dùng

Trong các hệ thống sức khỏe tại nhà, tính khả dụng của giao diện và tính dễ hiểu của trạng thái thiết bị là yếu tố ảnh hưởng trực tiếp đến giá trị ứng dụng. Eldercare có hai lớp giao diện song song: giao diện tại chỗ trên TFT và giao diện từ xa trên Flutter. Lớp tại chỗ giúp người thực hiện hoặc người dùng kỹ thuật xác nhận nhanh trạng thái BLE, Wi-Fi và readings cơ bản. Lớp từ xa giúp

người chăm sóc truy cập dữ liệu theo lịch sử, cảnh báo và ECG từ điện thoại. Cách tổ chức này giảm bớt gánh nặng lên từng lớp giao diện riêng lẻ.

Về UX, ứng dụng Flutter của hệ thống đi theo hướng làm rõ trạng thái. Một hệ thống giám sát sức khỏe không nên im lặng khi chưa có thiết bị, chưa có readings hoặc khi session hết hạn, vì người dùng sẽ dễ hiểu sai rằng hệ thống không hoạt động. Những màn hình empty state, lỗi ghép nối và lỗi xác thực trong ứng dụng là cần thiết để bắc cầu giữa trạng thái kỹ thuật và nhận thức của người dùng cuối. Đây là điểm làm cho phần mềm của Eldercare gần với thực tế sử dụng hơn so với các ứng dụng chỉ thiết kế theo trường hợp lý tưởng.

Tính khả dụng còn liên quan đến phần cứng. Một bo mạch có readings nhưng không có phản hồi trực quan rất khó chạy thử ngoài phòng lab. Việc bo S3 có TFT không chỉ phục vụ đẹp mắt, mà còn là một lựa chọn kiến trúc để tăng khả năng sử dụng và khả năng kiểm tra. Nhờ vậy, phần cứng và phần mềm cùng hỗ trợ cho tính khả dụng của hệ thống thay vì chỉ một phía phải gánh toàn bộ trải nghiệm.

3.10.3 Bảo trì, thay thế mô-đun và giá trị kế thừa

Một nguyên mẫu có giá trị dài hơn chính nó khi các thành phần của nó có thể được bảo trì hoặc thay thế mà không làm sụp đổ toàn bộ kiến trúc. Với Eldercare, mô-đun phần cứng có thể được thay thế ở mức tương đối độc lập: cảm biến nhiệt độ, mô-đun chuyển động hoặc thậm chí front-end ECG đều có thể được nâng cấp miễn là readings đầu ra vẫn tôn trọng hợp đồng dữ liệu chung. Điều này mở ra khả năng chuyển từ mô-đun nguyên mẫu sang mô-đun tốt hơn về sau.

Ở phía phần mềm, việc backend dùng document model và app dùng provider graph cũng mang lại giá trị kế thừa tương tự. Backend có thể thêm collection mới, thêm trường mới hoặc thay đổi chiến lược queue mà không buộc Flutter phải viết lại toàn bộ. Flutter có thể thêm dashboard, biểu đồ hay nhóm cảnh báo mới mà vẫn dựa trên current device và session đã có. Đây là kiểu kế thừa mang tính thực dụng, rất phù hợp với các đề tài có định hướng phát triển sau bảo vệ.

Chính vì vậy, giá trị của Eldercare không chỉ nằm ở chỗ nó chứng minh được một hệ thống có thể hoạt động, mà còn ở chỗ nó đặt nền cho một nhánh phát triển tiếp theo. Đây là lý do phần bảo trì và kế thừa xứng đáng được trình bày như một nội dung kiến trúc chứ không chỉ xuất hiện ở phần kiến nghị cuối báo cáo.

3.10.4 Mối liên hệ giữa phần cứng thật và contract phần mềm

Một rủi ro thường gặp trong các hệ thống tích hợp là phần mềm được phát triển dựa trên dữ liệu giả lập, trong khi phần cứng thật lại tạo ra các trạng thái không hoàn toàn trùng khớp. Eldercare giảm rủi ro này bằng cách để contract phần mềm xuất phát từ chính readings có nguồn gốc từ hai bo ESP32. Điều đó nghĩa là các collection, endpoint latest/history/ECG và cả dashboard Flutter đều được xây quanh dữ liệu mà phần cứng thực sự có thể cung cấp.

Mối liên hệ này giúp báo cáo có độ tin cậy cao hơn. Khi chương nói về `health_readings`, `alerts` hay `device_commands`, đó không phải là những cấu trúc dữ liệu trừu tượng được nghĩ ra độc lập với nguyên mẫu, mà là phản ứng của phần mềm trước cách thiết bị thực tế hoạt động. Đây là yếu tố giúp chương 3 mang màu sắc hệ thống rõ ràng hơn, đồng thời làm cho phần cứng có vị thế ngang bằng với phần mềm trong toàn bộ logic phân tích.

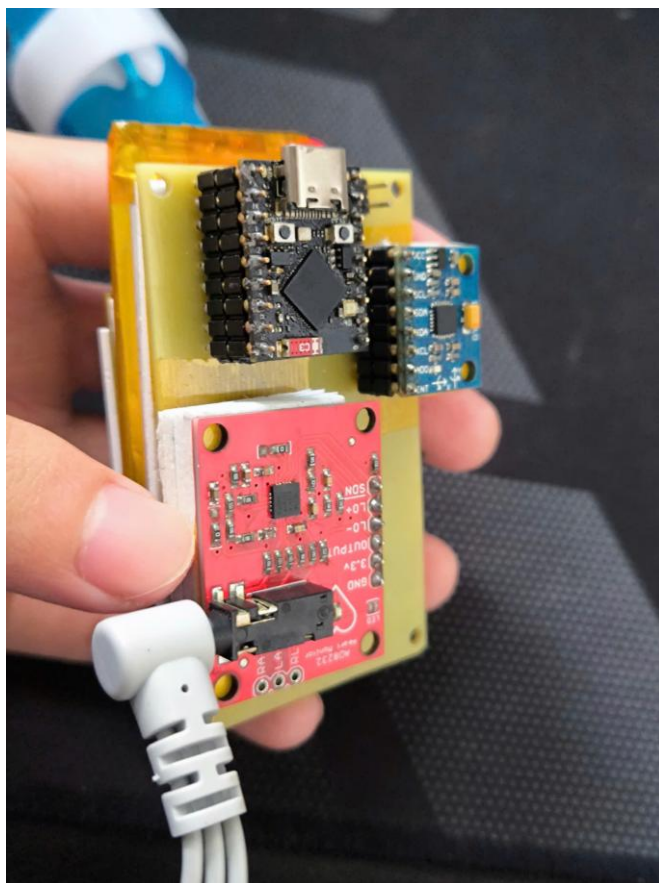
Từ góc độ học thuật, sự tương ứng giữa `contract` phần mềm và hành vi phần cứng là tiêu chí rất mạnh để đánh giá mức độ nghiêm túc của đề tài. Nó cho thấy việc thiết kế không được tiến hành theo kiểu “phần mềm một đường, phần cứng một nẻo”, mà có sự gắn kết ngay từ bước xác định dữ liệu, cách xác thực và luồng điều phối.

3.10.5 Định hướng mở rộng thành hệ thống nhiều thiết bị hoặc nhiều người theo dõi

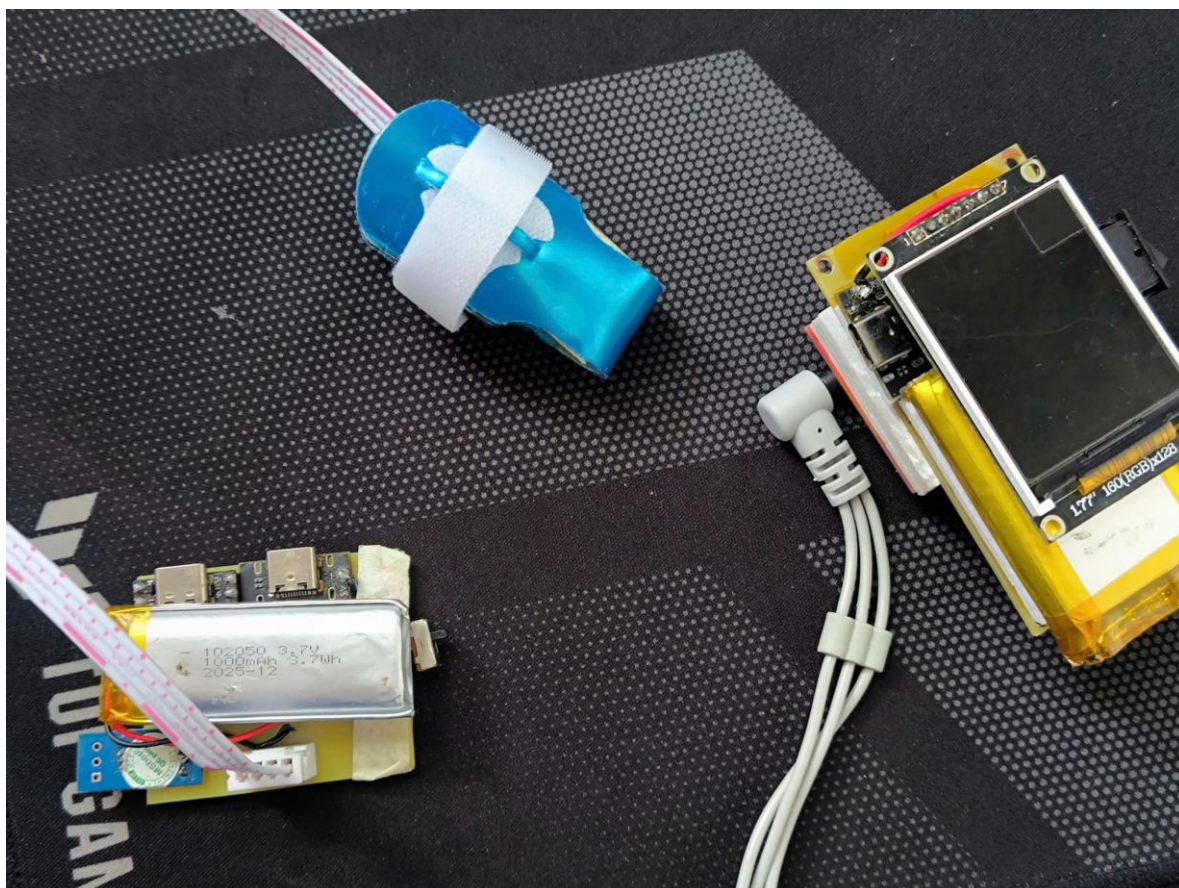
Mặc dù nguyên mẫu hiện tại được xây quanh số lượng bo và người dùng ở quy mô nhỏ, kiến trúc của Eldercare vẫn thể hiện rõ định hướng mở rộng. `Device_links` cho phép một thiết bị gắn với nhiều người dùng theo vai trò `owner` hoặc `viewer`. `Current device` trong app cho phép một tài khoản chuyển đổi giữa nhiều thiết bị liên kết. Các `readings` được lưu theo `device_id` và `timestamp`, khiến việc bổ sung thêm nhiều thiết bị về mặt dữ liệu không phá vỡ mô hình hiện tại.

Phía phần cứng cũng có thể được mở rộng theo hai hướng. Một là nhân thêm bộ đôi C3 – S3 thành nhiều nút giám sát khác nhau. Hai là trong tương lai, gom vai trò C3 và S3 vào các biến thể bo khác nếu phần cứng cho phép. Quan trọng là backend và app hiện tại đều đã vận hành theo mô hình `device-centric`, nên việc mở rộng số thiết bị chủ yếu đòi hỏi thay đổi ở lớp triển khai, không làm sai bản chất của kiến trúc.

Khả năng nhiều người theo dõi một thiết bị cũng rất phù hợp với bối cảnh gia đình có người cao tuổi. Một người thân có thể là `owner`, trong khi nhiều người còn lại chỉ có quyền `viewer`. Điều này làm cho hệ thống vừa mang tính kỹ thuật, vừa bám sát bối cảnh sử dụng thực tế – một điểm cần được nhấn mạnh trong báo cáo để cho thấy kiến trúc không chỉ giải bài toán công nghệ mà còn phản ánh đúng bối cảnh nghiệp vụ.



Hình 3.26. Cấu trúc lắp ráp thực tế của bo gateway ESP32-S3.



Hình 3.27. Toàn cảnh nguyên mẫu phân cứng khi kết nối bo đo và bo gateway.

Bảng 3.14. Tổng hợp các điểm giúp hệ thống có giá trị kế thừa sau đồ án.

Nhóm vấn đề	Hiện trạng trong nguyên mẫu	Ý nghĩa cho giai đoạn mở rộng
Phản cứng	Tách hai bo, dùng giao tiếp phổ biến, có PCB và ảnh thực tế	Dễ thay mô-đun và tối ưu đóng gói cơ khí
Backend	API phân lớp, command queue, health check, rate limit	Dễ nâng cấp hạ tầng và mở rộng số thiết bị
Dữ liệu	MongoDB theo device_id, TTL, device_links, auth_sessions	Mở rộng nhiều readings và nhiều người theo dõi
Flutter	Provider graph, session, current device, cache	Dễ thêm dashboard, biểu đồ và trạng thái mới
Tích hợp	BLE + Wi-Fi/HTTPS + REST API	Giữ được vòng kín dữ liệu khi hệ thống tăng quy mô

CHƯƠNG 4

TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Môi trường triển khai hệ thống

4.1.1 Môi trường chạy backend

Hệ thống được triển khai trong một môi trường có cấu trúc tương đối rõ ràng và phù hợp với định hướng phát triển của một hệ thống theo dõi sức khỏe từ xa có nhiều thành phần phối hợp với nhau. Ở phía backend, môi trường khuyến nghị là Linux hoặc WSL2 kết hợp với Docker Engine. Cách lựa chọn này có ý nghĩa thực tiễn vì nó giúp toàn bộ hệ thống được chạy trong một môi trường đồng nhất, hạn chế sự khác biệt giữa máy phát triển, máy kiểm thử và môi trường trình diễn. Trong trường hợp cần chạy backend theo cách truyền thống, hệ thống vẫn có thể hoạt động với Python 3.11 và pip, tuy nhiên phương án chuẩn vẫn là dựng toàn bộ stack bằng Docker Compose.

Backend hiện được xây dựng trên nền FastAPI và sử dụng image cơ sở python:3.11-slim. Các dịch vụ hạ tầng đi kèm như MongoDB và Redis cũng được tách thành các container riêng. Điều này cho thấy hệ thống không được tổ chức theo kiểu chạy mọi thứ trên một tiến trình duy nhất, mà đã có sự phân tách rõ giữa lớp ứng dụng và lớp dữ liệu hỗ trợ. Trong cấu hình triển khai, backend sử dụng nhiều biến môi trường quan trọng như `MONGO_URI`, `MONGO_DB`, `DEVICE_TOKEN_SECRET`, `JWT_SECRET`, `REFRESH_TOKEN_SECRET` và `REDIS_URL`. Việc cấu hình theo hướng này giúp hệ thống có thể thay đổi môi trường chạy mà không cần can thiệp trực tiếp vào mã nguồn, đồng thời cũng phản ánh tư duy triển khai gần với thực tế hơn so với cách hard-code giá trị trong chương trình.

Một điểm đáng chú ý là backend không khởi động một cách độc lập hoàn toàn, mà có kiểm soát phụ thuộc đối với MongoDB và Redis. Chỉ khi hai thành phần này đạt trạng thái healthy thì backend mới được đưa lên. Bản thân backend cũng có healthcheck thông qua endpoint `/ready`, nhờ đó Docker có thể nhận biết chính xác service đã sẵn sàng hay chưa. Điều này rất quan trọng trong môi trường nhiều thành phần vì nó làm giảm khả năng backend khởi động khi database hoặc Redis chưa sẵn sàng, từ đó hạn chế lỗi do phụ thuộc hạ tầng.

4.1.2 Thành phần hạ tầng đi kèm

Ngoài backend, hệ thống còn được triển khai cùng với các thành phần hạ tầng hỗ trợ khác như MongoDB, Redis, Cloudflare Tunnel và một số service phụ trợ. MongoDB đóng vai trò là lớp lưu trữ dữ liệu chính, Redis phục vụ cơ chế rate limit, còn Cloudflare Tunnel được sử dụng để công bố hệ thống ra Internet khi cần thử nghiệm hoặc trình diễn từ xa. Đây là một điểm thể hiện khá rõ ràng hệ thống đã được chuẩn bị cho các tình huống vận hành thực tế chứ không chỉ để chạy nội bộ trên một máy cá nhân.

Sau khi dựng xong hệ thống, một bước cần thiết là kiểm tra nhanh các chức năng nền tảng để xác nhận backend không chỉ chạy về mặt tiến trình mà còn hoạt động đúng ở mức API. Hiện tại hệ thống đã có cơ chế smoke test để hỗ trợ bước này. Sự có mặt của smoke test có ý nghĩa thực tế vì giúp người triển khai phát hiện sớm lỗi cấu hình, lỗi route hoặc lỗi phụ thuộc. Bộ ca kiểm thử chức năng được chuẩn hóa bổ sung trong Phụ lục D để thuận tiện cho việc đối chiếu khi nghiệm thu.

4.1.3 Các lệnh triển khai cơ bản

Quy trình triển khai của hệ thống hiện tại tương đối rõ ràng và có thể tái lập được từ mã nguồn. Nếu chạy local không dùng Docker, người triển khai cần tạo môi trường ảo Python, cài dependency và khởi động backend bằng uvicorn. Tuy nhiên, cách triển khai chính thức của hệ thống là sao chép file cấu hình môi trường, chỉnh lại các biến cần thiết rồi dùng Docker Compose để dựng toàn bộ stack. Sau khi dựng xong, hệ thống có thể được dừng, khởi động lại hoặc xóa môi trường chạy bằng các lệnh Docker Compose tương ứng.

Việc có sẵn các bước triển khai rõ ràng giúp hệ thống không rơi vào tình trạng “chỉ chạy được trên máy người phát triển”. Đây là một điểm rất quan trọng trong luận văn, vì nó chứng minh rằng sản phẩm đã có khả năng được dựng lại một cách nhất quán.

4.2 Cấu hình và khởi tạo backend

4.2.1 Biến môi trường cần thiết

Một phần quan trọng trong quá trình triển khai backend là cấu hình biến môi trường. Ở trạng thái hiện tại, hệ thống đã tách khá rõ các nhóm cấu hình liên quan đến MongoDB, Redis, JWT, refresh token, device token, rate limit, metrics, command queue và các tham số triển khai khác. Cách tổ chức này cho thấy backend được thiết kế theo hướng linh hoạt, trong đó mã nguồn giữ phần logic còn các thông số môi trường được quản lý bên ngoài.

Điều này đặc biệt có ý nghĩa với một hệ thống có nhiều lớp xác thực như hệ thống hiện tại. Access token, refresh token và device token đều là các thành phần nhạy cảm và không nên được ghi trực tiếp vào mã nguồn. Việc tách các secret thành biến môi trường không chỉ giúp an toàn hơn mà còn làm cho hệ thống dễ triển khai trên nhiều môi trường khác nhau.

4.2.2 Trình tự khởi động hệ thống

Khi khởi động hệ thống, MongoDB và Redis được đưa lên trước, sau đó backend mới khởi động. Đây không phải chỉ là một trình tự ngẫu nhiên mà là một yêu cầu kiến trúc, vì backend phụ thuộc trực tiếp vào database và Redis trong quá trình vận hành. Chỉ khi các dịch vụ phụ thuộc sẵn sàng, backend mới

có thể tạo kết nối, khởi tạo index, thiết lập rate limiter và đưa các router vào trạng thái hoạt động.

Sau khi khởi động xong, có thể kiểm tra tình trạng hệ thống bằng `/ready`, `/health`, danh sách container đang chạy và log của backend. Việc có một trình tự khởi động rõ ràng giúp giảm nhiều lỗi khó đoán trong triển khai, đặc biệt là khi hệ thống được dựng lại nhiều lần trong môi trường thử nghiệm.

4.2.3 Kiểm tra sau triển khai

Sau khi dựng xong hệ thống, một bước cần thiết là kiểm tra nhanh các chức năng nền tảng để xác nhận backend không chỉ chạy về mặt tiến trình mà còn hoạt động đúng ở mức API. Hiện tại dự án đã có cơ chế smoke test để hỗ trợ bước này. Sự có mặt của smoke test cho thấy dự án đã vượt ra khỏi mức “viết code xong” và đã bắt đầu có tư duy kiểm tra nhanh sau triển khai, điều rất cần thiết cho một hệ thống có nhiều thành phần phụ thuộc lẫn nhau.

4.3 Triển khai và kiểm tra lớp dữ liệu MongoDB – Redis

4.3.1 MongoDB trong môi trường triển khai

MongoDB trong hệ thống hiện tại không chỉ là nơi lưu readings mà là kho lưu trữ trạng thái trung tâm của gần như toàn bộ backend. Database chính của hệ thống là `wearable`, trong đó có các collection phục vụ cho nhiều lớp nghiệp vụ khác nhau như `readings`, `devices`, `users`, `alerts`, `device_commands`, `auth_sessions` và `device_links`. Điều này cho thấy lớp dữ liệu đã được tổ chức có chiều sâu, thay vì chỉ tập trung vào một bảng readings đơn giản như nhiều hệ thống thử nghiệm ban đầu.

Việc MongoDB được triển khai bằng container riêng và có volume để duy trì dữ liệu lâu dài cũng là một điểm quan trọng. Nó giúp backend có thể được dựng lại nhiều lần mà không làm mất dữ liệu hiện có, đồng thời phản ánh một cách tổ chức gần hơn với môi trường vận hành thực tế.

4.3.2 Redis trong môi trường triển khai

Redis hiện được triển khai như một thành phần phục vụ trực tiếp cho lớp bảo vệ API. Trong hệ thống này, Redis không giữ dữ liệu nghiệp vụ chính mà lưu trạng thái ngắn hạn cho rate limit. Điều này có nghĩa là Redis đang đóng vai trò thực tế trong vận hành, chứ không chỉ tồn tại ở mức cấu hình “cho có”. Khi hệ thống phải tiếp nhận request từ app, từ thiết bị và từ các kịch bản kiểm thử, rate limit bằng Redis là một lớp đệm cần thiết để tránh quá tải hoặc lạm dụng tài nguyên.

4.3.3 Kiểm tra kết nối backend với MongoDB và Redis

Một nội dung quan trọng trong triển khai là khả năng xác nhận backend có thực sự kết nối được với MongoDB và Redis hay không. Nếu backend báo lỗi ở /health hoặc /ready, việc kiểm tra đầu tiên phải hướng tới URI kết nối, credentials, trạng thái container và log của dịch vụ. Đây là loại lỗi rất điển hình trong môi trường nhiều service, và việc hệ thống đã có sẵn các hướng kiểm tra cơ bản cho thấy phần triển khai đã được nghĩ đến ở góc độ thực tế vận hành.

4.4 Triển khai và kiểm thử API backend

4.4.1 Nhóm API chính được triển khai

Backend hiện tại đã triển khai tương đối đầy đủ các nhóm API chính phục vụ cho cả người dùng và thiết bị. Ở phía app và admin, hệ thống hỗ trợ các nhóm API xác thực, người dùng, thiết bị, dữ liệu sức khỏe, cảnh báo và ECG. Ở phía thiết bị, hệ thống hỗ trợ nhóm API riêng cho readings, polling command và ACK command. Điều này cho thấy backend không còn là một dịch vụ “nhận dữ liệu rồi lưu” đơn thuần, mà đã trở thành lớp điều phối trung tâm cho nhiều tác nhân khác nhau trong hệ thống.

4.4.2 Kiểm thử health, live, ready và metrics

Việc backend có các endpoint như /health, /live, /ready và /metrics là một điểm rất có giá trị trong triển khai. Chúng không trực tiếp phục vụ nghiệp vụ người dùng nhưng lại đóng vai trò rất lớn trong debug, giám sát và xác định trạng thái hệ thống sau khi triển khai. Đây là dấu hiệu cho thấy backend đã được phát triển với nhận thức rằng một hệ thống tốt không chỉ cần chạy đúng chức năng mà còn phải dễ quan sát khi có sự cố.

4.4.3 Kiểm thử nhóm API ESP

Nhóm API dành cho ESP32 có thể được kiểm thử bằng curl hoặc trực tiếp bằng firmware. Điều này rất quan trọng vì nó cho phép kiểm tra độc lập lớp giao tiếp thiết bị – server mà không cần phụ thuộc hoàn toàn vào giao diện người dùng. Trong hệ thống hiện tại, việc có contract rõ ràng cho readings, command polling và ACK giúp lớp giao tiếp thiết bị có thể được xác minh một cách thực tế.

4.5 Triển khai và kiểm thử lớp thiết bị ESP32

Việc kiểm thử lớp thiết bị được thực hiện theo cách đi từ phần cứng đến tích hợp: trước hết kiểm tra nguồn và giao tiếp của từng mô-đun, sau đó xác nhận trao đổi BLE giữa hai bo, cuối cùng mới đánh giá khả năng hiển thị và upload dữ liệu lên backend. Cách kiểm thử này bám sát quy trình chế tạo thực tế của nguyên mẫu và giúp làm rõ mối liên hệ giữa lỗi phần cứng, lỗi truyền thông và lỗi tích hợp hệ thống.

4.5.1 Kiến trúc triển khai thiết bị thực tế

Điểm cần nhấn mạnh trong phiên bản hiện tại của hệ thống là lớp thiết bị không còn chỉ tồn tại dưới dạng ý tưởng hay contract giao thức, mà đã được hiện thực hóa bằng hai firmware cụ thể. Firmware thứ nhất chạy trên ESP32-S3 và đóng vai trò bo gateway chính, chịu trách nhiệm thu ECG, đọc MPU6050, hiển thị TFT và gửi dữ liệu lên backend. Firmware thứ hai chạy trên ESP32-C3 và đóng vai trò bo đo sinh hiệu ngoại vi, đọc MAX30102 và DS18B20 rồi gửi gói BLE sang S3. Cách tổ chức này phản ánh đúng kiến trúc phần cứng hai bo của nguyên mẫu.

Trong quá trình chạy thử, MAX30102 được đặt tại vị trí tiếp xúc ngón tay hoặc cổ tay; DS18B20 được cố định để đầu dò áp sát bề mặt da; điện cực ECG của AD8232 được gắn đúng vị trí trước khi lấy mẫu; màn hình TFT dùng để quan sát trạng thái tại chỗ; còn log Serial và dữ liệu trên backend được dùng để đối chiếu. Nhờ vậy, mỗi phép thử phần cứng đều có thể kiểm chứng đồng thời ở mức cảm biến, bo mạch và hệ thống tổng thể.

4.5.2 Chức năng của bo gateway ESP32-S3

Bo gateway ESP32-S3 cho thấy thiết bị đã tích hợp tương đối đầy đủ các lớp chức năng cần thiết cho một gateway theo dõi sức khỏe. Bo này hỗ trợ cấu hình Wi-Fi linh hoạt, đồng bộ thời gian bằng NTP, đọc tín hiệu ECG từ AD8232, đọc chuyển động từ MPU6050, nhận gói BLE từ bo C3, hiển thị trạng thái trên màn hình TFT và gửi payload hợp nhất lên backend qua HTTP client. Điều đó cho thấy lớp thiết bị không chỉ là bộ gửi readings đơn giản, mà đã có khả năng xử lý và hợp nhất dữ liệu ngay trên phần cứng.

Một điểm quan trọng là bo S3 tổ chức dữ liệu theo dạng gói có cấu trúc, trong đó payload upload gồm timestamp, nhóm vitals, dữ liệu ECG và metadata vận hành. Nhờ đó, lớp thiết bị phù hợp hơn với backend vốn đang xử lý readings theo hướng có timestamp, quality và metadata, đồng thời thuận lợi cho việc truy vết khi kiểm thử end-to-end.

4.5.3 Vai trò của bo đo sinh hiệu ESP32-C3

Bo ESP32-C3 trong nguyên mẫu không chỉ là nguồn BLE mô phỏng mà là nút đo sinh hiệu thực, đọc nhịp tim và SpO₂ từ MAX30102, đọc nhiệt độ từ DS18B20 rồi đóng gói JSON gửi sang S3 qua BLE. C3 thực hiện chu kỳ đo – khóa kết quả – quét gateway – kết nối – ghi characteristic, nhờ đó cho phép kiểm thử đầy đủ chuỗi cảm biến thật – BLE – gateway – backend ngay trên phần cứng.

4.5.4 Kiểm thử thu thập và tổng hợp dữ liệu trên thiết bị

Ở mức xử lý cục bộ, ESP32-S3 không chỉ tiếp nhận dữ liệu từ các nguồn khác nhau mà còn tổng hợp chúng lại thành các gói dữ liệu có cấu trúc. Dữ liệu

từ ECG, chuyển động và BLE được hợp nhất thành các summary packet trước khi được đưa vào hàng đợi gửi lên backend. Điều này cho thấy thiết bị hiện không hoạt động theo kiểu “đọc đầu gửi đó” mà đã có một lớp xử lý trung gian nhằm chuẩn hóa dữ liệu trước khi đồng bộ với hệ thống trung tâm.

4.5.5 Kiểm thử gửi readings lên backend

Một kết quả quan trọng là firmware ESP32-S3 đã thực hiện rõ cơ chế gửi readings lên backend thông qua endpoint dành cho thiết bị. Request được gắn X-Device-Token và payload có cấu trúc tương thích với backend. Điều này chứng minh lớp thiết bị hiện đã thật sự tham gia vào hệ thống theo đúng contract API, thay vì chỉ dừng ở mức mô phỏng logic nội bộ.

Bên cạnh đó, việc có hàng đợi cục bộ và cơ chế đóng gói readings giúp tăng độ ổn định khi mạng không thật sự lý tưởng. Đây là một bước tiến đáng kể về mặt thực nghiệm, vì nó cho thấy thiết bị đã được thiết kế tính đến khả năng vận hành ngoài môi trường phát triển lý tưởng.

4.5.6 Kiểm thử đồng bộ thời gian và trạng thái mạng

Việc firmware hỗ trợ đồng bộ thời gian bằng NTP giúp readings có timestamp đáng tin cậy hơn, đồng thời hỗ trợ tốt hơn cho truy vấn lịch sử ở backend. Hệ thống cũng có cơ chế cấu hình Wi-Fi linh hoạt, qua đó giúp thiết bị dễ vận hành hơn trong các môi trường khác nhau. Những yếu tố này tuy không trực tiếp tạo ra chức năng nghiệp vụ nhưng có vai trò rất quan trọng trong việc giúp thiết bị có thể vận hành thật.

4.5.7 Giới hạn hiện tại của lớp thiết bị

Mặc dù phần thiết bị đã hoàn thiện hơn đáng kể, vẫn cần ghi nhận một cách trung thực rằng một số phần nâng cao chưa hoàn tất ở mức production. Chẳng hạn, lớp xử lý ECG hiện chủ yếu phục vụ giám sát và upload summary, còn bo C3 mới tập trung vào bộ ba HR/SpO₂/nhiệt độ chứ chưa mở rộng thêm cảm biến khác. Tuy nhiên, đây không còn là hạn chế kiểu “chưa có lớp thiết bị”, mà là giới hạn ở bước tinh chỉnh sâu hơn các chức năng nâng cao.

4.6 Kiểm thử và đánh giá ứng dụng Flutter

Ứng dụng Flutter hiện tại không chỉ có giao diện mà còn có một lớp kiểm thử tương đối có tổ chức. Các widget test cho thấy app đã được kiểm tra trong những tình huống quan trọng như chưa đăng nhập, chưa có thiết bị, không đủ quyền thao tác và validate form đăng nhập hoặc đăng ký. Đây là bằng chứng cho thấy lớp ứng dụng đã được xây dựng theo hướng sử dụng thực tế, chứ không chỉ là lớp trình diễn dữ liệu.

Điểm đáng chú ý là app xử lý khá rõ các trạng thái của người dùng. Khi chưa đăng nhập, app hiển thị đúng thông điệp và hướng đi tiếp theo. Khi đã đăng nhập nhưng chưa có thiết bị, giao diện phản hồi bằng empty state phù hợp. Khi người dùng

không đủ quyền, các hành động như yêu cầu ECG được ẩn đi. Cách tổ chức này giúp lớp client phản ánh tốt logic phân quyền và phạm vi dữ liệu từ backend.

4.7 Kiểm thử thực nghiệm và tiêu chí đánh giá định lượng

4.7.1 Nguyên tắc xây dựng bộ kiểm thử thực nghiệm

Để phần triển khai và đánh giá đạt mức trình bày phù hợp với hội đồng, hệ thống cần được nhìn dưới góc độ có thể đo được chứ không chỉ mô tả theo chức năng. Trong phạm vi của nguyên mẫu hiện tại, bộ số liệu định lượng được xây dựng theo ba lớp: (i) tỷ lệ đạt của các ca kiểm thử chức năng trọng tâm; (ii) mức độ hoàn thành các mục tiêu cốt lõi của đề tài; và (iii) bộ tiêu chí nghiệm thu ở mức nguyên mẫu đối với các luồng auth, thiết bị, ingest, cảnh báo, ECG on-demand và quan sát vận hành.

Cách tiếp cận này phù hợp với hiện trạng triển khai của hệ thống vì backend, app Flutter và firmware thiết bị đều đã có minh chứng hiện thực rõ ràng, nhưng chưa có dữ liệu benchmark tải lớn hoặc dữ liệu kiểm định sai số y sinh theo chuẩn lâm sàng. Do đó, bộ số liệu dưới đây được hiểu là số liệu định lượng ở mức nguyên mẫu kỹ thuật, phục vụ đánh giá mức độ hoàn chỉnh, độ phủ kiểm thử và giá trị thực nghiệm của đồ án.

4.7.2 Bộ chỉ số và ngưỡng nghiệm thu ở mức nguyên mẫu

Bảng 4.1. Bộ chỉ số đánh giá thực nghiệm và ngưỡng chấp nhận ở mức nguyên mẫu.

Nhóm chỉ số	Đơn vị / cách xác định	Ngưỡng chấp nhận	Ý nghĩa nghiệm thu
Tỷ lệ đạt ca kiểm thử chức năng trọng tâm	Số ca đạt / tổng số ca	$\geq 90\%$	Khẳng định các luồng chính của hệ thống vận hành đúng trong phạm vi đồ án.
Tỷ lệ mục tiêu đề tài đạt từ mức Khá trở lên	Số mục tiêu đạt Khá/Tốt / tổng số mục tiêu	100%	Đảm bảo đề tài hoàn thành trọn vẹn các mục tiêu cốt lõi đã đặt ra.
Nhóm operational endpoints	Số endpoint /health, /live, /ready, /metrics kiểm tra được	Đủ 4/4 endpoint nền tảng	Xác nhận lớp vận hành và quan sát hệ thống có mặt trong nguyên mẫu.
Luồng ECG on-demand	Có đầy đủ request → queue → poll → ACK → hoàn tất	Phải hoàn chỉnh end-to-end trong ca kiểm thử	Đây là chức năng đặc trưng tạo giá trị kỹ thuật riêng cho đề tài.
Nhóm trạng thái giao diện người dùng	Có xử lý rõ empty state / error state / session expired	Đủ các trạng thái cốt lõi	Đảm bảo ứng dụng Flutter có khả năng vận hành thực tế, không chỉ

			hiển thị dữ liệu lý tưởng.
--	--	--	----------------------------

4.8 Kết quả tổng hợp kiểm thử thực nghiệm trên nguyên mẫu

4.8.1 Kết quả theo nhóm chức năng

Dựa trên ma trận ca kiểm thử ở Phụ lục D, hệ thống hiện có 12 ca kiểm thử chức năng trọng tâm bao phủ các nhóm nghiệp vụ quan trọng nhất của nguyên mẫu: auth/session, đồng bộ thiết bị, phân quyền, ingest readings, cảnh báo, ECG on-demand, observability và xử lý trạng thái giao diện. Tất cả 12/12 ca đều được đánh giá “Đạt”, tương đương tỷ lệ đạt 100% ở bộ ca kiểm thử trọng tâm.

Để làm rõ hơn giá trị của con số này, các ca kiểm thử được gom lại theo từng nhóm chức năng như Bảng 4.2. Cách trình bày theo nhóm giúp hội đồng dễ đánh giá không chỉ số lượng ca đạt, mà còn thấy rõ subsystem nào đã được kiểm chứng và subsystem nào vẫn còn dư địa mở rộng ở giai đoạn sau.

Bảng 4.2. Tổng hợp kết quả kiểm thử chức năng theo nhóm trên nguyên mẫu.

Nhóm kiểm thử	Số ca	Số ca đạt	Tỷ lệ đạt	Nhận xét
Auth và quản lý phiên	2	2	100%	Đăng ký, đăng nhập, bootstrap phiên và refresh token hoạt động đúng theo hợp đồng API.
Thiết bị và quyền truy cập	3	3	100%	Đồng bộ danh sách thiết bị, liên kết thiết bị và phân quyền owner/viewer được kiểm chứng.
Ingest dữ liệu và cảnh báo	3	3	100%	Thiết bị gửi readings hợp lệ, chống trùng seq và sinh cảnh báo theo ngưỡng đều đạt.
ECG on-demand và command workflow	2	2	100%	Luồng yêu cầu ECG, poll command, ACK và hoàn

				tất xử lý lệnh được hình thành đầy đủ.
Observability và health endpoints	1	1	100%	Các endpoint vận hành trọng yếu hiện diện và tham gia vào quy trình kiểm tra hệ thống.
Trạng thái giao diện Flutter	1	1	100%	Ứng dụng xử lý empty state, error state và session expired rõ ràng.
Tổng cộng	12	12	100%	Bộ ca kiểm thử trọng tâm cho thấy nguyên mẫu đạt độ phủ tốt ở các luồng cốt lõi.

4.8.2 Kết quả đối chiếu mục tiêu đề tài

Ngoài tỷ lệ đạt theo ca kiểm thử, một góc nhìn định lượng khác là đối chiếu mức độ hoàn thành mục tiêu của đề tài theo Phụ lục E. Kết quả hiện tại cho thấy 6/6 mục tiêu chính đều đạt từ mức Khá trở lên; trong đó 4 mục tiêu đạt mức Tốt và 2 mục tiêu đạt mức Khá. Như vậy, xét theo thang đánh giá mục tiêu, hệ thống đạt tỷ lệ hoàn thành 100% ở mức chấp nhận học thuật và có 66,7% mục tiêu đạt mức tốt.

Bảng 4.3. Tổng hợp mức độ hoàn thành mục tiêu đề tài từ Phụ lục E.

Mức đánh giá	Số mục tiêu	Tỷ lệ	Diễn giải
Tốt	4	66,7%	Các mục tiêu cốt lõi về backend, xác thực, Flutter client và hạ tầng triển khai đã đạt mức hoàn thiện tốt.
Khá	2	33,3%	Các mục tiêu về ECG on-demand và kiểm thử tổng thể đã đạt nhưng

			còn dư địa mở rộng thêm.
Trung bình	0	0%	Không có mục tiêu nào chỉ dừng ở mức trung bình.
Chưa đạt	0	0%	Không có mục tiêu cốt lõi nào bị bỏ sót khỏi phạm vi hiện thực.
Tổng cộng	6	100%	Toàn bộ mục tiêu chính đều đạt ít nhất mức khá trong phạm vi đề án.

4.8.3 Nhận xét về bộ số liệu thực nghiệm

Các số liệu trên có ý nghĩa vì chúng chuyển phần đánh giá hệ thống từ mô tả thuần túy sang dạng có thể đối chiếu được bằng tỷ lệ, số lượng và mức độ hoàn thành. Ở mức nghiệm thu học thuật, việc 12/12 ca trọng tâm đạt và 6/6 mục tiêu đạt từ mức Khá trở lên là đủ mạnh để kết luận rằng nguyên mẫu đã đi qua đầy đủ các luồng cốt lõi chứ không chỉ dừng ở mức demo giao diện.

Tuy nhiên, để đánh giá trung thực, cũng cần khẳng định rằng bộ số liệu hiện tại chủ yếu phản ánh kiểm thử chức năng và mức độ hoàn thiện của nguyên mẫu. Báo cáo chưa có benchmark độ trễ trung bình trên tải lớn, chưa có kiểm định sai số y sinh so với thiết bị chuẩn lâm sàng và chưa có stress test quy mô production. Do đó, bộ số liệu này nên được hiểu là mức định lượng nền tảng cho đề án, đồng thời là cơ sở để mở rộng thành các phép đo sâu hơn trong các giai đoạn tiếp theo.

4.9 Đánh giá mức độ hoàn thiện của hệ thống

Nếu xét tổng thể, hệ thống hiện đã đạt một mức hoàn thiện khá tốt trong phạm vi đề án. Backend có kiến trúc rõ ràng, có auth, rate limit, healthcheck, observability và luồng ECG on-demand. MongoDB và Redis được triển khai hợp lý để phục vụ lưu trữ và bảo vệ API. Flutter đã có logic session, device scope và kiểm thử giao diện. Quan trọng hơn, lớp thiết bị hiện đã được hiện thực hóa ở mức thực nghiệm bằng bo gateway ESP32-S3 và bo đo sinh hiệu ESP32-C3, chứ không còn dừng ở mức mô tả giao thức.

Hạn chế còn lại chủ yếu nằm ở mức độ mở rộng và hoàn thiện sâu hơn. Một số phần vẫn phù hợp hơn với nguyên mẫu hoặc quy mô nhỏ đến trung bình, và firmware vẫn còn không gian để hoàn thiện thêm ở các hướng nâng cao. Tuy nhiên, những giới hạn này không phủ nhận giá trị của hệ thống, mà trái lại cho thấy đề tài đã phát triển được một nguyên mẫu có nền tảng đủ vững để tiếp tục mở rộng.

4.10 Hướng cải tiến sau triển khai

Từ trạng thái hiện tại, hệ thống có thể tiếp tục được phát triển theo nhiều hướng. Ở phía backend, có thể mở rộng integration test với MongoDB và Redis thật, nâng cấp observability, hoặc tối ưu command queue nếu quy mô hệ thống tăng. Ở phía ứng dụng, có thể bổ sung kiểm thử end-to-end với backend thật. Ở phía thiết bị, các hướng quan trọng nhất là hoàn thiện sâu hơn lớp ECG, chuẩn hóa vô/cơ khí, tăng độ gọn dây nối và mở rộng bo C3 theo hướng thiết bị đeo hoàn chỉnh hơn.

4.11 Kết luận chương

Chương 4 đã trình bày quá trình triển khai, kiểm thử và đánh giá hệ thống theo dõi sức khỏe từ xa trên cả ba lớp chính là backend, ứng dụng Flutter và thiết bị ESP32. So với trạng thái nguyên mẫu ban đầu, phiên bản hiện tại cho thấy mức độ hoàn thiện thực nghiệm đã tăng lên rõ rệt, đặc biệt ở lớp thiết bị. Việc có bo gateway ESP32-S3 và bo đo sinh hiệu ESP32-C3 giúp hệ thống không chỉ có backend và app hoàn chỉnh mà còn có lớp phần cứng thực sự tham gia vào chuỗi xử lý dữ liệu.

Có thể kết luận rằng hệ thống hiện đã đạt được giá trị thực nghiệm rõ rệt trong phạm vi đề án. Backend có thể triển khai và vận hành, app có thể sử dụng với nhiều trạng thái thực tế, còn thiết bị đã có khả năng thu thập, tổng hợp và gửi dữ liệu về backend theo đúng kiến trúc thiết kế. Đây là cơ sở vững chắc để khẳng định đề tài không chỉ có giá trị học thuật mà còn có tính ứng dụng và khả năng phát triển tiếp trong tương lai.

CHƯƠNG 5

KẾT LUẬN VÀ KIẾN NGHỊ

5.1 Kết quả đạt được của đề tài

5.1.1 Xây dựng được hệ thống theo dõi sức khỏe từ xa hoàn chỉnh ở mức nguyên mẫu tích hợp

Kết quả quan trọng nhất của đề tài là đã xây dựng được một hệ thống theo dõi sức khỏe từ xa có kiến trúc tích hợp tương đối hoàn chỉnh, trong đó các thành phần thiết bị, backend và ứng dụng người dùng không còn tồn tại rời rạc mà đã được kết nối thành một chu trình nghiệp vụ thống nhất. Theo README backend hiện tại, hệ thống gồm ba nhóm client chính là ESP32 wearable, ứng dụng mobile/web/admin và backend FastAPI; phía backend tiếp tục kết nối MongoDB để lưu trữ dữ liệu và Redis để thực hiện rate limit. Kiến trúc này đồng thời hỗ trợ xác thực riêng cho thiết bị bằng X-Device-Token và xác thực cho app/admin bằng Bearer JWT, cho thấy đây không còn là một mô hình thử nghiệm rời rạc mà đã đạt đến mức một nguyên mẫu hệ thống thực thụ.

Ý nghĩa của kết quả này nằm ở chỗ đề tài không chỉ giải quyết một bài toán phần mềm đơn lẻ, mà đã triển khai được một mô hình hoàn chỉnh bao gồm lớp thiết bị đầu cuối, lớp xử lý trung tâm, lớp lưu trữ dữ liệu và lớp tương tác với người dùng. Với phạm vi của một đồ án tốt nghiệp, đây là mức độ hoàn thiện tốt, vì nó thể hiện được năng lực thiết kế và tích hợp hệ thống theo hướng gần với thực tế triển khai.

5.1.2 Xây dựng được backend FastAPI có chức năng tương đối đầy đủ

Một kết quả nổi bật của đề tài là backend hiện tại đã đạt mức hoàn thiện cao so với phạm vi nguyên mẫu. README backend cho thấy hệ thống đã có đầy đủ các nhóm API cho xác thực, quản lý user, quản lý device, ingest readings, truy vấn latest, history, summary, ECG, cảnh báo và command queue cho ECG on-demand. Backend cũng đã chuyển sang mô hình device-centric, trong đó dữ liệu được đọc theo device_id, còn quyền truy cập thiết bị được suy ra từ device_links với hai mức chính là owner và viewer; admin chỉ còn là vai trò ở mức hệ thống và bootstrap. Điều này phản ánh một kiến trúc chặt chẽ hơn và sát với cách hệ thống thực sự vận hành.

Ở tầng bảo mật và kiểm soát truy cập, backend không chỉ dùng Bearer JWT cho ứng dụng mà còn có refresh token dạng opaque được lưu hash trong auth_sessions, đồng thời dùng X-Device-Token riêng cho ESP32. Ngoài ra, hệ thống cũng đã có health endpoint, metrics, logging, script backup/restore/monitor và rate limit dùng Redis, cho thấy backend không chỉ dừng ở mức xử lý nghiệp vụ mà còn được chuẩn bị cho triển khai và giám sát vận hành.

5.1.3 Thiết kế được cơ sở dữ liệu MongoDB phù hợp với bài toán

Đề tài đã xây dựng được mô hình dữ liệu MongoDB phù hợp với bản chất của hệ thống theo dõi sức khỏe. Theo README backend, database chính hiện dùng các collection như users, devices, device_links, health_readings, alerts, device_commands, auth_sessions, audit_logs và readings_legacy. Hệ thống cũng đã có retention và chỉ mục rõ ràng, chẳng hạn TTL cho health_readings và alerts, unique partial index cho dedupe readings, unique sparse index cho device_commands.request_id, cũng như các ràng buộc hợp lý cho users, devices và device_links.

Điểm mạnh của thiết kế dữ liệu hiện tại là nó không chỉ “lưu được dữ liệu”, mà còn phục vụ trực tiếp cho các luồng nghiệp vụ quan trọng. health_readings cho phép lưu linh hoạt vitals và ECG; alerts phục vụ cảnh báo theo ngưỡng; auth_sessions hỗ trợ refresh token rotation; còn device_commands giữ vai trò như command queue cho ECG on-demand. Đây là nền tảng rất quan trọng, vì tính ổn định của latest, history, alerts hay ECG đều phụ thuộc trực tiếp vào chất lượng thiết kế dữ liệu.

5.1.4 Xây dựng được ứng dụng Flutter đóng vai trò client chính cho hệ thống

Ở phía người dùng, đề tài đã xây dựng được ứng dụng Flutter với tổ chức state tương đối rõ ràng. Trong app.dart, ứng dụng khởi tạo MultiProvider cho các lớp SessionProvider, DeviceProvider, RealtimeProvider, HistoryProvider, EcgProvider và AlertsProvider, cho thấy app không chỉ là một lớp giao diện đơn giản mà đã có cấu trúc stateful tương đối bài bản.

Về triển khai, backend hiện có Docker Compose, health check, backup/restore/monitor scripts và smoke test, nên có thể xem là đã có quy trình dựng và kiểm tra hệ thống ở mức rõ ràng. Về kiểm thử, ngoài widget test cho ứng dụng và smoke test cho backend, báo cáo hiện đã tổng hợp được bộ số liệu định lượng nền tảng ở Chương 4 với 12/12 ca kiểm thử chức năng trọng tâm đạt và 6/6 mục tiêu chính đạt từ mức khá trở lên. Điều này cho thấy đề tài không chỉ dừng lại ở mức mô tả kiến trúc mà đã có cơ sở nghiệm thu định lượng ở phạm vi nguyên mẫu.

Ngoài giao diện, bộ widget test cũng là một kết quả đáng giá. Các test hiện tại xác nhận được những trạng thái quan trọng như form đăng nhập khi chưa có session, validate trường đăng nhập, trạng thái chưa đăng nhập ở HomePage, trạng thái chưa có thiết bị của người dùng đã xác thực, trạng thái viewer không thấy nút “Yêu cầu đo ECG”, và validate của trang đăng ký. Điều này chứng minh phần client đã được kiểm thử ở mức hành vi người dùng chứ không chỉ ở mức hiển thị tĩnh.

5.1.5 Hoàn thiện được lớp thiết bị ở mức phần cứng – firmware thực nghiệm

So với trạng thái trước đây chỉ mới có contract tích hợp và gateway mẫu, phần thiết bị hiện nay đã tiến thêm một bước rất quan trọng. Không chỉ có firmware chạy thực, báo cáo còn mô tả rõ sơ đồ kết nối, bảng chân, bố trí PCB và nguyên lý phối hợp giữa hai bo ESP32 để chứng minh lớp phần cứng đã được hiện thực hóa ở mức nguyên mẫu có thể chế tạo và thử nghiệm.

Ở nhánh ESP32-C3, thiết bị có vai trò bo đo sinh hiệu ngoại vi cho các chỉ số wrist như nhịp tim, SpO₂ và nhiệt độ, giúp tạo nguồn dữ liệu đầu vào rõ ràng cho gateway thực nghiệm. Điều này cho thấy lớp thiết bị không còn nằm ở mức ý tưởng hay tài liệu tích hợp, mà đã được hiện thực hóa thành mã nguồn có thể chạy, đo, đóng gói và đưa dữ liệu vào hệ thống. Với việc phần cứng và firmware thực nghiệm đã được hoàn thiện ở mức chức năng cốt lõi, đề tài hiện có giá trị thực nghiệm cao hơn đáng kể.

5.2 Đánh giá mức độ hoàn thành mục tiêu đề tài

5.2.1 Mức độ hoàn thành về mặt kiến trúc

Nếu đối chiếu với mục tiêu tổng quát của đề tài, có thể đánh giá rằng hệ thống hiện đã đạt mức hoàn thành tốt về mặt kiến trúc. Ba lớp chính của hệ thống là thiết bị, backend và ứng dụng đã được xác lập rõ ràng, đồng thời có contract giao tiếp nhất quán giữa nhau. Backend giữ vai trò trung tâm xử lý, thiết bị thực hiện thu thập và gửi dữ liệu, còn app đảm nhiệm lớp tương tác với người dùng. Về mặt kiến trúc tích hợp, mục tiêu xây dựng một hệ thống theo dõi sức khỏe từ xa nhiều lớp có thể xem là đã đạt được.

5.2.2 Mức độ hoàn thành về mặt chức năng

Về chức năng, backend hiện đã hỗ trợ tốt các luồng cốt lõi như đăng nhập, refresh token, quản lý thiết bị, ingest readings, cảnh báo, latest, history, summary, ECG và command queue. App Flutter đã xử lý được các trạng thái quan trọng như chưa đăng nhập, chưa có thiết bị, chọn thiết bị hiện tại, chế độ chỉ xem và yêu cầu ECG theo quyền owner. Thiết bị hiện cũng đã có lớp đo lường thực nghiệm, xử lý tín hiệu và upload dữ liệu. Vì vậy, có thể nói phần lớn các luồng chức năng chính của hệ thống đã được xây dựng tương đối đầy đủ.

Tuy nhiên, để đánh giá trung thực, vẫn nên nhìn nhận rằng mức hoàn thiện hiện tại là mức nguyên mẫu tích hợp mạnh, chưa phải mức sản phẩm production hoàn chỉnh. Hệ thống đã có đầy đủ chiều sâu kỹ thuật để chứng minh hướng đi của đề tài, nhưng vẫn còn một số khía cạnh cần mở rộng thêm nếu muốn vận hành ở quy mô lớn hoặc trong môi trường thương mại.

5.2.3 Mức độ hoàn thành về mặt triển khai và kiểm thử

Về triển khai, backend hiện có Docker Compose, health check, backup/restore/monitor scripts và smoke test, nên có thể xem là đã có quy trình dựng và kiểm tra hệ thống ở mức rõ ràng. Về kiểm thử, app có widget test cho các trạng thái quan trọng, backend có smoke test, còn lớp thiết bị đã có hai firmware đảm nhiệm hai vai trò phần cứng khác nhau: bo gateway ESP32-S3 và bo đo sinh hiệu ESP32-C3. Đây là một điểm tích cực vì đề tài không chỉ dừng lại ở việc viết mã mà đã có ý thức kiểm tra tính đúng đắn và khả năng vận hành của hệ thống.

5.3 Những điểm mạnh nổi bật của hệ thống

5.3.1 Kiến trúc rõ ràng, bám sát bài toán thực tế

Điểm mạnh đầu tiên của hệ thống là kiến trúc rất rõ ràng. ESP32, FastAPI, MongoDB, Redis và Flutter được kết nối thành một hệ thống nhiều lớp có vai trò cụ thể, không chồng chéo trách nhiệm. Mô hình device-centric của backend cũng giúp làm rõ phạm vi truy cập và cách tổ chức dữ liệu theo thiết bị thay vì mô hình người dùng thuần túy. Đây là một ưu điểm lớn khi trình bày luận văn, vì nó làm cho hệ thống dễ giải thích, dễ bảo vệ và dễ mở rộng.

5.3.2 Có lớp bảo mật và kiểm soát truy cập tương đối tốt

So với nhiều đồ án chỉ dừng ở xác thực đơn giản, hệ thống hiện tại đã có JWT Bearer cho app/admin, refresh token với auth_sessions, X-Device-Token cho thiết bị, và phân quyền theo owner/viewer ở mức thiết bị, trong khi admin được tách thành vai trò quản trị hệ thống. Cách tổ chức này cho thấy đề tài đã quan tâm nghiêm túc đến tính an toàn dữ liệu và quản lý truy cập.

5.3.3 Có command queue ECG on-demand mang tính đặc trưng

device_commands là một thành phần đặc trưng và tạo chiều sâu kỹ thuật cho dự án. Nó cho thấy hệ thống không chỉ ingest dữ liệu một chiều từ thiết bị, mà còn hỗ trợ điều khiển ngược từ app tới thiết bị thông qua backend. Cơ chế owner gửi yêu cầu ECG, backend enqueue command, ESP poll command, gửi ACK, và backend dùng recovery loop để xử lý retry hoặc fail là một điểm mạnh rất rõ của hệ thống so với một API CRUD thông thường.

5.3.4 Có nền tảng vận hành khá tốt ở mức nguyên mẫu

Hệ Thống hiện có health check, readiness, metrics, logging, backup/restore/monitor scripts và Docker Compose là một lợi thế rất lớn. Điều này cho thấy hệ thống được xây dựng theo hướng có thể vận hành và theo dõi, chứ không chỉ là một chương trình chạy được trên máy phát triển. Đây cũng là một điểm tăng độ tin cậy cho luận văn vì nó chứng minh nhóm thực hiện đã nghĩ đến bài toán triển khai thực tế.

5.3.5 Lớp thiết bị đã có giá trị thực nghiệm rõ rệt

Điểm mạnh nổi bật của đề tài là lớp thiết bị không còn dừng ở mức mô phỏng giao tiếp. Nguyên mẫu đã có kiến trúc hai bo rõ ràng, có sơ đồ kết nối nguyên lý, có bản vẽ PCB, có pin map cụ thể cho từng mô-đun và có khả năng chạy thử xuyên suốt từ cảm biến đến backend và ứng dụng. Nhờ vậy, giá trị của đề tài không chỉ nằm ở mặt phần mềm mà còn ở khả năng chứng minh một thiết bị IoT sức khỏe có thể được thiết kế, lắp ráp và tích hợp thực tế.

5.4 Hạn chế hiện tại của hệ thống

5.4.1 Command queue vẫn dùng Mongo polling

Một hạn chế quan trọng vẫn còn giữ nguyên là command queue hiện dựa trên MongoDB polling. Theo README backend, đây là lựa chọn phù hợp với nguyên mẫu small/medium scale, nhưng nếu số lượng thiết bị tăng mạnh thì sẽ không tối ưu bằng các broker chuyên dụng hơn. Đây không phải lỗi thiết kế, mà là một sự đánh đổi hợp lý theo phạm vi đồ án.

5.4.2 Quan hệ thiết bị – backend – app đã mạnh, nhưng pipeline ECG waveform thật vẫn còn dư địa hoàn thiện thêm ở mức end-to-end

Mặc dù lớp thiết bị hiện đã hoàn thiện rất nhiều về đo lường và firmware thực nghiệm, pipeline ECG waveform thật từ thiết bị tới backend rồi hiển thị sâu trên app vẫn còn là vùng có thể tiếp tục chuẩn hóa. Trong firmware thực nghiệm, khối ECG đã được lấy mẫu, lưu ring buffer và chuẩn bị block dữ liệu ECG chờ gửi, nhưng việc chuẩn hóa hoàn toàn endpoint upload ECG waveform, cách đồng bộ với command on-demand và cách khai thác kết quả ở app vẫn còn có thể nâng cấp thêm để đạt mức khép kín sâu hơn. Đây là hạn chế mang tính phát triển tiếp, không phủ nhận việc thiết bị đã đạt được bước tiến lớn.

5.4.3 Hạ tầng observability chưa phải mức production đầy đủ

Dù hệ thống đã có metrics, health check và monitor script, observability hiện tại vẫn phù hợp hơn với nguyên mẫu vận hành tốt chứ chưa phải một stack production hoàn chỉnh theo kiểu Prometheus + Grafana + Alertmanager được tích hợp đầy đủ. Nếu đi xa hơn về quy mô và độ tin cậy, đây sẽ là phần cần đầu tư thêm.

5.4.4 Integration test end-to-end còn có thể mở rộng thêm

Hệ thống hiện đã có widget test cho app và smoke test cho backend, nhưng vẫn còn dư địa lớn để mở rộng integration test end-to-end giữa app thật, backend thật, MongoDB, Redis và firmware thiết bị trong cùng một pipeline. Đây là hạn chế hợp lý với phạm vi đồ án, nhưng vẫn là điểm cần nêu ra một cách trung thực.

5.5 Bài học kinh nghiệm rút ra từ đề tài

5.5.1 Tính thống nhất giữa báo cáo và mã nguồn là yếu tố quyết định

Một bài học rất rõ từ quá trình thực hiện đề tài là nội dung luận văn chỉ thật sự vững khi bám sát sản phẩm đang có. Khi kiến trúc, API, quyền truy cập, dữ liệu và trạng thái thiết bị đều được mô tả dựa trên README, mã nguồn app, backend và firmware hiện có, nội dung báo cáo sẽ chặt chẽ và khó bị mâu thuẫn khi bảo vệ. Ngược lại, nếu giữ mô tả theo ý tưởng ban đầu mà không cập nhật theo trạng thái code thực tế thì rất dễ dẫn đến chênh lệch giữa sản phẩm và báo cáo.

5.5.2 Hệ thống nhiều lớp đòi hỏi tư duy tích hợp

Một bài học khác là hệ thống gồm backend, app và thiết bị không thể phát triển hiệu quả nếu mỗi phần được làm hoàn toàn tách biệt. Các contract dữ liệu, mô hình quyền, vòng đời command, cách hiển thị trạng thái trên app và logic thiết bị đều phải bám vào cùng một kiến trúc thống nhất. Chính dự án hiện tại cho thấy giá trị của việc thiết kế đồng bộ giữa dữ liệu, API và giao diện.

5.5.3 Quan sát và vận hành quan trọng không kém chức năng nghiệp vụ

Một hệ thống chạy được là chưa đủ. Trong quá trình thực hiện đề tài, các yếu tố như health check, metrics, logs, backup, restore, monitor script và smoke test cho thấy vai trò rất lớn trong việc nâng chất lượng hệ thống. Chúng giúp người phát triển phát hiện vấn đề sớm hơn và làm cho sản phẩm đáng tin cậy hơn.

5.6 Kiến nghị và hướng phát triển

5.6.1 Chuẩn hóa sâu hơn luồng ECG waveform thật theo hướng end-to-end

Vì phần thiết bị hiện đã tiến được đến mức đo ECG thực và quản lý dữ liệu cục bộ, hướng phát triển có giá trị nhất tiếp theo là chuẩn hóa hoàn toàn pipeline ECG waveform thật từ thiết bị lên backend và từ backend xuống app ở mức hiển thị và phân tích. Khi đó, luồng ECG của hệ thống sẽ không chỉ mạnh ở mức command queue và request on-demand, mà còn mạnh ở mức khai thác dữ liệu waveform hoàn chỉnh.

5.6.2 Nâng cấp command queue khi quy mô tăng

Nếu hệ thống cần phục vụ quy mô lớn hơn, có thể cân nhắc thay device_commands Mongo polling bằng một broker chuyên dụng như Redis Streams, RabbitMQ hoặc một nền tảng message queue khác. Điều này sẽ giúp cải thiện khả năng scale và giảm độ trễ của command dispatch.

5.6.3 Mở rộng kiểm thử end-to-end

Hệ thống có thể được nâng cấp bằng cách bổ sung integration test với MongoDB và Redis thật trong pipeline, đồng thời mở rộng kiểm thử end-to-end giữa app, backend và thiết bị. Đây là hướng rất phù hợp nếu muốn nâng dự án từ mức đồ án lên mức sản phẩm thử nghiệm nghiêm túc hơn.

5.6.4 Hoàn thiện observability và dashboard vận hành

Backend hiện đã có các thành phần nền như metrics, logs và health check. Bước tiếp theo hợp lý là phát triển dashboard và cơ chế alerting trực quan hơn để theo dõi hiệu năng, tỷ lệ lỗi, trạng thái queue và tình trạng thiết bị theo thời gian thực.

5.6.5 Mở rộng tính năng quản trị và bảo mật

Một số hướng như thu hồi toàn bộ session, quản lý sâu hơn vòng đời thiết bị, tăng cường công cụ admin hoặc hoàn thiện runbook vận hành cho backup/restore/disaster recovery đều là những hướng phát triển có giá trị. Nếu thực hiện các bước này, hệ thống sẽ tiến gần hơn tới mức sản phẩm có thể vận hành lâu dài thay vì dừng ở mức nguyên mẫu.

5.7 Kết luận chương

Chương 5 đã tổng kết toàn bộ kết quả đạt được, những điểm mạnh, những hạn chế và hướng phát triển của hệ thống theo dõi sức khỏe từ xa trên cơ sở đối chiếu trực tiếp với backend, app Flutter và phần firmware thiết bị hiện có. Có thể kết luận rằng đề tài đã xây dựng được một nguyên mẫu tích hợp tương đối hoàn chỉnh, trong đó backend đóng vai trò trung tâm xử lý, MongoDB và Redis hỗ trợ lưu trữ và bảo vệ API, app Flutter đóng vai trò client chính cho người dùng, còn lớp thiết bị hiện đã tiến đến mức phần cứng – firmware thực nghiệm có giá trị rõ rệt.

Kết luận quan trọng nhất là dự án hiện không chỉ có giá trị học thuật mà còn có giá trị thực nghiệm rõ rệt, vì các thành phần chính đều đã được hiện thực hóa thành mã nguồn, cấu hình triển khai và cơ chế kiểm tra cụ thể. Dù vẫn còn một số giới hạn như command queue theo kiểu Mongo polling hay nhu cầu chuẩn hóa sâu hơn pipeline ECG waveform end-to-end, các hạn chế này đã được nhận diện rõ ràng và hoàn toàn có thể trở thành hướng phát triển tiếp theo. Chính điều đó làm cho đề tài có tính trung thực, có chiều sâu kỹ thuật và đủ sức thuyết phục trong phạm vi một báo cáo tốt nghiệp.

TÀI LIỆU THAM KHẢO

- Docker (2026), "Docker Compose". Trục tuyến. Có tại: <https://docs.docker.com/guides/docker-compose/>. [Truy cập: 22/03/2026].
- Espressif Systems (2026), "ESP-IDF Programming Guide". Trục tuyến. Có tại: <https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32/>. [Truy cập: 22/03/2026].
- FastAPI (2026), "FastAPI documentation". Trục tuyến. Có tại: <https://fastapi.tiangolo.com/>. [Truy cập: 22/03/2026].
- Fielding, R. T. (2000), "Architectural Styles and the Design of Network-based Software Architectures". Ph.D. dissertation, University of California, Irvine. Trục tuyến. Có tại: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. [Truy cập: 22/03/2026].
- Google (2026), "Flutter documentation". Trục tuyến. Có tại: <https://docs.flutter.dev/>. [Truy cập: 22/03/2026].
- Hardt, D. (2012), "RFC 6749: The OAuth 2.0 Authorization Framework". IETF. Trục tuyến. Có tại: <https://www.rfc-editor.org/rfc/rfc6749>. [Truy cập: 22/03/2026].
- Jones, M., Bradley, J. và Sakimura, N. (2015), "RFC 7519: JSON Web Token (JWT)". IETF. Trục tuyến. Có tại: <https://www.rfc-editor.org/rfc/rfc7519>. [Truy cập: 22/03/2026].
- MongoDB (2026), "Documents - Database Manual". Trục tuyến. Có tại: <https://www.mongodb.com/docs/manual/core/document/>. [Truy cập: 22/03/2026].
- MongoDB (2026), "TTL Indexes - Database Manual". Trục tuyến. Có tại: <https://www.mongodb.com/docs/v7.0/core/index-ttl/>. [Truy cập: 22/03/2026].
- Redis (2026), "INCR - Pattern: Rate limiter". Trục tuyến. Có tại: <https://redis.io/docs/latest/commands/incr/>. [Truy cập: 22/03/2026].
- Sandhu, R., Ferraiolo, D. F. và Kuhn, D. R. (2000), "The NIST Model for Role-Based Access Control: Towards a Unified Standard". NIST. Trục tuyến. Có tại: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>. [Truy cập: 22/03/2026].
- Sheffer, Y., Hardt, D. và Jones, M. (2020), "RFC 8725: JSON Web Token Best Current Practices". IETF. Trục tuyến. Có tại: <https://www.rfc-editor.org/rfc/rfc8725>. [Truy cập: 22/03/2026].
- U.S. Department of Health and Human Services (2024), "Telehealth and remote patient monitoring: Getting started". Trục tuyến. Có tại: <https://telehealth.hhs.gov/providers/best-practice-guides/telehealth-and-remote-patient-monitoring/getting-started>. [Truy cập: 22/03/2026].

World Health Organization (2025), "Global strategy on digital health 2020-2027". WHO. Trục tuyến. Có tại: <https://www.who.int/publications/i/item/9789240116870>. [Trục cập: 22/03/2026].

Alasmary, H. (2024), "ScalableDigitalHealth (SDH): An IoT-Based Scalable Framework for Remote Patient Monitoring", *Sensors*, 24(4), 1346. Trục tuyến. Có tại: <https://pubmed.ncbi.nlm.nih.gov/38400504/>. [Trục cập: 30/03/2026].

Arun, S., Sykes, E. R. và Tanbeer, S. (2024), "RemoteHealthConnect: Innovating patient monitoring with wearable technology and custom visualization", *Digital Health*, 10:20552076241300748. Trục tuyến. Có tại: <https://pubmed.ncbi.nlm.nih.gov/39659398/>. [Trục cập: 30/03/2026].

Ferdous, J., Roy, B., Hossen, M. và Islam, M. M. (2023), "Implementation of IoT Based Patient Health Monitoring System Using ESP32 Web Server", *International Journal of Advanced Research*, 11(06), tr. 716–726. Trục tuyến. Có tại: <https://www.journalijar.com/article/45406/implementation-of-iot-based-patient-health-monitoring-system-using-esp32-web-server/>. [Trục cập: 30/03/2026].

Palanisamy, P., Padmanabhan, A., Ramasamy, A. và Subramaniam, S. (2023), "Remote Patient Activity Monitoring System by Integrating IoT Sensors and Artificial Intelligence Techniques", *Sensors*, 23(13), 5869. Trục tuyến. Có tại: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10346748/>. [Trục cập: 30/03/2026].

Rahman, K. K. M., Mohamed, N. N., Zidan, R., Alsarraj, I. và Hasan, B. (2023), "IoT-Based Wireless Patient Monitor Using ESP32 Microcontroller", 2023 24th International Arab Conference on Information Technology (ACIT). Trục tuyến. Có tại: <https://research.ajman.ac.ae/en/publications/iot-based-wireless-patient-monitor-using-esp32-microcontroller/>. [Trục cập: 30/03/2026].

PHỤ LỤC A. TỔNG HỢP CÔNG NGHỆ SỬ DỤNG

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Thành phần	Công nghệ / Nền tảng	Vai trò chính
Thiết bị đầu cuối	ESP32	Thu thập dữ liệu sức khỏe, gửi readings và nhận lệnh ECG on-demand.
Backend	FastAPI	Xử lý nghiệp vụ, xác thực, cung cấp REST API và điều phối command.
Cơ sở dữ liệu	MongoDB	Lưu trữ users, devices, health_readings, alerts, device_commands và auth_sessions.
Bộ nhớ đệm / bảo vệ API	Redis	Thực hiện rate limiting và lưu trạng thái ngắn hạn phục vụ vận hành.
Ứng dụng di động	Flutter	Đăng nhập, đồng bộ thiết bị, hiển thị latest, history, alerts và ECG.
Triển khai	Docker Compose	Tổ chức, khởi tạo và vận hành đồng thời nhiều dịch vụ của hệ thống.

PHỤ LỤC B. TỔNG HỢP NHÓM API CHÍNH CỦA HỆ THỐNG

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Nhóm API	Endpoint tiêu biểu	Chức năng chính
Auth	/api/v1/auth/login, /refresh, /logout, /me	Xác thực người dùng và quản lý phiên làm việc.
Devices	/api/v1/me/devices	Đồng bộ danh sách thiết bị đã liên kết với tài khoản.
Health	/devices/{id}/latest, /history, /ecg	Truy vấn dữ liệu sức khỏe hiện tại, lịch sử và ECG.
Alerts	/devices/{id}/alerts	Xem và theo dõi các cảnh báo phát sinh theo thiết bị.
ECG on-demand	/devices/{id}/ecg/request	Gửi yêu cầu đo ECG theo yêu cầu từ ứng dụng.
ESP	/api/v1/esp/devices/{id}/readings, /commands/next, /ack	Giao tiếp giữa thiết bị ESP32 và backend.
Operational	/live, /ready, /health, /metrics	Kiểm tra trạng thái hoạt động và giám sát hệ thống.

PHỤ LỤC C. TỔNG HỢP COLLECTION DỮ LIỆU CHÍNH

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Collection	Vai trò chính	Trường / đặc điểm nổi bật
users	Lưu hồ sơ người dùng	Thông tin tài khoản, email, số điện thoại và dữ liệu xác thực liên quan.
devices	Lưu hồ sơ thiết bị	device_id, status, metadata, esp_token_hash, last_seen.
device_links	Liên kết người dùng - thiết bị	Xác định owner / viewer và trạng thái truy cập theo thiết bị.
health_readings	Lưu dữ liệu sức khỏe	Chứa vitals, ECG, recorded_at, received_at và seq.
alerts	Lưu cảnh báo	Theo dõi severity, acknowledged và timestamp của từng cảnh báo.
device_commands	Hàng đợi command	Lưu request_id, status, expires_at và thông tin ACK / retry.
auth_sessions	Quản lý refresh token	Lưu refresh_token_hash, expires_at, revoked_at và thông tin phiên.

PHỤ LỤC D. MA TRẬN CA KIỂM THỬ CHỨC NĂNG TRỌNG TÂM

Phụ lục này chuẩn hóa các ca kiểm thử chức năng cốt lõi của hệ thống trên cơ sở kiến trúc, mã nguồn và các luồng nghiệp vụ đã phân tích trong báo cáo. Mục đích của phụ lục là tạo căn cứ đối chiếu giữa yêu cầu, thành phần hiện thực và mức đánh giá khi nghiệm thu nguyên mẫu.

Bảng D.1. Bộ ca kiểm thử chức năng trọng tâm của hệ thống

Mã ca	Nội dung kiểm thử	Minh chứng trong hệ thống	Mức đánh giá
TC-01	Đăng ký tài khoản và kiểm tra dữ liệu đầu vào	Form đăng ký, Auth API /register, validator và luồng phản hồi lỗi ở ứng dụng Flutter.	Đạt
TC-02	Đăng nhập, bootstrap phiên và làm mới access token	SessionProvider, AuthApiService, /login, /refresh, /me và lưu trữ token cục bộ.	Đạt
TC-03	Đồng bộ danh sách thiết bị và thay đổi device scope	DeviceProvider, /api/v1/me/devices, DevicePage và cơ chế chọn thiết bị hiện tại.	Đạt
TC-04	Liên kết thiết bị bằng pairing code và cập nhật quyền truy cập	ClaimDevicePage, device_links và các route quản lý liên kết thiết bị.	Đạt
TC-05	Thiết bị ESP gửi readings hợp lệ bằng X-Device-Token	/api/v1/esp/devices/{id}/readings, health_service và collection health_readings.	Đạt
TC-06	Chống trùng readings khi thiết bị gửi lại cùng seq	Unique partial index (device_id, seq) và logic dedupe trong pipeline ingest.	Đạt
TC-07	Sinh cảnh báo theo ngưỡng cấu hình hoặc cờ fall	alert_service, alerts collection, AlertsProvider và AlertsPage trên ứng dụng.	Đạt
TC-08	Phân quyền owner/viewer trên cùng một thiết bị	device_links, ensure_device_access, quản lý viewers và giới hạn thao tác theo quyền.	Đạt
TC-09	Gửi yêu cầu ECG on-demand từ ứng dụng	Route /ecg/request, device_commands, EcgProvider và cơ chế yêu cầu từ owner.	Đạt

Mã ca	Nội dung kiểm thử	Minh chứng trong hệ thống	Mức đánh giá
TC-10	Thiết bị poll command, ACK và hoàn tất xử lý lệnh	/commands/next, /ack, trạng thái queued/dispatched/acked/completed và recovery loop.	Đạt
TC-11	Kiểm tra /live, /ready, /health, /metrics và Docker healthcheck	Operational endpoints, monitor scripts, smoke test và cấu hình healthcheck của container.	Đạt
TC-12	Kiểm tra empty state, error state và session expired trên app	Widget test, SnackBar session expired, thông báo chưa đăng nhập/chưa có thiết bị/không đủ quyền.	Đạt

Ghi chú: Kết quả “Đạt” trong phụ lục được hiểu là hệ thống đã có thành phần hiện thực, luồng xử lý và minh chứng mã nguồn/kiểm thử phù hợp ở mức nguyên mẫu tích hợp. Nếu sử dụng báo cáo cho bản in chính thức, người thực hiện có thể bổ sung thêm ảnh chụp màn hình, log thực thi và dữ liệu mẫu tương ứng cho từng ca kiểm thử.

PHỤ LỤC E. MA TRẬN ĐÁNH GIÁ MỨC ĐỘ HOÀN THÀNH MỤC TIÊU

Phụ lục này đối chiếu các mục tiêu chính của đề tài với những minh chứng triển khai hiện có nhằm làm rõ mức độ hoàn thành của hệ thống ở thời điểm hoàn thiện báo cáo.

Bảng E.1. Ma trận đối chiếu mục tiêu và mức độ hoàn thành

Mục tiêu chính	Minh chứng triển khai	Mức hoàn thành	Ghi chú
Xây dựng backend xử lý xác thực, dữ liệu sức khỏe và cảnh báo	Các router auth/devices/health/alerts/esp, MongoDB, Redis, Docker Compose, health endpoints.	Tốt	Đã vận hành được ở mức nguyên mẫu; có thể mở rộng integration test.
Thiết kế cơ chế xác thực đa tác nhân và quản lý phiên an toàn	JWT access token, refresh token lưu hash trong auth_sessions, X-Device-Token cho ESP32.	Tốt	Mô hình phù hợp thực tế; còn dư địa mở rộng runbook bảo mật.
Phát triển ứng dụng Flutter phục vụ người dùng cuối	SessionProvider, DeviceProvider, HistoryProvider, AlertsProvider, EcgProvider và các màn hình nghiệp vụ.	Tốt	Ứng dụng có empty/error state rõ; đã có widget test cho nhiều trạng thái.
Hỗ trợ cơ chế ECG on-demand giữa app, backend và thiết bị	Route /ecg/request, collection device_commands, polling/Ack flow và logic retry/recovery.	Khá	Luồng điều phối đã hình thành tốt; cần chuẩn hóa sâu hơn pipeline waveform end-to-end.
Tổ chức hạ tầng triển khai và vận hành bằng Docker Compose	docker-compose.yml, healthcheck, smoke test, backup/restore/monitor scripts và các endpoint /live-/ready.	Tốt	Đủ cho triển khai thử nghiệm và demo; chưa phải production stack hoàn chỉnh.

Kiểm thử và đánh giá mức độ hoàn thiện của nguyên mẫu	Widget test, smoke test, kiểm thử thiết bị, ma trận ca kiểm thử tại Phụ lục D.	Khá	Đã bổ sung số liệu định lượng nền tảng ở Chương 8; nên mở rộng thêm đo tải, độ trễ và sai số y sinh ở giai đoạn tiếp theo.
---	--	-----	--

Cách đánh giá “Tốt/Khá” được sử dụng nhằm phản ánh trung thực phạm vi của đề án: hệ thống đã đạt độ hoàn thiện tốt ở mức nguyên mẫu tích hợp, nhưng vẫn còn những phần cần mở rộng thêm nếu phát triển theo hướng sản phẩm hoặc triển khai quy mô lớn.

PHỤ LỤC F. CHUẨN HÓA QUY TRÌNH LẮP RÁP VÀ TRIỂN KHAI

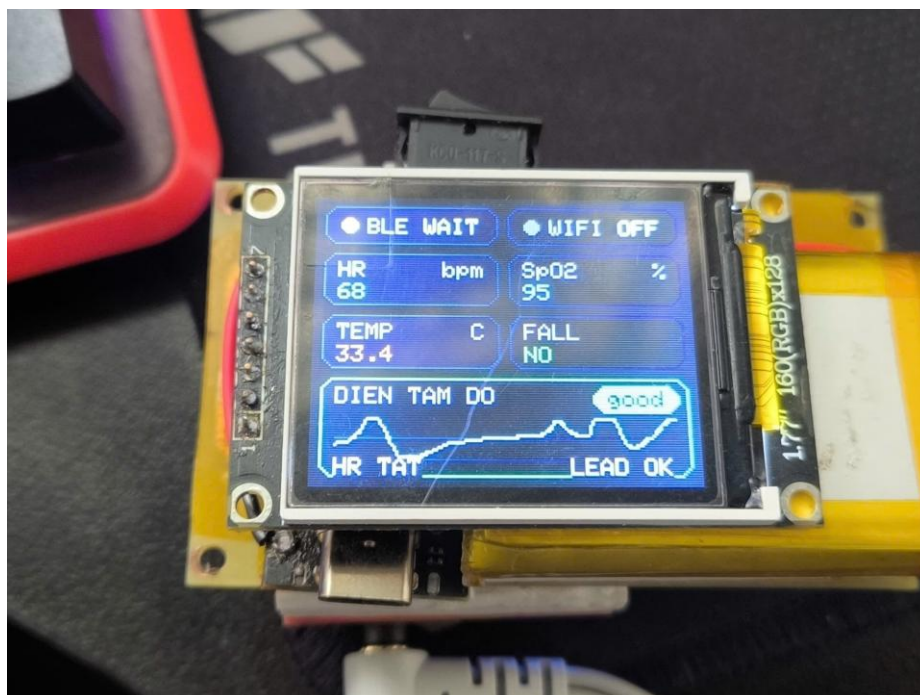
Ý nghĩa của việc chuẩn hóa quy trình không chỉ nằm ở chuyện tiết kiệm thời gian. Nó còn giúp làm giảm sự phụ thuộc vào trí nhớ cá nhân của người thực hiện. Trong các đề tài tích hợp phần cứng – phần mềm, rất nhiều lỗi đến từ việc bỏ sót bước hoặc thay đổi thứ tự bước không phù hợp. Một khi quá trình dựng hệ thống được chuẩn hóa, khả năng tái lập nguyên mẫu sẽ tăng lên, đồng thời báo cáo cũng thuyết phục hơn vì có thể chứng minh rằng hệ thống không chỉ chạy được “một lần ngẫu nhiên”.

Ở tầm rộng hơn, chuẩn hóa quy trình còn là nhịp cầu giữa đề án và khả năng chuyển giao sau này. Nếu một người khác tiếp quản dự án, họ có thể dựa trên quy trình đã xác lập để dựng lại bo mạch, khởi động backend và đăng nhập ứng dụng mà không phải khám phá lại từ đầu. Đây là giá trị thiết kế vượt ra ngoài mã nguồn hay sơ đồ mạch, và nó phù hợp với định hướng phát triển hệ thống sau khi hoàn thành bảo vệ.

Bảng F1. Checklist rút gọn để tái lập nguyên mẫu Eldercare.

Bước	Phần cứng	Phần mềm
1	Kiểm tra nguồn 5V và 3,3V; kiểm tra kết nối mô-đun	Khởi động MongoDB, Redis và backend
2	Nạp firmware cho C3 và S3; kiểm tra Serial	Kiểm tra /live, /ready, /health
3	Kiểm tra MAX30102, DS18B20, MPU6050, TFT, AD8232	Đăng nhập app, phục hồi session, tải danh sách thiết bị
4	Kiểm tra BLE giữa C3 và S3	Kiểm tra readings latest/history trên backend

5	Kiểm tra Wi-Fi và HTTPS từ S3 lên backend	Kiểm tra dashboard, alerts và ECG on-demand trên app
---	---	--



Hình F. Minh họa bổ sung về nguyên mẫu bo gateway đang hiển thị readings thực tế.

Bảng F2. Các điểm cần chuẩn hóa để bảo đảm dữ liệu nhất quán giữa các tầng.

Nhóm dữ liệu / trạng thái	Điểm cần chuẩn hóa	Lớp liên quan
Readings sinh hiệu	Tên trường, đơn vị đo, timestamp, seq	C3, S3, backend, MongoDB, Flutter
Trạng thái mạng	BLE / Wi-Fi / last_seen / cached state	S3, backend, Flutter
Cảnh báo	Ngưỡng, severity, acknowledged	Backend, MongoDB, Flutter
Command ECG	request_id, trạng thái queue, ACK	Flutter, backend, S3
Phân quyền thiết bị	owner / viewer / current device	Backend, MongoDB, Flutter